

Code Reference

August 5, 2026

Contents

0.1	Data Structure	3
0.1.1	DynamicPersistentSegmentTree	3
0.1.2	DynamicSegmentTree	4
0.1.3	FenwickTree	5
0.1.4	Lazy-PST	7
0.1.5	LazySegmentTree	9
0.1.6	Mo's	11
0.1.7	OrderedSTLS	12
0.1.8	SegmentTree	13
0.1.9	SGTBeats	14
0.1.10	Sparse	18
0.1.11	SqrtDecomposition	20
0.1.12	Treap	21
0.1.13	WaveletMatrix	22
0.1.14	XorSegmentTree	23
0.2	DP optimization	24
0.2.1	CHT	24
0.2.2	SOSDP	25
0.2.3	Subset-Sum	26
0.3	Geometry	27
0.3.1	Circles	27
0.3.2	LiChao	28
0.3.3	Lines	29
0.3.4	Notes	31
0.3.5	Polygons	33
0.3.6	totalArea	36
0.4	Graphs	38
0.4.1	BCCTree	38
0.4.2	Blossom	39
0.4.3	DominatorTree	40
0.4.4	DSU	41
0.4.5	GraphArray	43
0.4.6	Johnson	45
0.4.7	Matching	46
0.4.8	MaxFlow	48
0.4.9	MinCostMaxFlow	50
0.4.10	Notes	51
0.4.11	TarjanSCC	52
0.4.12	TreeIsomorphism	53
0.4.13	TwoSat	54
0.5	HelpFulMethods	55
0.6	Math	57
0.6.1	Berlekamp-Massey	57
0.6.2	BigNumber	59
0.6.3	BinaryTrie	60
0.6.4	combinatorics	61
0.6.5	Convolutions	63
0.6.6	Equation	64
0.6.7	FFT	66
0.6.8	FWHT	68

0.6.9	Gauss	69
0.6.10	Generating-Functions	71
0.6.11	HelpfulFunctions	72
0.6.12	Lehmer	75
0.6.13	Matrices	76
0.6.14	Notes	78
0.6.15	NTT	79
0.6.16	polynomial	81
0.6.17	Sieve	83
0.6.18	XorBasis	85
0.7	Strings	87
0.7.1	Corasick	87
0.7.2	DynamicSuffixArray	89
0.7.3	Eertree	91
0.7.4	KMP	93
0.7.5	LargestLexSubstring	94
0.7.6	Manacher	95
0.7.7	RollingHash	96
0.7.8	SuffixArray	97
0.7.9	SuffixAutomaton	100
0.7.10	Z_Algo	102
0.8	Trees	103
0.8.1	Centroid	103
0.8.2	HLD	104
0.8.3	LCA	106
0.8.4	Link-Cut Tree	108

0.1 Data Structure

0.1.1 DynamicPersistentSegmentTree

```
1  segment tree with persistence, allowing multiple versions of the array while supporting point updates and range
   ↪ queries on huge index ranges.
2  template<class info>
3  class dynamicPersistentSegmentTree {
4      struct node {
5          int l{}, r{};
6          info v;
7      };
8      vector<node> tr;
9      int n;
10     static info defaultVal;
11
12     int clone(int x) {
13         tr.emplace_back(tr[x]);
14         return int(tr.size()) - 1;
15     }
16     info get(int x, int lx, int rx, int l, int r) {
17         if(!x || lx > r || l > rx) return defaultVal;
18         if(lx >= l && rx <= r) return tr[x].v;
19         int m = (lx + rx) >> 1;
20         return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
21     }
22     int set(int x, int lx, int rx, int i, info val) {
23         if (i < lx || i > rx) return x;
24         x = clone(x);
25         if(lx == rx) return tr[x].v = val, x;
26         int m = (lx + rx) >> 1;
27         tr[x].l = set(tr[x].l, lx, m, i, val), tr[x].r = set(tr[x].r, m + 1, rx, i, val);
28         return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
29     }
30 public:
31     explicit dynamicPersistentSegmentTree(int _n = 1'000'000'000, int expectedOps = 1) : n(_n), tr(1) {
32         ↪ tr.reserve(expectedOps * __lg(n) * 2); }
33
34     int set(int root, int i, info v) {
35         return set(root, 0, n, i, v);
36     }
37     info get(int root, int l, int r) {
38         return get(root, 0, n, l, r);
39     }
40 };
41 struct info {
42     int64_t sum;
43     info(int64_t x) {
44         sum = x;
45     }
46     info() {
47         sum = 0;
48     }
49     friend info operator+(const info &l, const info &r) {
50         info res;
51         res.sum = l.sum + r.sum;
52         return res;
53     }
54 };
55 template<> info dynamicPersistentSegmentTree<info>::defaultVal = info();
```

0.1.2 DynamicSegmentTree

Segment tree that dynamically allocates nodes to handle very large index ranges (e.g., up to `1e9`) while supporting
↪ point updates and range queries.

```
template<class info>
class dynamicSegmentTree {
    struct node {
        int l{}, r{};
        info v;
    };
    vector<node> tr;
    int n, root;
    static info defaultVal;

    int new_node() {
        tr.emplace_back();
        return int(tr.size()) - 1;
    }

    info get(int x, int lx, int rx, int l, int r) {
        if(!x || lx > r || l > rx) return defaultVal;
        if(lx >= l && rx <= r) return tr[x].v;
        int m = (lx + rx) >> 1;
        return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
    }

    int set(int x, int lx, int rx, int i, info val) {
        if (i < lx || i > rx) return x;
        if(!x) x = new_node();
        if(lx == rx) return tr[x].v = val, x;
        int m = (lx + rx) >> 1;
        tr[x].l = set(tr[x].l, lx, m, i, val), tr[x].r = set(tr[x].r, m + 1, rx, i, val);
        return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
    }
public:
    explicit dynamicSegmentTree(int _n = 1'000'000'000, int expectedOps = 1) : n(_n), tr(1), root(0) {
        ↪ tr.reserve(expectedOps * __lg(n) * 2); }

    void set(int i, info v) {
        root = set(root, 0, n, i, v);
    }
    info get(int l, int r) {
        return get(root, 0, n, l, r);
    }
};

struct info {
    int64_t sum;
    info(int64_t x) {
        sum = x;
    }
    info() {
        sum = 0;
    }
    friend info operator+(const info &l, const info &r) {
        info res;
        res.sum = l.sum + r.sum;
        return res;
    }
};

template<> info dynamicSegmentTree<info>::defaultVal = info();
```

0.1.3 FenwickTree

```

1 0-based array: add(i,val) updates one position in O(log n), query(i) returns prefix sum, and range(l,r) returns range
   ↪ sum.
2
3 lower_bound(target) finds the smallest index whose prefix sum | target using the Fenwick binary lifting trick in O(log
   ↪ n).
4 template<class T>
5 struct BIT {
6     int n;
7     vector<T> tree;
8     explicit BIT(int size) : n(size + 5), tree(n + 1) {}
9
10    void add(int i, T val) {
11        for (i++; i <= n; i += i & -i)
12            tree[i] += val;
13    }
14
15    T query(int i) {
16        T sum = 0;
17        for (i++; i > 0; i -= i & -i)
18            sum += tree[i];
19        return sum;
20    }
21
22    T range(int l, int r) {
23        if(l > r) return T();
24        return query(r) - query(l - 1);
25    }
26
27    int lower_bound(T target) {
28        if(target <= 0) return 0;
29        int pos = 0;
30        T sum = 0;
31        for (int i = 1 << __lg(n); i > 0; i >>= 1) {
32            if(pos + i <= n && sum + tree[pos + i] < target) {
33                sum += tree[pos + i];
34                pos += i;
35            }
36        }
37        return pos;
38    }
39 };
40
41 Fenwick trick using two BITs: supports add on a range and sum on a range.
42 template<typename T>
43 class BITRange { // 0-based
44     int n;
45     vector<T> B1, B2;
46
47     void add(vector<T>& bit, int i, T x) {
48         for (++i; i <= n; i += i & -i)
49             bit[i] += x;
50     }
51
52     T query(vector<T>& bit, int i) {
53         T res = 0;
54         for (++i; i > 0; i -= i & -i)
55             res += bit[i];
56         return res;
57     }
58
59 public:
60     explicit BITRange(int size) : n(size + 5), B1(n + 2), B2(n + 2) {}
61
62     void add(int l, int r, T x) {
63         add(B1, l, x);
64         add(B1, r + 1, -x);
65         add(B2, l, x * (l - 1));
66         add(B2, r + 1, -x * r);
67     }
68     void add(int i, T x) { add(B2, i, -x); }
69
70     T query(int i) {
71         return query(B1, i) * i - query(B2, i);
72     }
73
74     T range(int l, int r) {
75         if (l > r) return T();
76         return query(r) - query(l - 1);
77     }
78 };

```

```

79 Fenwick 2D trick using 4 BITs: supports add on a range and sum on a range.
80 class BIT2D {
81     int n, m;
82     vector<vector<array<ll, 2>>> M, A;
83     void upd2(vector<vector<array<ll, 2>>> &t, int x, int y, ll mul, ll add) {
84         for(int i = x; i < n; i += i & -i) {
85             for(int j = y; j < m; j += j & -j) {
86                 t[i][j][0] += mul;
87                 t[i][j][1] += add;
88             }
89         }
90     }
91     void upd1(int x, int y1, int y2, ll mul, ll add) {
92         upd2(M, x, y1, mul, -mul * (y1 - 1));
93         upd2(M, x, y2, -mul, mul * y2);
94         upd2(A, x, y1, add, -add * (y1 - 1));
95         upd2(A, x, y2, -add, add * y2);
96     }
97     ll query2(vector<vector<array<ll, 2>>> &t, int x, int y) {
98         ll mul = 0, add = 0;
99         for(int i = y; i > 0; i -= i & -i) {
100             mul += t[x][i][0];
101             add += t[x][i][1];
102         }
103         return mul * y + add;
104     }
105     ll query1(int x, int y) {
106         ll mul = 0, add = 0;
107         for(int i = x; i > 0; i -= i & -i) {
108             mul += query2(M, i, y);
109             add += query2(A, i, y);
110         }
111         return mul * x + add;
112     }
113 public:
114     BIT2D(int n, int m) : M(n + 2, vector<array<ll, 2>>(m + 2)), A(n + 2, vector<array<ll, 2>>(m + 2)), n(n + 2), m(m
        ↪ + 2) { }
115     void upd(int x1, int y1, int x2, int y2, ll val) {
116         x1++, y1++, x2++, y2++;
117         upd1(x1, y1, y2, val, -val * (x1 - 1));
118         upd1(x2, y1, y2, -val, val * x2);
119     }
120     ll query(int x1, int y1, int x2, int y2) {
121         x1++, y1++, x2++, y2++;
122         return query1(x2, y2) - query1(x1 - 1, y2) - query1(x2, y1 - 1) + query1(x1 - 1, y1 - 1);
123     }
124 };

```

0.1.4 Lazy-PST

```
1 struct tag {
2     int64_t add{};
3     void apply(tag const &p) {
4         add += p.add;
5     }
6     bool has() { return add; }
7 };
8 struct info {
9     int64_t sum;
10    info(int64_t x) { sum = x; }
11    info() { sum = 0; }
12    void apply(tag const &lz, int lx, int rx) {
13        sum += (rx - lx + 1) * lz.add;
14    }
15    friend info operator+(const info &l, const info &r) {
16        info res;
17        res.sum = l.sum + r.sum;
18        return res;
19    }
20 };
21
22 const int N = 250000, Q = 20000;
23 const int M = 2e6; // N * 2 + Q * (1 + __lg(N)) * 4
24
25 int lc[M], rc[M];
26 info vv[M];
27 tag llz[M];
28
29 int ptr = 1;
30 int clone(int x) {
31     lc[ptr] = lc[x], rc[ptr] = rc[x], vv[ptr] = vv[x], llz[ptr] = llz[x];
32     return ptr++;
33 }
34 int put(int x, tag lazy, int lx, int rx) {
35     if(!lazy.has()) return x;
36     x = clone(x);
37     llz[x].apply(lazy);
38     vv[x].apply(lazy, lx, rx);
39     return x;
40 }
41
42 int a[N];
43 int build(int lx, int rx) {
44     if(lx == rx) return vv[ptr] = a[lx], ptr++;
45     int mid = lx + rx >> 1;
46     int x = ptr++;
47     lc[x] = build(lx, mid), rc[x] = build(mid + 1, rx);
48     vv[x] = vv[lc[x]] + vv[rc[x]];
49     return x;
50 }
51
52 auto query(int x, int lx, int rx, int l, int r, tag pushed_lazy = tag()) {
53     if(lx >= l && rx <= r) {
54         auto res = vv[x];
55         res.apply(pushed_lazy, lx, rx);
56         return res;
57     }
58     int mid = lx + rx >> 1;
59     {
60         tag tmp_pushed = pushed_lazy;
61         (pushed_lazy = llz[x]).apply(tmp_pushed);
62     }
63     if(r <= mid) return query(lc[x], lx, mid, l, r, pushed_lazy);
64     if(mid < l) return query(rc[x], mid + 1, rx, l, r, pushed_lazy);
65     return query(lc[x], lx, mid, l, r, pushed_lazy) + query(rc[x], mid + 1, rx, l, r, pushed_lazy);
66 }
67
68 int update(int x, int lx, int rx, int l, int r, tag val, tag pushed_lazy = tag()) {
69     if(l > rx || lx > r) return put(x, pushed_lazy, lx, rx);
70     if(lx >= l && rx <= r) return put(x, val, lx, rx);
71     int mid = lx + rx >> 1;
72     x = clone(x);
73     {
74         tag tmp_pushed = pushed_lazy, tmp_val = val;
75         (pushed_lazy = llz[x]).apply(tmp_pushed), (val = llz[x]).apply(tmp_val);
76         llz[x] = tag();
77     }
78     lc[x] = update(lc[x], lx, mid, l, r, val, pushed_lazy);
79     rc[x] = update(rc[x], mid + 1, rx, l, r, val, pushed_lazy);
```

```
80     vv[x] = vv[lc[x]] + vv[rc[x]];
81     return x;
82 }
```


0.1.5 LazySegmentTree

Segment tree with lazy propagation to support range updates and range queries efficiently.

```

template<class info, class tag>
class lazySegment {
    int n;
    vector<info> tr;
    vector<tag> lz;
    static info defaultVal;

    inline void push(int x, int lx, int rx) {
        if(lx != rx) lz[x << 1].apply(lz[x]), lz[x << 1 | 1].apply(lz[x]);
        tr[x].apply(lz[x], lx, rx);
        lz[x] = tag();
    }

    template<class U>
    void build(int x, int lx, int rx, U &arr) {
        if(lx == rx) return void(tr[x] = arr[lx]);
        int m = (lx + rx) >> 1;
        build(x << 1, lx, m, arr);
        build(x << 1 | 1, m + 1, rx, arr);
        tr[x] = tr[x << 1] + tr[x << 1 | 1];
    }

    info get(int x, int lx, int rx, int l, int r) {
        push(x, lx, rx);
        if(lx > r || l > rx) return defaultVal;
        if(lx >= l && rx <= r) return tr[x];
        int m = (lx + rx) >> 1;
        return get(x << 1, lx, m, l, r) + get(x << 1 | 1, m + 1, rx, l, r);
    }

    void set(int x, int lx, int rx, int i, info val) {
        push(x, lx, rx);
        if(i < lx || i > rx) return;
        if(lx == rx) return void(tr[x] = val);
        int m = (lx + rx) >> 1;
        set(x << 1, lx, m, i, val), set(x << 1 | 1, m + 1, rx, i, val);
        tr[x] = tr[x << 1] + tr[x << 1 | 1];
    }

    void applyRange(int x, int lx, int rx, int l, int r, tag v) {
        push(x, lx, rx);
        if(lx > r || l > rx) return;
        if(lx >= l && rx <= r) return lz[x] = v, push(x, lx, rx);
        int m = (lx + rx) >> 1;
        applyRange(x << 1, lx, m, l, r, v), applyRange(x << 1 | 1, m + 1, rx, l, r, v);
        tr[x] = tr[x << 1] + tr[x << 1 | 1];
    }

public:
    explicit lazySegment(int _n) : n(_n + 1), tr(4 << __lg(n)), lz(4 << __lg(n)) { }
    template<class U>
    explicit lazySegment(U &arr) : n(int(arr.size())), tr(4 << __lg(n)), lz(4 << __lg(n)) {
        build(1, 0, n - 1, arr);
    }
    void set(int i, info v) {
        set(1, 0, n - 1, i, v);
    }
    void applyRange(int l, int r, tag v) {
        applyRange(1, 0, n - 1, l, r, v);
    }
    info get(int l, int r) {
        return get(1, 0, n - 1, l, r);
    }
};

struct tag {
    int64_t add{}, st{};
    void apply(tag const &p) {
        if(p.st) st = p.st, add = 0;
        else if(st) st += p.add;
        else add += p.add;
    }
};

struct info {
    int64_t sum;
    info(int64_t x) {
        sum = x;
    }
    info() {

```

```

80     sum = 0;
81 }
82 void apply(tag const &lz, int lx, int rx) {
83     if(lz.st) sum = (rx - lx + 1) * lz.st;
84     else if(lz.add) sum += (rx - lx + 1) * lz.add;
85 }
86 friend info operator+(const info &l, const info &r) {
87     info res;
88     res.sum = l.sum + r.sum;
89     return res;
90 }
91 };
92 template<> info lazySegment<info, tag>::defaultVal = info();
93
94 template<class info, class tag>
95 class lazySegment {
96     int n, K, h, clz_K;
97     vector<info> tr;
98     vector<tag> lz;
99     inline void apply_node(int p, const tag &v) {
100         int lvl = __builtin_clz(p) - clz_K, lx = (p << lvl) - K, rx = lx + (1 << lvl) - 1;
101         tr[p].apply(v, lx, rx);
102         if(p < K) lz[p].apply(v);
103     }
104     inline void push(int p) {
105         for(int s = h; s > 0; --s) {
106             int i = p >> s;
107             apply_node(i << 1, lz[i]), apply_node(i << 1 | 1, lz[i]), lz[i] = tag();
108         }
109     }
110     inline void pull(int p) {
111         while (p > 1) {
112             p >>= 1;
113             int lvl = __builtin_clz(p) - clz_K, lx = (p << lvl) - K, rx = lx + (1 << lvl) - 1;
114             tr[p] = tr[p << 1] + tr[p << 1 | 1], tr[p].apply(lz[p], lx, rx);
115         }
116     }
117     void init(int _n) {
118         n = _n, K = 1;
119         while (K < n) K <<= 1;
120         h = (K > 1 ? __builtin_ctz(K) : 0), clz_K = __builtin_clz(K);
121         tr.assign(2 * K, defaultVal), lz.assign(K, tag());
122     }
123 public:
124     static info defaultVal;
125     explicit lazySegment(int _n = 0) { init(_n); }
126     template<class U>
127     explicit lazySegment(const U &arr) {
128         init(int(arr.size()));
129         for(int i = 0; i < n; i++) tr[K + i] = arr[i];
130         for(int i = K - 1; i >= 1; i--) tr[i] = tr[i << 1] + tr[i << 1 | 1];
131     }
132     void set(int i, info val) {
133         int p = i + K;
134         push(p), tr[p] = val, pull(p);
135     }
136     void applyRange(int l, int r, tag v) {
137         if (l > r) return;
138         int l0 = l + K, r0 = r + K;
139         push(l0), push(r0);
140         for (int l1 = l0, r1 = r0; l1 <= r1; l1 >>= 1, r1 >>= 1) {
141             if(l1 & 1) apply_node(l1++, v);
142             if(!(r1 & 1)) apply_node(r1--, v);
143         }
144         pull(l0), pull(r0);
145     }
146     info get(int l, int r) {
147         if (l > r) return defaultVal;
148         int l0 = l + K, r0 = r + K;
149         push(l0), push(r0);
150         info resL = defaultVal, resR = defaultVal;
151         for (int l1 = l0, r1 = r0; l1 <= r1; l1 >>= 1, r1 >>= 1) {
152             if (l1 & 1) resL = resL + tr[l1++];
153             if (!(r1 & 1)) resR = tr[r1--] + resR;
154         }
155         return resL + resR;
156     }
157 };

```

0.1.6 Mo's

```

1  Mo's Algorithm Offline algorithm to answer range queries
2
3  int n, q;
4  cin >> n >> q;
5  int z = int(sqrt(n)) + 1;
6
7  vector<int> arr(n);
8  for(int i = 0; i < n; i++) {
9      cin >> arr[i];
10 }
11
12 vector<int> ans(q);
13 vector<array<int, 2>> qu(q);
14 vector<vector<int>> _mo(z);
15 for(int i = 0; i < q; i++) {
16     cin >> qu[i][0] >> qu[i][1];
17     _mo[qu[i][0] / z].push_back(i);
18 }
19
20 int curr = 0;
21 vector<int> freq(100001), f_f(100001);
22 auto add = [&](int i) {
23     f_f[++freq[arr[i]]]++;
24     curr = max(curr, freq[arr[i]]);
25 };
26 auto del = [&](int i) {
27     if(!--f_f[freq[arr[i]]] && curr == freq[arr[i]])
28         curr--;
29     --freq[arr[i]];
30 };
31
32 int c = 0, l = 0, r = -1;
33 for(auto &mo : _mo) {
34     if(mo.empty()) continue;
35     c? std::sort(mo.begin(), mo.end(), [&](int i, int j){return qu[i][1] < qu[j][1];}):
36     std::sort(mo.begin(), mo.end(), [&](int i, int j){return qu[i][1] > qu[j][1];});
37     c ^= 1;
38
39     for(int i : mo) {
40         while(r < qu[i][1]) add(++r);
41         while(l > qu[i][0]) add(--l);
42         while(r > qu[i][1]) del(r--);
43         while(l < qu[i][0]) del(l++);
44         ans[i] = curr;
45     }
46 }
47 for(int i : ans)
48     cout << i << '\n';
49
50 vector<pair<int64_t, Query>> blc; // {hilbert_order_id, Query}
51 // pow = __lg(n) + 1, rot = 0
52 function<int64_t(int, int, int, int)> hilbert_order = [&](int x, int y, int pow, int rot) -> int64_t {
53     if(pow == 0) return 0;
54     int h_pow = 1 << (pow - 1);
55     int seg = ((x < h_pow? (y < h_pow? 0: 3): (y < h_pow? 1: 2)) + rot) & 3;
56     const static int rotate_delta[] = {3, 0, 0, 1};
57     int nx = x & (x ^ h_pow), ny = y & (y ^ h_pow), n_rot = (rot + rotate_delta[seg]) & 3;
58     int64_t sub_square_size = 1LL << (2 * pow - 2), add = hilbert_order(nx, ny, pow - 1, n_rot);
59     return seg * 1LL * sub_square_size + (seg == 1 || seg == 2? add: sub_square_size - add - 1);
60 };
61 hilbert_order(l, r, __lg(n) + 1, 0);

```

0.1.7 OrderedSTLS

```
1  Tree-based container supporting order statistics: find k-th element and index of element in  $O(\log n)$ 
2
3  order_of_key, find_by_order =>  $O(\log(n))$ 
4  #include <ext/pb_ds/assoc_container.hpp>
5  #include <ext/pb_ds/tree_policy.hpp>
6  using namespace __gnu_pbds;
7
8  template<typename K, typename V, typename Comp = less<K>>
9  using ordered_map = tree<K, V, Comp, rb_tree_tag, tree_order_statistics_node_update>;
10 template<typename K, typename Comp = less<K>>
11 using ordered_set = ordered_map<K, null_type, Comp>;
12
13 template<typename K, typename V, typename Comp = less_equal<K>>
14 using ordered_multimap = tree<K, V, Comp, rb_tree_tag, tree_order_statistics_node_update>;
15 template<typename K, typename Comp = less_equal<K>>
16 using ordered_multiset = ordered_multimap<K, null_type, Comp>;
```

0.1.8 SegmentTree

```
1 Segment tree implemented iteratively (bottom-up) to support range queries and point updates in  $O(\log n)$  without  
2 ↪ recursion.  
3  
4 Build  $O(n)$   
5 query, set  $O(\log(n))$   
6 template<class info>  
7 class segmentTreeIterative {  
8     int size;  
9     vector<info> tree;  
10    static info defaultVal;  
11 public:  
12     explicit segmentTreeIterative(int n) : size(n), tree(size << 1, defaultVal) { }  
13  
14     template<class U>  
15     explicit segmentTreeIterative(const U &arr) : size(arr.size()), tree(size << 1, defaultVal) {  
16         for(int i = 0; i < arr.size(); i++)  
17             tree[i + size] = arr[i];  
18         for(int i = size - 1; i > 0; i--)  
19             tree[i] = tree[i << 1] + tree[i << 1 | 1];  
20     }  
21     void set(int i, info v) {  
22         tree[i += size] = v;  
23         for(i >>= 1; i; i >>= 1)  
24             tree[i] = tree[i << 1] + tree[i << 1 | 1];  
25     }  
26     info get(int l, int r) {  
27         info resL = defaultVal, resR = defaultVal;  
28         l += size, r += size + 1;  
29         while(l < r) {  
30             if(l & 1) resL = resL + tree[l++];  
31             if(r & 1) resR = tree[--r] + resR;  
32             l >>= 1, r >>= 1;  
33         }  
34         return resL + resR;  
35     }  
36 };  
37 struct info {  
38     int64_t sum;  
39     info(int64_t x) {  
40         sum = x;  
41     }  
42     info() {  
43         sum = 0;  
44     }  
45     friend info operator+(const info &l, const info &r) {  
46         info res;  
47         res.sum = l.sum + r.sum;  
48         return res;  
49     }  
50 };  
51  
52 template<> info segmentTreeIterative<info>::defaultVal = info();
```

0.1.9 SGTBeats

```

1  const int N = 2e5 + 9;
2  struct SGTBeats {
3      const ll inf = 1e18;
4      int n, n0;
5      ll max_v[4 * N], smax_v[4 * N], max_c[4 * N];
6      ll min_v[4 * N], smin_v[4 * N], min_c[4 * N];
7      ll sum[4 * N];
8      ll len[4 * N], ladd[4 * N], lval[4 * N];
9
10     void update_node_max(int k, ll x) {
11         sum[k] += (x - max_v[k]) * max_c[k];
12
13         if (max_v[k] == min_v[k]) {
14             max_v[k] = min_v[k] = x;
15         } else if (max_v[k] == smin_v[k]) {
16             max_v[k] = smin_v[k] = x;
17         } else {
18             max_v[k] = x;
19         }
20
21         if (lval[k] != inf && x < lval[k]) {
22             lval[k] = x;
23         }
24     }
25     void update_node_min(int k, ll x) {
26         sum[k] += (x - min_v[k]) * min_c[k];
27
28         if (max_v[k] == min_v[k]) {
29             max_v[k] = min_v[k] = x;
30         } else if (smax_v[k] == min_v[k]) {
31             min_v[k] = smax_v[k] = x;
32         } else {
33             min_v[k] = x;
34         }
35
36         if (lval[k] != inf && lval[k] < x) {
37             lval[k] = x;
38         }
39     }
40     void push(int k) {
41         if (n0 - 1 <= k) return;
42         if (lval[k] != inf) {
43             updateall(2 * k + 1, lval[k]);
44             updateall(2 * k + 2, lval[k]);
45             lval[k] = inf;
46             return;
47         }
48         if (ladd[k] != 0) {
49             addall(2 * k + 1, ladd[k]);
50             addall(2 * k + 2, ladd[k]);
51             ladd[k] = 0;
52         }
53         if (max_v[k] < max_v[2 * k + 1]) {
54             update_node_max(2 * k + 1, max_v[k]);
55         }
56         if (min_v[2 * k + 1] < min_v[k]) {
57             update_node_min(2 * k + 1, min_v[k]);
58         }
59
60         if (max_v[k] < max_v[2 * k + 2]) {
61             update_node_max(2 * k + 2, max_v[k]);
62         }
63         if (min_v[2 * k + 2] < min_v[k]) {
64             update_node_min(2 * k + 2, min_v[k]);
65         }
66     }
67     void update(int k) {
68         sum[k] = sum[2 * k + 1] + sum[2 * k + 2];
69
70         if (max_v[2 * k + 1] < max_v[2 * k + 2]) {
71             max_v[k] = max_v[2 * k + 2];
72             max_c[k] = max_c[2 * k + 2];
73             smax_v[k] = max(max_v[2 * k + 1], smax_v[2 * k + 2]);
74         } else if (max_v[2 * k + 1] > max_v[2 * k + 2]) {
75             max_v[k] = max_v[2 * k + 1];
76             max_c[k] = max_c[2 * k + 1];
77             smax_v[k] = max(smax_v[2 * k + 1], max_v[2 * k + 2]);
78         } else {
79             max_v[k] = max_v[2 * k + 1];
80             max_c[k] = max_c[2 * k + 1] + max_c[2 * k + 2];

```

```

81         smax_v[k] = max(smax_v[2 * k + 1], smax_v[2 * k + 2]);
82     }
83
84     if (min_v[2 * k + 1] < min_v[2 * k + 2]) {
85         min_v[k] = min_v[2 * k + 1];
86         min_c[k] = min_c[2 * k + 1];
87         smin_v[k] = min(smin_v[2 * k + 1], min_v[2 * k + 2]);
88     } else if (min_v[2 * k + 1] > min_v[2 * k + 2]) {
89         min_v[k] = min_v[2 * k + 2];
90         min_c[k] = min_c[2 * k + 2];
91         smin_v[k] = min(min_v[2 * k + 1], smin_v[2 * k + 2]);
92     } else {
93         min_v[k] = min_v[2 * k + 1];
94         min_c[k] = min_c[2 * k + 1] + min_c[2 * k + 2];
95         smin_v[k] = min(smin_v[2 * k + 1], smin_v[2 * k + 2]);
96     }
97 }
98 void _update_min(ll x, int a, int b, int k, int l, int r) {
99     if (b <= l || r <= a || max_v[k] <= x) {
100         return;
101     }
102     if (a <= l && r <= b && smax_v[k] < x) {
103         update_node_max(k, x);
104         return;
105     }
106     push(k);
107     _update_min(x, a, b, 2 * k + 1, l, (l + r) / 2);
108     _update_min(x, a, b, 2 * k + 2, (l + r) / 2, r);
109     update(k);
110 }
111 void _update_max(ll x, int a, int b, int k, int l, int r) {
112     if (b <= l || r <= a || x <= min_v[k]) {
113         return;
114     }
115     if (a <= l && r <= b && x < smin_v[k]) {
116         update_node_min(k, x);
117         return;
118     }
119     push(k);
120     _update_max(x, a, b, 2 * k + 1, l, (l + r) / 2);
121     _update_max(x, a, b, 2 * k + 2, (l + r) / 2, r);
122     update(k);
123 }
124 void addall(int k, ll x) {
125     max_v[k] += x;
126     if (smax_v[k] != -inf) smax_v[k] += x;
127     min_v[k] += x;
128     if (smin_v[k] != inf) smin_v[k] += x;
129
130     sum[k] += len[k] * x;
131     if (lval[k] != inf) {
132         lval[k] += x;
133     } else {
134         ladd[k] += x;
135     }
136 }
137 void updateall(int k, ll x) {
138     max_v[k] = x; smax_v[k] = -inf;
139     min_v[k] = x; smin_v[k] = inf;
140     max_c[k] = min_c[k] = len[k];
141
142     sum[k] = x * len[k];
143     lval[k] = x; ladd[k] = 0;
144 }
145 void _add_val(ll x, int a, int b, int k, int l, int r) {
146     if (b <= l || r <= a) {
147         return;
148     }
149     if (a <= l && r <= b) {
150         addall(k, x);
151         return;
152     }
153     push(k);
154     _add_val(x, a, b, 2 * k + 1, l, (l + r) / 2);
155     _add_val(x, a, b, 2 * k + 2, (l + r) / 2, r);
156     update(k);
157 }
158 void _update_val(ll x, int a, int b, int k, int l, int r) {
159     if (b <= l || r <= a) {
160         return;
161     }

```

```

162     if (a <= l && r <= b) {
163         updateall(k, x);
164         return;
165     }
166     push(k);
167     _update_val(x, a, b, 2 * k + 1, l, (l + r) / 2);
168     _update_val(x, a, b, 2 * k + 2, (l + r) / 2, r);
169     update(k);
170 }
171
172 ll _query_max(int a, int b, int k, int l, int r) {
173     if (b <= l || r <= a) {
174         return -inf;
175     }
176     if (a <= l && r <= b) {
177         return max_v[k];
178     }
179     push(k);
180     ll lv = _query_max(a, b, 2 * k + 1, l, (l + r) / 2);
181     ll rv = _query_max(a, b, 2 * k + 2, (l + r) / 2, r);
182     return max(lv, rv);
183 }
184
185 ll _query_min(int a, int b, int k, int l, int r) {
186     if (b <= l || r <= a) {
187         return inf;
188     }
189     if (a <= l && r <= b) {
190         return min_v[k];
191     }
192     push(k);
193     ll lv = _query_min(a, b, 2 * k + 1, l, (l + r) / 2);
194     ll rv = _query_min(a, b, 2 * k + 2, (l + r) / 2, r);
195     return min(lv, rv);
196 }
197
198 ll _query_sum(int a, int b, int k, int l, int r) {
199     if (b <= l || r <= a) {
200         return 0;
201     }
202     if (a <= l && r <= b) {
203         return sum[k];
204     }
205     push(k);
206     ll lv = _query_sum(a, b, 2 * k + 1, l, (l + r) / 2);
207     ll rv = _query_sum(a, b, 2 * k + 2, (l + r) / 2, r);
208     return lv + rv;
209 }
210
211 SGTBeats(int n, ll *a) : n(n) {
212     n0 = 1;
213     while (n0 < n) n0 <<= 1;
214     for (int i = 0; i < 2 * n0; ++i) ladd[i] = 0, lval[i] = inf;
215     len[0] = n0;
216     for (int i = 0; i < n0 - 1; ++i) len[2 * i + 1] = len[2 * i + 2] = (len[i] >> 1);
217
218     for (int i = 0; i < n; ++i) {
219         max_v[n0 - 1 + i] = min_v[n0 - 1 + i] = sum[n0 - 1 + i] = (a != nullptr ? a[i] : 0);
220         smax_v[n0 - 1 + i] = -inf;
221         smin_v[n0 - 1 + i] = inf;
222         max_c[n0 - 1 + i] = min_c[n0 - 1 + i] = 1;
223     }
224     for (int i = n; i < n0; ++i) {
225         max_v[n0 - 1 + i] = smax_v[n0 - 1 + i] = -inf;
226         min_v[n0 - 1 + i] = smin_v[n0 - 1 + i] = inf;
227         max_c[n0 - 1 + i] = min_c[n0 - 1 + i] = 0;
228     }
229     for (int i = n0 - 2; i >= 0; i--) {
230         update(i);
231     }
232 }
233
234 // all queries are performed on [l, r) segment (right exclusive)
235 // 0 indexed
236
237 // range minimize query
238 void update_min(int a, int b, ll x) {
239     _update_min(x, a, b, 0, 0, n0);
240 }
241
242 // range maximize query
243 void update_max(int a, int b, ll x) {

```



```

243     _update_max(x, a, b, 0, 0, n0);
244 }
245 // range add query
246 void add_val(int a, int b, ll x) {
247     _add_val(x, a, b, 0, 0, n0);
248 }
249 // range update query
250 void update_val(int a, int b, ll x) {
251     _update_val(x, a, b, 0, 0, n0);
252 }
253 // range minimum query
254 ll query_max(int a, int b) {
255     return _query_max(a, b, 0, 0, n0);
256 }
257 // range maximum query
258 ll query_min(int a, int b) {
259     return _query_min(a, b, 0, 0, n0);
260 }
261 // range sum query
262 ll query_sum(int a, int b) {
263     return _query_sum(a, b, 0, 0, n0);
264 }
265 };

```

0.1.10 Sparse

```

1  static range queries: it preprocesses an array so any query [l, r] is answered in O(1) using an idempotent merge (like
   ↪ min, max, gcd).
2
3  build in O(n log(n)), not allow overlapping like(sum, mul, ...)
4  template<typename T, class F>
5  struct sparse{
6      int n, Log;
7      vector<vector<T>>> tb;
8      F op;
9      explicit sparse(vector<T> const &a, F f) : op(f) {
10         n = int(a.size()), Log = __lg(n) + 1, tb = vector<vector<T>>>(Log);
11         if(n == 0) return;
12         tb[0] = a;
13         for(int l = 1; l < Log; l++) {
14             tb[l].resize(n + 1 - (1 << l));
15             for(int i = 0; i < tb[l].size(); i++) {
16                 tb[l][i] = op(tb[l - 1][i], tb[l - 1][i + (1 << (l - 1))]);
17             }
18         }
19     }
20     T query(int l, int r) {
21         if(l > r) return {};
22         int len = __lg(r - l + 1);
23         return op(tb[len][l], tb[len][r - (1 << len) + 1]);
24     }
25 };
26
27 like sparse table but allow overlap(min, sum, mul)
28
29 build O(n log(n)), query O(1)
30 template<typename T, typename F>
31 struct DST {
32     int N, Log;
33     vector<vector<T>>> st;
34     F op;
35     DST(const vector<T>& a, F f) : N((int)a.size()), Log(__lg(N) + 1), op(f), st(Log, vector<T>(N)) {
36         if (N == 0) return;
37         st[0] = a;
38         for (int k = 1, blc = 4; k < Log; ++k, blc <= 1) {
39             for (int m = 0; m < N; m += blc) {
40                 int md = min(m + (1 << k) - 1, N - 1), ed = min(m + blc - 1, N - 1);
41                 st[k][md] = st[0][md];
42                 for(int i = md - 1; i >= m; i--) st[k][i] = op(st[0][i], st[k][i + 1]);
43                 if(md + 1 < N) {
44                     st[k][md + 1] = st[0][md + 1];
45                     for(int i = md + 2; i <= ed; i++) st[k][i] = op(st[k][i - 1], st[0][i]);
46                 }
47             }
48         }
49     }
50     T query(int l, int r) {
51         if (l > r) return T{};
52         if (l == r) return st[0][l];
53         int k = __lg(l ^ r);
54         return op(st[k][l], st[k][r]);
55     }
56 };
57
58 template <typename T, class Combine>
59 struct Sparse2D {
60     int n, m, logN, logM;
61     Combine op;
62     vector<T> st;
63     inline int idx(int k1, int k2, int r, int c) { return ((k1 * logM + k2) * n + r) * m + c; }
64     Sparse2D(const vector<vector<T>>& grid, Combine op_) : op(op_) {
65         n = grid.size(), m = grid[0].size(), logN = 32 - __builtin_clz(n), logM = 32 - __builtin_clz(m);
66         st.assign(logN * logM * n * m, T());
67         for (int i = 0; i < n; ++i)
68             for (int j = 0; j < m; ++j)
69                 st[idx(0, 0, i, j)] = grid[i][j];
70         for (int k2 = 1; k2 < logM; ++k2)
71             for (int i = 0; i < n; ++i)
72                 for (int j = 0; j + (1 << k2) <= m; ++j)
73                     st[idx(0, k2, i, j)] = op(st[idx(0, k2 - 1, i, j)], st[idx(0, k2 - 1, i, j + (1 << (k2 - 1)))]);
74         for (int k1 = 1; k1 < logN; ++k1)
75             for (int k2 = 0; k2 < logM; ++k2)
76                 for (int i = 0; i + (1 << k1) <= n; ++i)
77                     for (int j = 0; j + (1 << k2) <= m; ++j)
78                         st[idx(k1, k2, i, j)] = op(st[idx(k1 - 1, k2, i, j)], st[idx(k1 - 1, k2, i + (1 << (k1 - 1)),
   ↪ j)]);

```

```

79     }
80     T query(int r1, int c1, int r2, int c2) {
81         int k1 = 31 - __builtin_clz(r2 - r1 + 1), k2 = 31 - __builtin_clz(c2 - c1 + 1);
82         T top = op(st[idx(k1, k2, r1, c1)], st[idx(k1, k2, r1, c2 - (1 << k2) + 1)]);
83         T bot = op(st[idx(k1, k2, r2 - (1 << k1) + 1, c1)], st[idx(k1, k2, r2 - (1 << k1) + 1, c2 - (1 << k2) + 1)]);
84         return op(top, bot);
85     }
86 };
87
88 template <typename T, class Combine>
89 struct SquareSparse2D {
90     int n, m, logK;
91     Combine op;
92     vector<T> st;
93
94     inline int idx(int L, int r, int c) { return (L * n + r) * m + c; }
95     SquareSparse2D(const vector<vector<T>>& grid, Combine op_) : op(op_) {
96         n = grid.size(), m = grid[0].size();
97         int maxK = min(n, m);
98         logK = 32 - __builtin_clz(maxK);
99         st.assign(logK * n * m, T());
100         for (int i = 0; i < n; ++i)
101             for (int j = 0; j < m; ++j)
102                 st[idx(0, i, j)] = grid[i][j];
103         for (int L = 1; L < logK; ++L) {
104             int len = 1 << L, half = 1 << (L - 1);
105             for (int i = 0; i + len <= n; ++i) {
106                 for (int j = 0; j + len <= m; ++j) {
107                     T top = op(st[idx(L - 1, i, j)], st[idx(L - 1, i, j + half)]);
108                     T bot = op(st[idx(L - 1, i + half, j)], st[idx(L - 1, i + half, j + half)]);
109                     st[idx(L, i, j)] = op(top, bot);
110                 }
111             }
112         }
113     }
114
115     T query(int i, int j, int k) {
116         int L = 31 - __builtin_clz(k);
117         int offset = k - (1 << L);
118         T top = op(st[idx(L, i, j)], st[idx(L, i, j + offset)]);
119         T bot = op(st[idx(L, i + offset, j)], st[idx(L, i + offset, j + offset)]);
120         return op(top, bot);
121     }
122 };

```

0.1.11 SqrtDecomposition

```

1 Maintains a sequence in blocks of size  $\sqrt{n}$  to support insertion, deletion, access, and partial range queries
  ↳ efficiently by splitting the array into chunks.
2 class sqrtDecomposition {
3 public:
4     size_t n, z;
5     vector<vector<int>>> arr;
6
7     explicit sqrtDecomposition(const vector<int> &a : n(a.size()), z(int(sqrt(a.size())) + 1), arr(z) {
8         for(int i = 0; i < n; i++) arr[i / z].push_back(a[i]);
9     }
10
11     void rebuild() {
12         int tot{};
13         vector<int> a(n);
14         for(auto &B : arr) {
15             for(auto i : B) a[tot++] = i;
16             B.clear();
17         }
18         z = int(sqrt(a.size()) + 1);
19         arr.resize(z);
20         for(tot = 0; tot < n; tot++) arr[tot / z].push_back(a[tot]);
21     }
22
23     void insert(int j, int x) {
24         for(int i = 0; i < z; i++) {
25             if(j <= arr[i].size()) {
26                 arr[i].insert(arr[i].begin() + j, x), n++;
27                 if(arr[i].size() > z * 10) rebuild();
28                 return;
29             }
30             j -= int(arr[i].size());
31         }
32     }
33
34     int erase(int j) {
35         for(int i = 0; i < z; i++) {
36             if(j < arr[i].size()) {
37                 int x = arr[i][j];
38                 for(j++; j < arr[i].size(); j++)
39                     arr[i][j - 1] = arr[i][j];
40                 arr[i].pop_back();
41                 return n--, x;
42             }
43             j -= int(arr[i].size());
44         }
45         return -1;
46     }
47
48     int element(int j) {
49         for(int i = 0; i < z; j -= int(arr[i++].size()))
50             if(j < arr[i].size()) return arr[i][j];
51         return -1;
52     }
53
54     int count(int l, int r) {
55         int j = 0;
56         for(int b = 0; b < z; b++) {
57             if(j > r) break;
58             if(j >= l) {
59                 if(j + arr[b].size() - 1 <= r) { // take all block
60                     j += int(arr[b].size());
61                     continue;
62                 }
63                 for(int i = 0; i < arr[b].size() && j <= r; j++, i++) /* count i */;
64             }
65             else if(j + arr[b].size() - 1 >= l) {
66                 int i = l - j;
67                 for(j = l; i < arr[b].size() && j <= r; j++, i++) /* count i */;
68             }
69             else j += int(arr[b].size());
70         }
71         return 0; // return something
72     }
73 };

```

0.1.12 Treap

```
1 Implicit Treap for sequences: supports split/merge by index, stores subtree size and sum, with lazy reverse flag.
2
3 O(log n) amortized
4 mt19937 rnd(time(nullptr));
5 struct Treap {
6     struct node {
7         uint32_t pri = rnd();
8         int sz = 1, l = 0, r = 0, val{};
9         int64_t sum{};
10        bool rev = false;
11        node(int x) : sum(val = x) { }
12    };
13    vector<node> tr;
14    Treap() : tr(1, 0) { tr[0].sz = 0; }
15    inline void pull(int x) {
16        tr[x].sz = tr[tr[x].l].sz + tr[tr[x].r].sz + 1;
17        tr[x].sum = tr[tr[x].l].sum + tr[tr[x].r].sum + tr[x].val;
18    }
19    inline void push(int x) {
20        if(tr[x].rev) {
21            tr[tr[x].l].rev ^= 1, tr[tr[x].r].rev ^= 1;
22            swap(tr[x].l, tr[x].r);
23            tr[x].rev = false;
24        }
25    }
26    inline int new_node(int val) {
27        tr.emplace_back(val);
28        return int(tr.size()) - 1;
29    }
30    int merge(int lx, int rx) {
31        if(!lx || !rx) return rx + lx;
32        push(lx), push(rx);
33        if(tr[lx].pri < tr[rx].pri) return tr[rx].l = merge(lx, tr[rx].l), pull(rx), rx;
34        return tr[lx].r = merge(tr[lx].r, rx), pull(lx), lx;
35    }
36    void split(int x, int k, int &lx, int &rx) {
37        if(!x) return lx = rx = 0, void();
38        push(x);
39        if(k <= tr[tr[x].l].sz) split(tr[x].l, k, lx, tr[x].l), pull(rx = x);
40        else split(tr[x].r, k - tr[tr[x].l].sz - 1, tr[x].r, rx), pull(lx = x);
41    }
42    array<int, 3> split(int x, int l, int r) {
43        int a, b, c;
44        split(x, r + 1, b, c);
45        split(b, l, a, b);
46        return {a, b, c};
47    }
48 };
```

0.1.13 WaveletMatrix

A Wavelet Matrix supports range counting and rank queries on a static array in $O(\log \text{MaxValue})$ per query.

`count_less(l,r,x)` counts numbers $< x$ in `a[l..r]`

Time Complexity

build in $O(n \log \text{MaxValue})$

`count_less` / `count_ge` in $O(\log \text{MaxValue})$ per query

Memory $O(n \log \text{MaxValue})$

struct WaveletMatrix {

int n, LOG;

vector<int> bv, mid;

vector<ll> ps;

WaveletMatrix(vector<int> a) : n(int(a.size())) {

if(n == 0) { LOG = 0; return; }

int mx = *max_element(a.begin(), a.end());

LOG = mx > 0 ? __lg(mx) + 1 : 1;

bv.assign(LOG * (n + 1), 0), ps.assign((LOG + 1) * (n + 1), 0), mid.resize(LOG);

vector<int> nxt(n);

for (int l = LOG - 1; l >= 0; --l) {

int OFF = (1 + 1) * (n + 1), OFFPrv = 1 * (n + 1);

for (int i = 0; i < n; ++i) ps[OFF + i + 1] = ps[OFF + i] + a[i];

int c1 = 0;

for (int i = 0; i < n; ++i) {

int b = (a[i] >> 1) & 1;

bv[OFFPrv + i + 1] = bv[OFFPrv + i] + b;

c1 += b;

}

int i0 = 0, i1 = mid[l] = n - c1;

for (int i = 0; i < n; ++i) {

if ((a[i] >> 1) & 1) nxt[i0++] = a[i];

else nxt[i1++] = a[i];

}

a = nxt;

}

for (int i = 0; i < n; ++i)

ps[0 * (n + 1) + i + 1] = ps[0 * (n + 1) + i] + a[i];

}

int count_less(int l, int r, int x) {

if (l > r || x <= 0 || LOG == 0) return 0;

if (x >= (1 << LOG)) return r - l + 1;

int li = l, ri = r + 1, count = 0;

for (int lvl = LOG - 1; lvl >= 0; --lvl) {

int b = (x >> lvl) & 1;

int obl = bv[lvl * (n + 1) + li], obr = bv[lvl * (n + 1) + ri];

int zbl = li - obl, zbr = ri - obr;

if (b) count += (zbr - zbl), li = mid[lvl] + obl, ri = mid[lvl] + obr;

else li = zbl, ri = zbr;

}

return count;

}

ll sum_less(int l, int r, int x) {

if (l > r || x <= 0 || LOG == 0) return 0;

if (x >= (1 << LOG)) return ps[LOG * (n + 1) + r + 1] - ps[LOG * (n + 1) + l];

int li = l, ri = r + 1;

ll res = 0;

for (int lvl = LOG - 1; lvl >= 0; --lvl) {

int b = (x >> lvl) & 1;

int obl = bv[lvl * (n + 1) + li], obr = bv[lvl * (n + 1) + ri];

int zbl = li - obl, zbr = ri - obr;

if(b) res += ps[lvl * (n + 1) + zbr] - ps[lvl * (n + 1) + zbl], li = mid[lvl] + obl, ri = mid[lvl] + obr;

else li = zbl, ri = zbr;

}

return res;

}

int range_count(int l, int r, int x, int y) {

if (x > y) return 0;

return count_less(l, r, y + 1) - count_less(l, r, x);

}

ll range_sum(int l, int r, int x, int y) {

if (x > y) return 0;

return sum_less(l, r, y + 1) - sum_less(l, r, x);

}

};

0.1.14 XorSegmentTree

```

1  const int N = 2e5 + 9;
2
3  using T = long long;
4  T a[N];
5  struct XorSegmentTree {
6      // the length of the array should be 2^LOG for some LOG
7      vector<T> t[4 * N]; // O(LOG 2^LOG) memory
8      T lazy[4 * N];
9      XorSegmentTree() {
10         for (int i = 0; i < 4 * N; i++) {
11             t[i].clear();
12             lazy[i] = 0;
13         }
14     }
15     // b = 0, e = 2^LOG - 1
16     // O(LOG 2^LOG)
17     void build(int n, int b, int e) {
18         lazy[n] = 0;
19         t[n].clear();
20         if (b == e) {
21             t[n].push_back(a[b]);
22             return;
23         }
24         int mid = (b + e) >> 1, l = n << 1, r = l | 1;
25         build(l, b, mid);
26         build(r, mid + 1, e);
27         int len = e - b + 1;
28         // t[n][i] = sum of a[p ^ i] over all b <= p <= e
29         t[n].resize(len);
30         for (int i = 0; i < len; i++) {
31             if (i < (len >> 1)) t[n][i] = t[l][i] + t[r][i];
32             else t[n][i] = t[r][i - (len >> 1)] + t[l][i - (len >> 1)];
33         }
34     }
35     // add x to a[i], basically the change occurs for all t[n][i] where n is a candidate node
36     // so we just create a lazy array to remember the change for each node
37     // O(LOG)
38     void upd(int n, int b, int e, int i, T x) {
39         if (b > i || e < i) return;
40         lazy[n] += x;
41         if (b == e && b == i) {
42             return;
43         }
44         int mid = (b + e) >> 1, l = n << 1, r = l | 1;
45         upd(l, b, mid, i, x);
46         upd(r, mid + 1, e, i, x);
47     }
48     // layer = LOG - 1
49     // return sum of a[p ^ x] over all i <= p <= j
50     // O(LOG)
51     T query(int n, int b, int e, int i, int j, int x, int layer) {
52         if (i > j or b > j or e < i) return 0;
53         if (b >= i and e <= j) return lazy[n] + (layer == -1 ? t[n][0] : t[n][x & ((1 << layer + 1) - 1)]);
54         int mid = (b + e) >> 1, l = n << 1, r = l | 1;
55         if (~x >> layer & 1) return query(l, b, mid, i, j, x, layer - 1) + query(r, mid + 1, e, i, j, x, layer - 1);
56         else {
57             int i1 = i, j1 = min(mid, j), i2 = max(i, mid + 1), j2 = j;
58             return query(r, mid + 1, e, mid + 1 + i1 - b, mid + 1 + j1 - b, x, layer - 1) +
59                 query(l, b, mid, b + i2 - (mid + 1), b + j2 - (mid + 1), x, layer - 1);
60         }
61     }
62 }t;

```

0.2 DP optimization

0.2.1 CHT

```
1 Maintains a set of linear functions  $y = mx + c$  to efficiently query the maximum value at any  $x$ , supporting dynamic
  ↪ insertion of lines.
2
3 add, query  $O(\log(n)) \Rightarrow n$  number of lines added
4
5 This Structure for max value
6
7 For min
8 add(-k, -m)
9 -query(x)
10 struct Line {
11     mutable int64_t m, c, p;
12     bool operator<(const Line& o) const { return m < o.m; }
13     bool operator<(int64_t x) const { return p < x; }
14 };
15 struct HullDynamic : multiset<Line, less<>> {
16     const int64_t inf = 2e18;
17     int64_t div(int64_t a, int64_t b) { return a / b - ((a ^ b) < 0 && a % b); }
18     bool isect(iterator x, iterator y) {
19         if (y == end()) { x->p = inf; return false; }
20         if (x->m == y->m) x->p = x->c > y->c ? inf : -inf;
21         else x->p = div(y->c - x->c, x->m - y->m);
22         return x->p >= y->p;
23     }
24     void add(int64_t k, int64_t m) {
25         auto z = insert({k, m, 0}), y = z++, x = y;
26         while(isect(y, z)) z = erase(z);
27         if(x != begin() && isect(--x, y)) isect(x, y = erase(y));
28         while((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y));
29     }
30     int64_t query(int64_t x) {
31         assert(!empty());
32         auto l = *lower_bound(x);
33         return l.m * x + l.c;
34     }
35 };
36
37 ###  $O(n)$  CHT
38 #### Upper Hull (Minimum), must insert slopes decreasing, and ask queries increasing
39
40 ####  $\text{Intersection}(l1, l2) = (l2.b - l1.b) / (l1.m - l2.m)$ 
41 class CHT {
42     struct Line { int64_t m, b; };
43     deque<Line> dq;
44     bool bad(Line l1, Line l2, Line l3) {
45         //  $\text{Intersection}(l1, l3) \leq \text{Intersection}(l1, l2)$ 
46         return __int128(l3.b - l1.b) * (l1.m - l2.m) <= __int128(l2.b - l1.b) * (l1.m - l3.m);
47     }
48     int64_t f(Line l, int64_t x) { return l.m * x + l.b; }
49 public:
50     // slopes decreasing
51     void add(int64_t m, int64_t b) {
52         Line l = {m, b};
53         while (dq.size() >= 2 && bad(dq[dq.size()-2], dq.back(), l)) dq.pop_back();
54         dq.push_back(l);
55     }
56     // queries increasing
57     int64_t query(int64_t x) {
58         while(dq.size() >= 2 && f(dq[0], x) >= f(dq[1], x)) dq.pop_front();
59         return f(dq[0], x);
60     }
61 };
```


0.2.2 SOSDP

```
1 Computes sum over all subsets (or superset) of bitmask values efficiently using bitwise DP.
2
3 build O(n log(n))
4
5 f[x]++;
6
7 f1[(N - 1) & ~x]++;
8
9 number of y such that:
10
11 x | y = x  => f[x]
12
13 x & y = x  => f1[(N - 1) & ~x]
14
15 x & y = 0  => f[(N - 1) & ~x]
16
17 !invert -> sum of subsets, invert -> undo pre operation
18 void sos(vector<int> &dp, bool invert = false) {
19     for (int i = 0; 1 << i < dp.size(); i++) {
20         for (int mask = 0; mask < dp.size(); mask++) {
21             if (mask & (1 << i)) {
22                 dp[mask] += invert? -dp[mask ^ (1 << i)] : dp[mask ^ (1 << i)];
23                 // dp[mask] >= mod? dp[mask] -= mod: dp[mask] < 0? dp[mask] += mod: 0;
24             }
25         }
26     }
27 }
28
29 template<typename T>
30 void SubsetZetaTransform(vector<T>& v) {
31     const int n = v.size();
32     for (int j = 1; j < n; j <= 1) {
33         for (int i = 0; i < n; i++) if (i & j) v[i] += v[i ^ j];
34     }
35 }
36
37 template<typename T>
38 void SubsetMobiusTransform(vector<T>& v) {
39     const int n = v.size();
40     for (int j = 1; j < n; j <= 1) {
41         for (int i = 0; i < n; i++) if (i & j) v[i] -= v[i ^ j];
42     }
43 }
44
45 template<typename T>
46 void SupersetZetaTransform(vector<T>& v) {
47     const int n = v.size(); // n must be a power of 2
48     for (int j = 1; j < n; j <= 1) {
49         for (int i = 0; i < n; i++) if (i & j) v[i ^ j] += v[i];
50     }
51 }
52
53 template<typename T>
54 void SupersetMobiusTransform(vector<T>& v) {
55     const int n = v.size(); // n must be a power of 2
56     for (int j = 1; j < n; j <= 1) {
57         for (int i = 0; i < n; i++) if (i & j) v[i ^ j] -= v[i];
58     }
59 }
```

0.2.3 Subset-Sum

```
1  **The time complexity is roughly  $O(n * \sqrt{n} * D / 64)$ , easily handling  $N, C \leq 10^5$  within a 1s time limit.**
2
3  **By using a greedy prefix and shuffling, it centers the search space to a bitset of size  $O(D * \sqrt{N})$  rather than
4  ↪ the full  $O(C)$ .**
5
6  **This heuristic provides massive speedups and memory efficiency over standard bitset DP, especially when the target
7  ↪ sum C is large.**
8
9  https://codeforces.com/blog/entry/98663`
10 auto subset_sum = [&](int C, vector<int> const &d) -> bool {
11     if(!C) return true; // empty subset
12     if(C < 0) return false;
13     vector<int> dd;
14     int D = 0;
15     for(int i : d) if(i <= C) dd.push_back(i), D = max(D, i);
16     const int n = int(dd.size());
17     int k = 0, cur = 0;
18     while(k < n && cur + dd[k] < C) cur += dd[k], dd[k++] *= -1;
19     if(k == n) return false;
20     shuffle(dd.begin(), dd.end(), rnd);
21     const int sq = int(sqrt(n)) + 1, mx_sum = (D * sq) << 1, off = mx_sum >> 1;
22     bits dp(mx_sum + 1);
23     dp[off] = true;
24     for(int i : dd) {
25         if(i > 0) dp |= dp << i;
26         else dp |= dp >> (-i);
27     }
28     return dp[C - cur + off];
29 };
30
31 auto build_subset_sum_C = [&](int C, vector<int> const &d) -> pair<bits, bool> {
32     if(!C) return {bits(d.size()), true};
33     if(C < 0) return {{}, false};
34
35     int D = 0;
36     vector<array<int, 2>> dd;
37     for(int i = 0; i < d.size(); i++) if(d[i] <= C) dd.push_back({d[i], i}), D = max(D, d[i]);
38     const int n = int(dd.size());
39
40     bits flp(d.size());
41     int k = 0, cur = 0;
42     while(k < n && cur + dd[k][0] < C) cur += dd[k][0], dd[k][0] *= -1, flp.set(dd[k++][1]);
43     if(k == n) return {{}, false};
44     shuffle(dd.begin(), dd.end(), rnd);
45
46     const int sq = int(sqrt(n)) + 1, mx_sum = (D * sq) << 1, off = mx_sum >> 1;
47     bits dp(mx_sum + 1);
48     vector<int> last(mx_sum + 1, -1);
49     dp[off] = true;
50     for(int i = 0; i < n; i++) {
51         int v = dd[i][0];
52         bits nxt = dp | (v > 0? dp << v: dp >> (-v)), nw = nxt ^ dp;
53         for(int j = nw.find_first(); j < nw.size(); j = nw.find_next(j)) last[j] = i;
54         swap(dp, nxt);
55         if(!last[C - cur + off]) {
56             int t = C - cur + off;
57             bits ans(d.size());
58             while(!last[t]) ans.set(dd[last[t]][1]), t -= dd[last[t]][0];
59             assert(t == off);
60             return {ans ^ flp, true};
61         }
62     }
63     return {{}, false};
64 };
```

0.3 Geometry

0.3.1 Circles

```
1 pair<ld, point> findCircle(point a, point b, point c) {
2     point m1 = (a + b) / 2.0, m2 = (b + c) / 2.0;
3     point ab = b - a, bc = c - b;
4     point center = lineIntersect(m1, m1 + point(-ab.y, ab.x),
5                                 m2, m2 + point(-bc.y, bc.x));
6     return {(center - a).dist(), center};
7 }
8
9 vector<point> lineCircleIntersect(point a, point b, point center, ld r) {
10     point ab = b - a, ao = center - a;
11     point proj = a + ab * ao.dot(ab) / ab.dist2();
12     ld d = (proj - center).dist();
13     if (d > r + EPS) return {};
14     if (abs(d - r) < EPS) return {proj};
15     ld h = (ld)sqrtl(r*r - d*d);
16     point dir = ab / ab.dist();
17     return {proj + dir * h, proj - dir * h};
18 }
19
20 in 0, 1, 2 points
21 vector<point> circleCircleIntersect(point c1, ld r1, point c2, ld r2) {
22     ld d = (c2 - c1).dist();
23     if(d > r1 + r2 + EPS || d < abs(r1 - r2) - EPS) return {};
24     if(abs(d) < EPS && abs(r1 - r2) < EPS) return vector(3, c1); // infinity intersection
25
26     ld a = (r1*r1 - r2*r2 + d*d) / (2 * d), h2 = r1*r1 - a*a;
27     if (h2 < -EPS) return {};
28
29     point dir = (c2 - c1) / d, p = c1 + dir * a;
30     if (abs(h2) < EPS) return {p};
31     ld h = sqrt(h2);
32     point offset(-dir.y * h, dir.x * h);
33     return {p + offset, p - offset};
34 }
35
36 pair<ld, point> minimumEnclosingCircle(vector<point> p) {
37     using circle = pair<ld, point>;
38     shuffle(p.begin(), p.end(), mt19937(random_device{}()));
39     auto contains = [](circle c, const vector<point>& pts) {
40         return all_of(pts.begin(), pts.end(),
41             [&](auto p) {return (p - c.second).dist() <= c.first + EPS;});
42     };
43     auto circleFrom2 = [](point a, point b) {
44         point c = (a + b) / 2.0;
45         return circle{(a - c).dist(), c};
46     };
47     auto circleFrom3 = [](point a, point b, point c) {
48         point ab = (a + b) / 2.0, ac = (a + c) / 2.0;
49         point ab_perp(-(b - a).y, (b - a).x), ac_perp(-(c - a).y, (c - a).x);
50         point o = lineIntersect(ab, ab + ab_perp, ac, ac + ac_perp);
51         return circle{(o - a).dist(), o};
52     };
53     vector<point> R;
54     function<circle(int)> welzl = [&](int n) -> circle {
55         if (n == 0 || R.size() == 3) {
56             if (R.empty()) return {};
57             if (R.size() == 1) return {0, R[0]};
58             if (R.size() == 2) return circleFrom2(R[0], R[1]);
59             return circleFrom3(R[0], R[1], R[2]);
60         }
61         point q = p[n - 1];
62         circle D = welzl(n - 1);
63         if (contains(D, {q})) return D;
64         R.push_back(q);
65         auto res = welzl(n - 1);
66         R.pop_back();
67         return res;
68     };
69     return welzl((int)p.size());
70 }
```

0.3.2 LiChao

```
1 struct Line {
2     ll a = 0, b = -inf;
3     inline ll eval(ll x) { return a * x + b; }
4 };
5 struct LiChao {
6     int n;
7     vector<Line> tr;
8
9     LiChao(int size) : n(size), tr(n << 2) { }
10
11 void update(int x, int lx, int rx, int l, int r, Line v) {
12     if (l > rx || lx > r) return;
13     if (lx == rx) {
14         if (tr[x].eval(lx) < v.eval(lx)) swap(tr[x], v);
15         return;
16     }
17     int mdx = (lx + rx) >> 1;
18     if (lx >= l && rx <= r) {
19         bool good_L = v.eval(lx) > tr[x].eval(lx);
20         bool good_M = v.eval(mdx) > tr[x].eval(mdx);
21         if (good_M) swap(tr[x], v);
22         good_L != good_M? update(x << 1, lx, mdx, l, r, v):
23         update(x << 1 | 1, mdx + 1, rx, l, r, v);
24         return;
25     }
26     update(x << 1, lx, mdx, l, r, v);
27     update(x << 1 | 1, mdx + 1, rx, l, r, v);
28 }
29
30 ll query(int x, int lx, int rx, int i) {
31     if (lx == rx) return tr[x].eval(i);
32     int mdx = (lx + rx) >> 1;
33     return max(tr[x].eval(i),
34               i <= mdx? query(x << 1, lx, mdx, i):
35               query(x << 1 | 1, mdx + 1, rx, i));
36 }
37
38 void add_line(int l, int r, Line v) {
39     update(1, 0, n, l, r, v);
40 }
41
42 ll get_max(int i) {
43     return query(1, 0, n, i);
44 }
45 };
```

0.3.3 Lines

```

1  projection of point p onto line ab
2  point project(point a, point b, point p) {
3      point ab = b - a;
4      return a + ab * (p - a).dot(ab) / ab.dist2();
5  }
6
7  works for any orientation
8  bool onSegment(point a, point b, point p) {
9      return sign((b - a).cross(p - a)) == 0 &&
10         sign((p - a).dot(p - b)) <= 0;
11  }
12
13  ccw: >0 left, <0 right, =0 collinear
14  int ccw(point a, point b, point c) {
15      return sign((b - a).cross(c - a));
16  }
17
18  works for any points
19  ld distanceToLine(point a, point b, point p) {
20      return fabs((b - a).cross(p - a)) / (b - a).dist();
21  }
22
23  works for any points
24  ld distanceToSegment(point a, point b, point p) {
25      if ((b - a).dot(p - a) < 0) return (p - a).dist();
26      if ((a - b).dot(p - b) < 0) return (p - b).dist();
27      return distanceToLine(a, b, p);
28  }
29
30  works for intersecting lines (not parallel)
31  point lineIntersect(point a, point b, point c, point d) {
32      point ab = b - a, cd = d - c;
33      return a + ab * ((c - a).cross(cd) / ab.cross(cd));
34  }
35
36  works for all segments (returns intersection point if exists)
37  bool segmentsIntersect(point a, point b, point c, point d, point &inter) {
38      int d1 = ccw(a, b, c), d2 = ccw(a, b, d);
39      int d3 = ccw(c, d, a), d4 = ccw(c, d, b);
40
41      if(d1 * d2 < 0 && d3 * d4 < 0)
42          return inter = lineIntersect(a, b, c, d), true;
43
44      if(d1 == 0 && onSegment(a, b, c)) return inter = c, true;
45      if(d2 == 0 && onSegment(a, b, d)) return inter = d, true;
46      if(d3 == 0 && onSegment(c, d, a)) return inter = a, true;
47      if(d4 == 0 && onSegment(c, d, b)) return inter = b, true;
48
49      return false;
50  }
51
52  ld triangleArea(point a, point b, point c) {
53      return 0.5 * fabs((b - a).cross(c - a));
54  }
55
56  bool pointInTriangle(point a, point b, point c, point p) {
57      ld s1 = (b - a).cross(p - a);
58      ld s2 = (c - b).cross(p - b);
59      ld s3 = (a - c).cross(p - c);
60      return (sign(s1) >= 0 && sign(s2) >= 0 && sign(s3) >= 0) ||
61         (sign(s1) <= 0 && sign(s2) <= 0 && sign(s3) <= 0);
62  }
63
64  angle abc in radians
65  ld angle_abc(point a, point b, point c) {
66      return acos(clamp<ld>((a - b).dot(c - b) / ((a - b).dist() * (c - b).dist()), -1, 1));
67  }
68
69  Lines
70  struct Line {
71      ld a, b, c;
72      Line() : a(0), b(0), c(0) {}
73      Line(ld a, ld b, ld c) : a(a), b(b), c(c) { normalize(); }
74      Line(point p1, point p2) {
75          a = p1.y - p2.y, b = p2.x - p1.x, c = -(a * p1.x + b * p1.y), normalize();
76      }
77      // Construct from a point and an angle (in radians)
78      Line(point p, ld angle) {
79          a = -sin(angle), b = cos(angle), c = -(a * p.x + b * p.y), normalize();
80      }

```

```

81 void normalize() {
82     ld z = hypot(a, b);
83     if(sign(z) != 0) a /= z, b /= z, c /= z;
84 }
85 // Returns the normal vector of the line
86 point normal() const { return {a, b}; }
87
88 // Returns the direction vector of the line
89 point direction() const {
90     return {-b, a};
91 }
92
93 // This returns the exact directed distance, The sign indicates which side of the line the point lies on.
94 ld eval(point p) const { return a * p.x + b * p.y + c; }
95
96 // Absolute distance from point to line
97 ld dist(point p) const { return abs(eval(p)); }
98
99 // Projection of a point onto the line
100 point proj(point p) const {
101     return p - normal() * eval(p);
102 }
103
104 // Reflection of a point across the line
105 point refl(point p) const {
106     return p - normal() * (2 * eval(p));
107 }
108
109 // Check if two lines are parallel
110 bool isParallel(const Line &l) const {
111     return sign(a * l.b - b * l.a) == 0;
112 }
113
114 // Check if two lines are perfectly identical
115 bool operator==(const Line &l) const {
116     return isParallel(l) && sign(a * l.c - c * l.a) == 0 && sign(b * l.c - c * l.b) == 0;
117 }
118
119 // Intersection of two lines (returns true if they intersect, stores result in 'res')
120 bool intersect(const Line &l, point &res) const {
121     if (isParallel(l)) return false;
122     ld det = a * l.b - b * l.a;
123     res.x = (b * l.c - c * l.b) / det;
124     res.y = (c * l.a - a * l.c) / det;
125     return true;
126 }
127
128 // Returns a line parallel to this one, passing through point p
129 Line parallelThrough(point p) const {
130     return Line(a, b, -(a * p.x + b * p.y));
131 }
132 // Returns a line perpendicular to this one, passing through point p
133 Line perpThrough(point p) const {
134     return Line(-b, a, b * p.x - a * p.y);
135 }
136 };

```

0.3.4 Notes

```

1  conj(a) -> a.imag() *=-1
2
3  abs(point) distance between (0,0) to this point
4
5  norm(point) squared magnitude -> real2 + imag2
6
7  hypot(x, y) -> sqrt(x2 + y2)
8
9  arg(vector) angle between this vector and x-axis
10
11 clamp(a, l, r) == min(r, max(l, a))
12
13 polar(rho, theta) -> make vector with length rho and angle theta
14
15 internal angle = (n - 2) * 180 / n
16
17 number of diagonals n * (n - 3) / 2
18
19 Area(p) = internal_points_cnt + (boundary_points/2) - 1
20
21 boundary_point in vector = gcd(|x2-x1|, |y2-y1|) + 1
22
23 line have infinity point, segment have to end points
24
25 vector(x, y) perpendicular to vector(-y, x) and (y, -x)
26
27 using ld = double;
28 const ld EPS = 1e-9;
29 const ld pi = acos(-1);
30 int sign(ld x) { return (x > EPS) - (x < -EPS); }
31 int sign(int64_t x) { return (x > 0) - (x < 0); }
32
33 template<class T>
34 struct Point {
35     using P = Point;
36     T x, y;
37     Point(T x = T(), T y = T()) : x(x), y(y) {}
38     bool operator<(P const &o) const { return sign(x - o.x)? x < o.x: sign(y - o.y) && y < o.y; }
39     bool operator==(P const &o) const { return !sign(x - o.x) && !sign(y - o.y); }
40     P operator+(P p) const { return {x + p.x, y + p.y}; }
41     P operator-(P p) const { return {x - p.x, y - p.y}; }
42     P operator*(T d) const { return {x * d, y * d}; }
43     P operator/(T d) const { return {x / d, y / d}; }
44     T dot(P p) { return x * p.x + y * p.y; }
45     T cross(P p) { return x * p.y - y * p.x; }
46     T cross(P p1, P p2) { return (p1 - *this).cross(p2 - *this); }
47     T dist2() { return x * x + y * y; }
48     ld dist() { return sqrt(dist2()); }
49     ld angle() { return atan2(y, x); }
50     ld absAngle() { ld res = atan2(y, x); return res + (sign(res) < 0? 2 * pi: 0); }
51     P unit() { return *this / dist(); }
52     P perp() { return {-y, x}; }
53     P rotate(ld a) { return {x * cos(a) - y * sin(a), x * sin(a) + y * cos(a)}; }
54     int half() { return int(sign(y) < 0 || (sign(y) == 0 && sign(x) < 0)); }
55 };
56 using point = Point<ld>;
57 ostream &operator<<(ostream &os, point p) { return os << '(' << p.x << ', ' << p.y << ')'; }
58 istream &operator>>(istream &is, point &p) { int x, y; is >> x >> y; p = point(x, y); return is; }
59
60 // To sort by rad & integer point
61 int quadrant(point const &a) {
62     if(a.x > 0 && a.y >= 0) return 0;
63     if(a.x <= 0 && a.y > 0) return 1;
64     return a.x < 0? 2: 3;
65 }
66 bool comp_rad(point const &a, point const &b) {
67     const int fa = quadrant(a), fb = quadrant(b);
68     if(fa != fb) return fa < fb;
69     return __int128(a.y) * b.x < __int128(b.y) * a.x;
70 }
71 template<class T> struct Point3D {
72     typedef Point3D P;
73     typedef const P &R;
74     T x, y, z;
75     explicit Point3D(T x = 0, T y = 0, T z = 0) : x(x), y(y), z(z) {}
76     bool operator<(R p) const { return tie(x, y, z) < tie(p.x, p.y, p.z); }
77     bool operator==(R p) const { return tie(x, y, z) == tie(p.x, p.y, p.z); }
78     P operator+(R p) const { return P(x + p.x, y + p.y, z + p.z); }
79     P operator-(R p) const { return P(x - p.x, y - p.y, z - p.z); }
80     P operator*(T d) const { return P(x * d, y * d, z * d); }

```

```

81 P operator/(T d) const { return P(x / d, y / d, z / d); }
82 T dot(R p) const { return x * p.x + y * p.y + z * p.z; }
83 P cross(R p) const { return P(y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x); }
84 T dist2() const { return x * x + y * y + z * z; }
85 double dist() { return sqrt((double) dist2()); }
86 double phi() { return atan2(y, x); }
87 double theta() { return atan2(sqrt(x * x + y * y), z); }
88 P unit() const { return *this / (T) dist(); }
89 P normal(P p) const { return cross(p).unit(); }
90 P rotate(double angle, P axis) const {
91     double s = sin(angle), c = cos(angle);
92     P u = axis.unit();
93     return u * dot(u) * (1 - c) + (*this) * c - cross(u) * s;
94 }
95 };

```


0.3.5 Polygons

```
1  works for any polygon (returns +1 for ccw, -1 for cw)
2  ld polygonSign(vector<point>& p) {
3      ld area = 0;
4      int n = (int)p.size();
5      p.push_back(p[0]);
6      for(int i = 0; i < n; ++i) area += p[i].cross(p[i + 1]);
7      p.pop_back();
8      return sign(0.5 * area);
9  }
10
11 works for any polygon (removes dups, enforces ccw order)
12 void normPolygon(vector<point>& p) {
13     vector<point> res;
14     for(auto i : p) if(res.empty() || (i - res.back()).dist() > EPS)
15         res.push_back(i);
16
17     if(res.size() > 1 && (res.front() - res.back()).dist() < EPS)
18         res.pop_back();
19
20     if(polygonSign(res) < 0) reverse(res.begin(), res.end());
21
22     p = res;
23 }
24
25 works for simple polygons with integer coordinates
26 ll internalPointsCount(vector<point>& p) {
27     ll A2 = 0, B = 0;
28     int n = (int)p.size();
29     p.push_back(p[0]);
30     for (int i = 0; i < n; ++i) {
31         point a = p[i], b = p[i + 1];
32         A2 += ll(a.x * b.y - a.y * b.x);
33         B += __gcd((ll)abs(b.x - a.x), (ll)abs(b.y - a.y));
34     }
35     p.pop_back();
36     return (abs(A2) - B + 2) / 2;
37 }
38
39 works for any polygon (cw or ccw, convex or not)
40 ld polygonArea(vector<point> &p) {
41     int n = (int)p.size();
42     p.push_back(p[0]);
43     ld area = 0;
44     for (int i = 0; i < n; ++i) area += p[i].cross(p[i + 1]);
45     p.pop_back();
46     return fabs(area) / 2.0;
47 }
48
49 works for any polygon (cw or ccw, convex or not)
50 bool pointInPolygon(vector<point> &p, point o, bool strictly = false) {
51     int in = 0, n = (int)p.size();
52     p.push_back(p[0]);
53     for (int i = 0; i < n; ++i) {
54         point a = p[i], b = p[i + 1];
55         if (onSegment(a, b, o)) return p.pop_back(), !strictly;
56         if ((a.y > o.y) != (b.y > o.y)) {
57             in ^= ((a.y < b.y ? ccw(a, b, o) : ccw(b, a, o)) > 0);
58         }
59     }
60     p.pop_back();
61     return in;
62 }
63
64 work for simple convex polygon
65 bool pointInConvex(vector<point> &poly, point p) {
66     int n = (int)poly.size();
67     if(n == 1) return sign((poly[0] - p).dist()) == 0;
68     if(n == 2) return onSegment(poly[0], poly[1], p);
69
70     point f = poly[0];
71
72     if(sign((poly[1] - f).cross(p - f)) < 0 || sign((poly[n - 1] - f).cross(p - f)) > 0) return false;
73
74     int l = 1, r = n - 1;
75     while(r > l + 1) {
76         int mid = (l + r) >> 1;
77         if(sign((poly[mid] - f).cross(p - f)) > 0) l = mid;
78         else r = mid;
79     }
```

```

80     return pointInTriangle(f, poly[l], poly[r], p);
81 }
82
83 works for any simple polygon (cw or ccw)
84 point polygonCentroid(vector<point>& p) {
85     ld A = 0, c;
86     point C(0, 0);
87     int n = (int)p.size();
88     p.push_back(p[0]);
89     for (int i = 0; i < n; ++i) {
90         point cur = p[i], nxt = p[i + 1];
91         c = cur.cross(nxt);
92         A += c;
93         C = C + ((cur + nxt) * c);
94     }
95     p.pop_back();
96     A *= 0.5;
97     if (abs(A) < EPS) return C;
98     return C / (6.0 * A);
99 }
100
101 ccw convex polygon (cuts polygon into left/right of directed line)
102 auto polygonCut(vector<point> &p, point a, point b) {
103     vector<point> l, r;
104     point inter;
105     int n = int(p.size());
106     p.push_back(p[0]);
107     auto f = [](auto &x, auto p) {return (x.empty() || sign((x.back() - p).dist()));};
108     for(int i = 0; i < n; i++) {
109         point curr = p[i], nxt = p[i + 1];
110         if(ccw(a, b, curr) <= 0 && f(r, curr)) r.push_back(curr);
111         if(ccw(a, b, curr) >= 0 && f(l, curr)) l.push_back(curr);
112         if(segmentsIntersect(a, b, curr, nxt, inter)) {
113             if(f(l, inter)) l.push_back(inter);
114             if(f(r, inter)) r.push_back(inter);
115         }
116     }
117     p.pop_back();
118     return pair{l, r};
119 }
120
121
122 convex hull return (CCW, convex polygon)
123 void convexHull(vector<point>& a, bool include_collinear = false) {
124     int n = (int)a.size();
125     if (n <= 1) return;
126     point p0 = *min_element(a.begin(), a.end(), [](point a, point b) {
127         return pair{a.y, a.x} < pair{b.y, b.x};
128     });
129     sort(a.begin(), a.end(), [&p0](const point& a, const point& b) {
130         int o = ccw(p0, a, b);
131         return o > 0 || (o == 0 && (p0 - a).dist2() < (p0 - b).dist2());
132     });
133     if (include_collinear) {
134         int i = (int)a.size() - 1;
135         while(i >= 0 && ccw(p0, a[i], a.back()) == 0) i--;
136         reverse(a.begin() + i + 1, a.end());
137     }
138     vector<point> st;
139     for(auto i : a) {
140         while(st.size() > 1 && (include_collinear? ccw(st[st.size() - 2], st.back(), i) < 0: ccw(st[st.size() - 2],
141             ↪ st.back(), i) <= 0))
142             st.pop_back();
143         if(st.empty() || i != st.back()) st.push_back(i);
144     }
145     if(!include_collinear && st.size() == 2 && st[0] == st[1]) st.pop_back();
146     a = st;
147 }
148
149 Sub problems
150 1. Distance between two polygons
151 2. Shift polygon by vector (check intersect with other polygon)
152 - The Problem: What is the minimum Euclidean distance between two separate convex polygons A and B?
153 - The Minkowski Solution: The minimum distance between A and B is exactly equal to the minimum distance from the
154     ↪ origin (0,0) to the boundary of the Minkowski Difference B - A.
155 - How to code it: Compute D = B - A. Iterate through all edges of D and find the minimum distance from the origin
156     ↪ (0,0) to each line segment. The overall minimum is your answer.
157
158 to get all vectors from b to a [minkowskiSum(a, -b)]

```

```

158  $O(N + M)$ 
159 vector<point> minkowskiSum(vector<point> a, vector<point> b) {
160     auto rot = [&](vector<point> &poly) {
161         int mi = int(min_element(poly.begin(), poly.end(), [](point p, point q) {
162             return pair{p.y, p.x} < pair{q.y, q.x};
163         }) - poly.begin());
164         rotate(poly.begin(), poly.begin() + mi, poly.end());
165     };
166     rot(a), rot(b);
167     int n = int(a.size()), m = int(b.size());
168     a.push_back(a[0]), a.push_back(a[1]);
169     b.push_back(b[0]), b.push_back(b[1]);
170     vector<point> res;
171     int i = 0, j = 0;
172     while (i < n || j < m) {
173         res.push_back(a[i] + b[j]);
174         int cr = sign((a[i + 1] - a[i]).cross(b[j + 1] - b[j]));
175         i += cr >= 0 && i < n;
176         j += cr <= 0 && j < m;
177     }
178     return res;
179 }
180
181 Finding faces of a planar graph
182
183 Two edges intersect only at a common vertex (if it exists). Such graphs are called planar.
184 // integer point
185 auto find_faces(vector<point> const &pts, vector<vector<int>> g) {
186     int n = int(pts.size());
187     vector<vector<bool>> vis(n);
188     for(int i = 0; i < n; i++) {
189         vis[i].resize(g[i].size());
190         sort(g[i].begin(), g[i].end(), [&](int l, int r) {
191             auto pl = pts[l] - pts[i], pr = pts[r] - pts[i];
192             return pl.half() != pr.half()? (pl.half() < pr.half()): (sign(pl.cross(pr)) > 0);
193         });
194     }
195     vector<vector<int>> faces;
196     for (int i = 0; i < n; i++) {
197         for (int id = 0; id < g[i].size(); id++) {
198             if(vis[i][id]) continue;
199             vector<int> face;
200             int v = i, e = id;
201             while(!vis[v][e]) {
202                 vis[v][e] = true;
203                 face.push_back(v);
204                 int u = g[v][e], e1 = int(lower_bound(g[u].begin(), g[u].end(), v, [&](int l, int r) {
205                     auto pl = pts[l] - pts[u], pr = pts[r] - pts[u];
206                     return pl.half() != pr.half()? (pl.half() < pr.half()): (pl.cross(pr) > 0);
207                 }) - g[u].begin()) + 1;
208                 if(e1 == g[u].size()) e1 = 0;
209                 v = u, e = e1;
210             }
211             reverse(face.begin(), face.end());
212             auto p1 = pts[face[0]];
213             __int128 sum = 0;
214             for (int j = 0; j < face.size(); ++j) {
215                 auto p2 = pts[face[j]], p3 = pts[face[(j + 1) % face.size()]];
216                 sum += (p2 - p1).cross(p3 - p2);
217             }
218             if (sum <= 0) faces.insert(faces.begin(), face);
219             else faces.emplace_back(face);
220         }
221     }
222     return faces;
223 }

```

0.3.6 totalArea

```
1  [Get total union area of rectangles]
2
3  struct Event {
4      int x, y1, y2, type;
5      bool operator<(const Event &e) const {
6          return x < e.x;
7      }
8  };
9
10 struct SegmentTree {
11     int n;
12     vector<int> cnt;
13     vector<int64_t> len;
14     vector<int> ys;
15
16     SegmentTree(const vector<int> &coords) : ys(coords) {
17         n = ys.size();
18         cnt.assign(4 * n, 0);
19         len.assign(4 * n, 0);
20     }
21
22     void update(int l, int r, int val, int i = 1, int tl = 0, int tr = -1) {
23         if (tr == -1) tr = n - 1;
24         if (r < tl || tr < l) return;
25         if (l <= tl && tr <= r) {
26             cnt[i] += val;
27         } else {
28             int tm = (tl + tr) / 2;
29             update(l, r, val, i * 2, tl, tm);
30             update(l, r, val, i * 2 + 1, tm + 1, tr);
31         }
32
33         if (cnt[i]) {
34             len[i] = ys[tr + 1] - ys[tl];
35         } else if (tl == tr) {
36             len[i] = 0;
37         } else {
38             len[i] = len[i * 2] + len[i * 2 + 1];
39         }
40     }
41
42     int64_t totalLen() const {
43         return len[1];
44     }
45 };
46
47 int64_t totalArea(vector<array<int, 4>> const &rects) {
48     vector<Event> events;
49     set<int> y_coords;
50
51     for (auto &[x1, y1, x2, y2] : rects) {
52         events.push_back({x1, y1, y2, 1});
53         events.push_back({x2, y1, y2, -1});
54         y_coords.insert(y1);
55         y_coords.insert(y2);
56     }
57
58     vector<int> ys(y_coords.begin(), y_coords.end());
59     map<int, int> y_map;
60     for (int i = 0; i < ys.size(); i++) {
61         y_map[ys[i]] = i;
62     }
63
64     for (auto &e : events) {
65         e.y1 = y_map[e.y1];
66         e.y2 = y_map[e.y2] - 1;
67     }
68
69     sort(events.begin(), events.end());
70     SegmentTree st(ys);
71
72     int64_t ans = 0;
73     for (int i = 0; i + 1 < events.size(); ++i) {
74         st.update(events[i].y1, events[i].y2, events[i].type);
75         int64_t dx = events[i + 1].x - events[i].x;
76         ans += dx * st.totalLen();
77     }
78
79     return ans;
```


0.4 Graphs

0.4.1 BCCTree

```
1 Computes bi-connected components of an undirected graph, identifies articulation points, and builds the BCC tree.
2
3 struct BCC {
4     int n;
5     vector<int> id;
6     vector<bool> isArt;
7     vector<vector<int>> tree, g;
8     explicit BCC(int n) : n(n), g(n) { }
9     void addEdge(int u, int v) {
10         g[u].push_back(v);
11         g[v].push_back(u);
12     }
13     void init() {
14         id.assign(n, -1);
15         int m = 0, cnt = 0;
16         vector<int> in(n, -1), low(n, -1), st; st.reserve(n);
17         vector<vector<int>> bcc(n);
18         function<void(int, int)> dfs = [&](int u, int p) -> void {
19             in[u] = low[u] = cnt++; st.push_back(u);
20             for(int v : g[u]) {
21                 if(!in[v]) {
22                     dfs(v, u), low[u] = min(low[u], low[v]);
23                     if(low[v] >= in[u]) {
24                         int x = -1;
25                         while(x ^ v) x = st.back(), st.pop_back(), bcc[x].push_back(m);
26                         bcc[u].push_back(m++);
27                     }
28                 }
29                 else if(v != p) low[u] = min(low[u], in[v]);
30             }
31         };
32         dfs(0, -1);
33         for(int u = 0; u < n; u++) {
34             if(bcc[u].size() == 1) id[u] = bcc[u].front();
35             else id[u] = m++;
36         }
37         tree.assign(m, vector<int>()), isArt.assign(m, false);
38         for(int u = 0; u < n; u++) {
39             if(bcc[u].size() ^ 1) {
40                 isArt[id[u]] = true;
41                 for(int v : bcc[u]) {
42                     tree[id[u]].push_back(v);
43                     tree[v].push_back(id[u]);
44                 }
45             }
46         }
47     }
48 };
```

0.4.2 Blossom

```
1  int n;
2  vector<vector<int>> g;
3  vector<int> match, p, base;
4  vector<bool> used, blossom;
5
6  int lca(int a, int b) {
7      vector vis(n, false);
8      for(;;) {
9          a = base[a], vis[a] = true;
10         if(match[a] == -1) break;
11         a = p[match[a]];
12     }
13     for(;;) {
14         b = base[b];
15         if(vis[b]) return b;
16         b = p[match[b]];
17     }
18 }
19
20 void markPath(int v, int b, int x) {
21     while (base[v] != b) {
22         blossom[base[v]] = blossom[base[match[v]]] = true;
23         p[v] = x;
24         x = match[v];
25         v = p[match[v]];
26     }
27 }
28
29 int findPath(int src) {
30     used.assign(n, false);
31     p.assign(n, -1);
32     iota(base.begin(), base.end(), 0);
33     queue<int> q;
34     q.push(src);
35     used[src] = true;
36     while(!q.empty()){
37         int v = q.front();
38         q.pop();
39         for (int u : g[v]) {
40             if(base[v] == base[u] || match[v] == u) continue;
41             if(u == src || (match[u] != -1 && p[match[u]] != -1)) {
42                 int cur = lca(v, u);
43                 blossom.assign(n, false);
44                 markPath(v, cur, u);
45                 markPath(u, cur, v);
46                 for (int i = 0; i < n; i++) {
47                     if(blossom[base[i]]) {
48                         base[i] = cur;
49                         if (!used[i]) used[i] = true, q.push(i);
50                     }
51                 }
52             } else if(p[u] == -1) {
53                 p[u] = v;
54                 if(match[u] == -1) return u;
55                 used[match[u]] = true;
56                 q.push(match[u]);
57             }
58         }
59     }
60     return -1;
61 }
62
63 void findMaximumMatching() {
64     match.assign(n, -1);
65     base.resize(n);
66     for (int i = 0; i < n; i++) {
67         if (match[i] == -1) {
68             int v = findPath(i);
69             while (v != -1) {
70                 int pv = p[v], w = match[pv];
71                 match[v] = pv, match[pv] = v, v = w;
72             }
73         }
74     }
75 }
```

0.4.3 DominatorTree

```
1 Computes the immediate dominators of each node in a directed graph from a given source
2
3 auto dominator_tree(vector<vector<int>> const &g, int src = 0) {
4     const int n = int(g.size());
5     vector<vector<int>> rg(n), bkt(n);
6     vector<int> sdom(n), idom(n), dsu(n), label(n), par(n), a(n, -1), rev(n);
7     int t = -1;
8     function<void(int)> dfs = [&](int u) -> void {
9         a[u] = ++t, rev[t] = u, label[t] = sdom[t] = dsu[t] = t;
10        for (int v : g[u]) {
11            if (!a[v]) dfs(v), par[a[v]] = a[u];
12            rg[a[v]].push_back(a[u]);
13        }
14    };
15    dfs(src);
16    function<int(int)> find = [&](int u) -> int {
17        if (u == dsu[u]) return u;
18        int v = find(dsu[u]);
19        if (sdom[label[dsu[u]]] < sdom[label[u]]) label[u] = label[dsu[u]];
20        return dsu[u] = v;
21    };
22    for (int i = t; i > 0; --i) {
23        for (int v : rg[i]) find(v), sdom[i] = min(sdom[i], sdom[label[v]]);
24        bkt[sdom[i]].push_back(i), dsu[i] = par[i];
25        for (int v : bkt[par[i]])
26            find(v), idom[v] = sdom[label[v]] < sdom[v] ? label[v] : par[i];
27        bkt[par[i]].clear();
28    }
29    for (int i = 1; i <= t; ++i)
30        if (idom[i] != sdom[i])
31            idom[i] = idom[idom[i]];
32    vector<int> ret(n); // real idom
33    for (int i = 0; i < n; i++)
34        ret[rev[i]] = rev[idom[i]];
35    return ret;
36 }
```


0.4.4 DSU

```

1 Maintains connected components with union by size and path compression, supporting fast merge and connectivity
  ↪ queries.
2
3 O((n)), almost constant, (n) = inverse Ackermann function
4 class DSU {
5     vector<int> p;
6 public: // 0-based
7     explicit DSU(int _n) : p(_n, -1) { }
8     int find(int i) {
9         return p[i] < 0 ? i : p[i] = find(p[i]);
10    }
11    bool merge(int u, int v) {
12        u = find(u);
13        v = find(v);
14        if(u == v)
15            return false;
16        if(p[u] < p[v]) swap(u, v);
17        p[v] += p[u];
18        p[u] = v;
19        return true;
20    }
21    int size(int i) {
22        return -p[find(i)];
23    }
24    bool same(int u, int v) {
25        return find(u) == find(v);
26    }
27 };
28
29 Rollback
30 class DSURollback {
31     vector<int> p;
32     vector<array<int, 2>> st;
33 public:
34     explicit DSURollback(int _n) : p(_n, -1) { }
35     int find(int i) {
36         if(p[i] < 0) return i;
37         if(p[p[i]] < 0) return p[i];
38         st.push_back({i, p[i]});
39         return p[i] = find(p[i]);
40     }
41     bool merge(int u, int v) {
42         u = find(u), v = find(v);
43         if(u == v) return false;
44         if(p[u] < p[v]) swap(u, v);
45         st.push_back({v, p[v]});
46         st.push_back({u, p[u]});
47         p[v] += p[u], p[u] = v;
48         return true;
49     }
50     int size(int i) { return -p[find(i)]; }
51     bool same(int u, int v) { return find(u) == find(v); }
52     int save() { return int(st.size()); }
53     void rollback(int lst) {
54         while(lst < st.size()) p[st.back()[0]] = st.back()[1], st.pop_back();
55     }
56 };
57
58 struct ParallelDSU {
59     int Log;
60     vector<DSU> ds;
61     ParallelDSU(int n) : Log(__lg(n) + 1), ds(Log, DSU(n)) { }
62     void _merge(int u, int v, int p2) {
63         if(ds[p2].same(u, v)) return;
64         ds[p2].merge(u, v);
65         if(!p2) return;
66         _merge(u, v, p2 - 1);
67         _merge(u + (1 << (p2 - 1)), v + (1 << (p2 - 1)), p2 - 1);
68     }
69     void merge(int u, int v, int len) {
70         while(len) {
71             int ctz = __builtin_ctz(len);
72             _merge(u, v, ctz);
73             len &= len - 1, u += 1 << ctz, v += 1 << ctz;
74         }
75     }
76     int find(int i) { return ds[0].find(i); }
77     int size(int i) { return ds[0].size(i); }
78     bool same(int u, int v) { return ds[0].same(u, v); }

```


0.4.5 GraphArray

```

1 Handles a functional graph (each node has exactly one outgoing edge) with cycle detection, leveling, and k-step
  ↳ queries using binary lifting.
2
3 lvl[u] distance from u to its cycle (0 if in a cycle)
4
5 index[u] position in cycle or pointer to cycle entry
6
7 cycleId[u] identifies which cycle a node belongs to
8
9 build_up() prepares binary lifting table up[u][l] for 2^l jumps along the graph
10
11 up_to_k(u, k) returns the node reached by k steps from u
12
13 go(from, to) minimum steps to reach to from from if possible; -1 if unreachable
14
15 if u in cycle: cycles[cycleId[u]][index[u]] = u
16 if u not in cycle: index[u] = first node in cycle
17 class GraphArray {
18 public:
19     int n, Log{};
20     vector<int> g, lvl, cycleId, index;
21     vector<vector<int>>> up, cycles;
22
23     explicit GraphArray(vector<int> const &a) : n(int(a.size())), g(a), lvl(n), cycleId(n, -1), index(n) {
24         iota(index.begin(), index.end(), 0);
25         vector<int> in(n);
26         for(int i = 0; i < n; i++) in[g[i]]++;
27
28         queue<int> q;
29         for(int i = 0; i < n; i++)
30             if (!in[i]) q.push(i);
31
32         vector<int> v;
33         while(!q.empty()) {
34             int u = q.front(); q.pop();
35             v.push_back(u);
36             if(--in[g[u]]) q.push(g[u]);
37         }
38
39         reverse(v.begin(), v.end());
40         for(int u : v) lvl[u] = lvl[g[u]] + 1, index[u] = index[g[u]];
41
42         for(int i = 0; i < n; i++) {
43             if(in[i]) {
44                 int u = i;
45                 vector<int> x;
46                 do {
47                     index[u] = int(x.size());
48                     cycleId[u] = int(cycles.size());
49                     x.push_back(u);
50                     in[u] = 0;
51                     u = g[u];
52                 } while(u != i);
53                 cycles.emplace_back(std::move(x));
54             }
55         }
56     }
57
58     void build_up() {
59         Log = __lg(n) + 1;
60         up.resize(n, vector(Log, 0));
61         for(int u = 0; u < n; u++) up[u][0] = g[u];
62
63         for(int l = 1; l < Log; l++) {
64             for(int u = 0; u < n; u++) {
65                 up[u][l] = up[up[u][l - 1]][l - 1];
66             }
67         }
68     }
69
70     int up_to_k(int u, int k) {
71         while(k) {
72             if(~cycleId[u]) {
73                 int i = cycleId[u];
74                 return cycles[i][(index[u] + k) % cycles[i].size()];
75             }
76             if(k <= lvl[u]) {
77                 u = up[u][__builtin_ctz(k)];
78                 k ^= k & -k;

```

```

79         }
80         else k -= lvl[u], u = index[u];
81     }
82     return u;
83 }
84
85 int go(int from, int to) { // minimum steps
86     if(lvl[from] < lvl[to]) return -1;
87     if(lvl[from] && !lvl[to]) {
88         if(cycleId[index[from]] != cycleId[to]) return -1;
89         return int(lvl[from] + (index[to] - index[index[from]] + cycles[cycleId[to]].size()) %
            ↪ cycles[cycleId[to]].size());
90     }
91     if(!lvl[from] && !lvl[to]) {
92         if(cycleId[from] != cycleId[to]) return -1;
93         return int((index[to] - index[from] + cycles[cycleId[to]].size()) % cycles[cycleId[to]].size());
94     }
95     return up_to_k(from, lvl[from] - lvl[to]) == to? lvl[from] - lvl[to]: -1;
96 }
97 };

```

0.4.6 Johnson

```
1  const int64_t inf = 1e18;
2  bool johnson(const vector<vector<pair<int, int64_t>>>& g, vector<vector<int64_t>>& dist) {
3      const int n = int(g.size());
4      vector<int64_t> h(n, 0);
5      vector<int> cnt(n, 0);
6      vector<bool> in_q(n, true);
7      queue<int> q;
8      for (int i = 0; i < n; ++i) q.push(i);
9      while (!q.empty()) {
10         int u = q.front();
11         q.pop(), in_q[u] = false;
12         for (const auto& [v, w] : g[u]) {
13             if (h[u] + w < h[v]) {
14                 h[v] = h[u] + w;
15                 if (!in_q[v]) {
16                     q.push(v);
17                     in_q[v] = true;
18                     if (++cnt[v] > n) return false;
19                 }
20             }
21         }
22     }
23     dist.assign(n, vector<int64_t>(n, inf));
24     auto dij = [&](int src) {
25         priority_queue<pair<int64_t, int>, vector<pair<int64_t, int>>, greater<>> pq;
26         dist[src][src] = 0, pq.emplace(0, src);
27
28         while (!pq.empty()) {
29             auto [d, u] = pq.top(); pq.pop();
30             if (d > dist[src][u]) continue;
31             for (const auto& [v, w] : g[u]) {
32                 int64_t new_w = w + h[u] - h[v];
33                 if (dist[src][u] + new_w < dist[src][v]) {
34                     dist[src][v] = dist[src][u] + new_w;
35                     pq.emplace(dist[src][v], v);
36                 }
37             }
38         }
39         for (int v = 0; v < n; ++v) {
40             if (dist[src][v] != inf) {
41                 dist[src][v] += h[v] - h[src];
42             }
43         }
44     };
45     for(int i = 0; i < n; ++i) dij(i);
46     return true;
47 }
```

0.4.7 Matching

```
1 Computes maximum matching in a bipartite graph with nl left nodes and nr right nodes, and can extract a minimum vertex
  ↪ cover.
2
3 (Hopcroft-Karp) =>  $O(\sqrt{V} \cdot E)$ 
4 If need minCover you should run minCover()
5 struct matching {
6     int nl, nr;
7     vector<vector<int>> g;
8     vector<int> dis, ml, mr;
9     explicit matching(int nl, int nr) : nl(nl), nr(nr), g(nl), dis(nl), ml(nl, -1), mr(nr, -1) { }
10
11     void add(int l, int r) {
12         g[l].push_back(r);
13     }
14
15     void bfs() {
16         queue<int> q;
17         for(int u = 0; u < nl; u++) {
18             if(ml[u] == -1) q.push(u), dis[u] = 0;
19             else dis[u] = -1;
20         }
21         while(!q.empty()) {
22             int l = q.front(); q.pop();
23             for(int r : g[l]) {
24                 if(mr[r] != -1 && dis[mr[r]] == -1)
25                     q.push(mr[r]), dis[mr[r]] = dis[l] + 1;
26             }
27         }
28     }
29
30     bool canMatch(int l) {
31         for(int r : g[l]) if(mr[r] == -1)
32             return mr[r] = l, ml[l] = r, true;
33         for(int r : g[l]) if(dis[l] + 1 == dis[mr[r]] && canMatch(mr[r]))
34             return mr[r] = l, ml[l] = r, true;
35         return false;
36     }
37
38     int maxMatch() {
39         int ans = 0, turn = 1;
40         while(turn) {
41             bfs(), turn = 0;
42             for(int l = 0; l < nl; l++) if(ml[l] == -1)
43                 turn += canMatch(l);
44             ans += turn;
45         }
46         return ans;
47     }
48     int maxMatchKarp() {
49         int ans = 0, visid = 0;
50         vector<int> vis(nl);
51         for(int l = 0; l < nl; l++) {
52             for(int r : g[l]) {
53                 if(mr[r] == -1) {
54                     mr[r] = l, ml[l] = r, ans++;
55                     break;
56                 }
57             }
58         }
59         function<bool(int)> Karp = [&](int l) -> bool {
60             if(vis[l] == visid) return false;
61             vis[l] = visid;
62             for(int r : g[l]) if(mr[r] == -1 || Karp(mr[r]))
63                 return mr[r] = l, ml[l] = r, true;
64             return false;
65         };
66
67         for(int l = 0; l < nl; l++) if(ml[l] == -1)
68             visid++, ans += Karp(l);
69         return ans;
70     }
71     pair<vector<int>, vector<int>> minCover() {
72         vector<int> L, R;
73         for (int u = 0; u < nl; ++u) {
74             if(dis[u] == -1) L.push_back(u);
75             else if(ml[u] != -1) R.push_back(ml[u]);
76         }
77         return {L, R};
78     }
```


0.4.8 MaxFlow

Computes the maximum flow from start to end in a directed graph with capacities using BFS-based augmenting paths.

$O(V^3)$ in worst case for Edmonds-Karp implementation

```
int max_flow(vector<vector<int>>> g, int start, int end) {
    if(start == end) return INT_MAX;
    int n = int(g.size());
    vector<int> par(n);
    int mxFlow = 0;
    auto bfs = [&]() {
        fill(par.begin(), par.end(), -1);
        queue<int> q;
        q.emplace(start);
        while(!q.empty()) {
            auto u = q.front(); q.pop();
            for(int v = 0; v < n; v++) {
                if(par[v] == -1 && g[u][v] > 0) {
                    par[v] = u;
                    if(v == end) return true;
                    q.emplace(v);
                }
            }
        }
        return false;
    };
    while(bfs()) {
        int res = INT_MAX, v = end;
        while(v != start) {
            int u = par[v];
            res = min(res, g[u][v]);
            v = u;
        }
        mxFlow += res;
        v = end;
        while(v != start) {
            int u = par[v];
            g[u][v] -= res, g[v][u] += res;
            v = u;
        }
    }
    return mxFlow;
}
```

Dinic algorithm run in $O(E * \sqrt{V})$

```
static constexpr ll inf = 2e18;
struct Dinic {
    struct Edge {
        int to, rev;
        ll cap, flow;
    };
    int n;
    vector<vector<Edge>>> g;
    vector<int> lvl, ptr;
    Dinic(int n) : n(n), g(n), lvl(n), ptr(n) {}
    void add_edge(int u, int v, ll cap, ll r_cap = 0) {
        g[u].push_back({v, int(g[v].size()) + (u == v), cap, 0});
        g[v].push_back({u, int(g[u].size()) - 1, r_cap, 0});
    }

    bool bfs(int s, int t) {
        fill(lvl.begin(), lvl.end(), -1);
        queue<int> q;
        q.push(s);
        lvl[s] = 0;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto &e : g[u]) {
                if (e.cap - e.flow > 0 && lvl[e.to] == -1) {
                    lvl[e.to] = lvl[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return lvl[t] != -1;
    }

    ll dfs(int u, int t, ll f) {
        if (u == t || f == 0) return f;
        for (int &i = ptr[u]; i < g[u].size(); i++) {
            auto &e = g[u][i];
            if (e.cap - e.flow > 0 && lvl[e.to] == lvl[u] + 1) {
                ll f2 = dfs(e.to, t, min(f, e.cap - e.flow));
                e.flow += f2;
                g[e.to][e.rev].flow -= f2;
                return f2;
            }
        }
        return 0;
    }
};
```



```

81         if(! psh = dfs(e.to, t, min(f, e.cap - e.flow))) {
82             return e.flow += psh, g[e.to][e.rev].flow -= psh, psh;
83         }
84     }
85 }
86 return lvl[u] = -1, 0;
87 }
88 ll max_flow(int s, int t) {
89     ll flow = 0;
90     while (bfs(s, t)) {
91         fill(ptr.begin(), ptr.end(), 0);
92         while(! psh = dfs(s, t, inf)) flow += psh;
93     }
94     return flow;
95 }
96 bool in_min_cut(int u) { return lvl[u] != -1; }
97 };

```

0.4.9 MinCostMaxFlow

```

1 Computes the maximum flow with minimum cost from source s to sink t using Dijkstra + potentials on residual graph.
2
3 O(F E logV)
4 const int inf = 1e9;
5 class MinCostMaxFlow {
6     struct Edge { int to, rev, cap, cost; };
7     int n;
8     vector<vector<Edge>> g;
9     vector<int> dis, pru, pri, h;
10
11 public:
12     MinCostMaxFlow(int n) : n(n), g(n), dis(n), pru(n), pri(n), h(n) {}
13
14     void addEdge(int u, int v, int cap, int cost) {
15         g[u].push_back({v, (int)g[v].size(), cap, cost});
16         g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
17     }
18
19     pair<int, int> flow(int s, int t, int maxF = inf) {
20         int res = 0, flow = 0;
21         fill(h.begin(), h.end(), 0);
22
23         while (maxF > 0) {
24             priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> q;
25             fill(dis.begin(), dis.end(), inf);
26             dis[s] = 0, q.emplace(0, s);
27
28             while (!q.empty()) {
29                 auto [d, v] = q.top(); q.pop();
30                 if (dis[v] < d) continue;
31                 for (int i = 0; i < g[v].size(); i++) {
32                     auto& e = g[v][i];
33                     if (e.cap > 0 && dis[e.to] > dis[v] + e.cost + h[v] - h[e.to]) {
34                         dis[e.to] = dis[v] + e.cost + h[v] - h[e.to];
35                         pru[e.to] = v, pri[e.to] = i;
36                         q.emplace(dis[e.to], e.to);
37                     }
38                 }
39             }
40             if(dis[t] == inf) break;
41
42             for(int v = 0; v < n; ++v) h[v] += dis[v];
43             int d = maxF;
44             for(int v = t; v != s; v = pru[v]) d = min(d, g[pru[v]][pri[v]].cap);
45
46             maxF -= d, flow += d, res += d * h[t];
47             for (int v = t; v != s; v = pru[v]) {
48                 auto& e = g[pru[v]][pri[v]];
49                 e.cap -= d, g[v][e.rev].cap += d;
50             }
51         }
52         return {flow, res};
53     }
54 };

```

0.4.10 Notes

```
1 int dx[] = {-1, 0, 0, 1, 1, -1, 1, -1};
2 int dy[] = {0, -1, 1, 0, 1, -1, -1, 1};
3 string dc = "ULRD"s;
4 subGraph that have all nodes of the original graph and some of its edges.
5 A subgraph is called Eulerian if each node has even degree.
6 Eulerian of n nodes and m edges = power(2, m - n - numberOfConnectedGraph)
```

0.4.11 TarjanSCC

```

1 Finds strongly connected components (SCCs) in a directed graph or bi-connected components, bridges, and articulation
  ↪ points in an undirected graph.
2
3 struct TarjanSCC {
4     bool directed;
5     int n, timer, sccCount;
6     vector<int> in, low, sccId;
7     vector<bool> inStack, isArticulation;
8     stack<int> st;
9     vector<pair<int, int>> bridges;
10    vector<vector<int>> graph, sccs, dag;
11
12    explicit TarjanSCC(int n, bool directed = false) : directed(directed), n(n), timer(0), sccCount(0) {
13        graph.resize(n);
14        in = low = sccId = vector(n, -1);
15        inStack = isArticulation = vector(n, false);
16    }
17
18    void addEdge(int u, int v) {
19        graph[u].push_back(v);
20        if(!directed) graph[v].push_back(u);
21    }
22
23    void dfs(int u, int parent = -1) {
24        in[u] = low[u] = ++timer;
25        st.push(u), inStack[u] = true;
26
27        int children = 0;
28        for (int v : graph[u]) if(directed || v != parent) {
29            if (in[v] == -1) {
30                children++;
31                dfs(v, u);
32
33                if(low[v] > in[u]) bridges.emplace_back(u, v);
34                if(parent != -1 && low[v] >= in[u]) isArticulation[u] = true;
35                low[u] = min(low[u], low[v]);
36            }
37            else if(inStack[v])
38                low[u] = min(low[u], in[v]);
39        }
40        if(parent == -1 && children > 1) isArticulation[u] = true;
41
42        if (low[u] == in[u]) {
43            sccs.emplace_back();
44            int v;
45            do {
46                v = st.top(), st.pop(), inStack[v] = false;
47                sccId[v] = sccCount;
48                sccs.back().push_back(v);
49            } while (v != u);
50            ++sccCount;
51        }
52    }
53
54    void init() { for(int i = 0; i < n; ++i) if(in[i] == -1) dfs(i); }
55
56    void buildDAG() {
57        dag.assign(sccCount, {});
58        set<pair<int, int>> edgeSet;
59
60        for (int u = 0, su, sv; u < n; ++u) {
61            su = sccId[u];
62            for(int v : graph[u]) {
63                sv = sccId[v];
64                if(su != sv && edgeSet.insert({su, sv}).second)
65                    dag[su].push_back(sv);
66            }
67        }
68    }
69 };

```

0.4.12 TreeIsomorphism

```
1 Computes a canonical string hash of a tree, allowing comparison of tree structures for isomorphism.
2
3 Use Hash instead of String
4 string hashTree(vector<vector<int>> &g) {
5     int n = int(g.size()), rem = n;
6     vector<int> deg(n);
7     queue<int> q;
8
9     for(int i = 0; i < n; i++) {
10         if(g[i].size() <= 1) q.push(i);
11         else deg[i] = int(g[i].size());
12     }
13     vector<vector<string>> hash(n);
14     auto calc = [&](int i) {
15         sort(hash[i].begin(), hash[i].end());
16         string res = "(";
17         for(string &s : hash[i]) res += s;
18         res += ')';
19         return res;
20     };
21     while(rem > 2) {
22         int sz = int(q.size()); rem -= sz;
23         while(sz-- > 0) {
24             int u = q.front(); q.pop();
25             string curr = calc(u);
26             for(int nxt : g[u]) {
27                 hash[nxt].push_back(curr);
28                 if(--deg[nxt] == 1) q.push(nxt);
29             }
30         }
31     }
32     int h1 = q.front();
33     string s1 = calc(q.front()); q.pop();
34     if(q.empty()) return s1;
35     int h2 = q.front();
36     string s2 = calc(q.front()); q.pop();
37     hash[h1].push_back(s2);
38     hash[h2].push_back(s1);
39     return min(calc(h1), calc(h2));
40 }
```

0.4.13 TwoSat

```
1 Solves 2-satisfiability (2-SAT) problems using Strongly Connected Components (SCC) in the implication graph.
2
3 either(f, j) encodes (f OR j) as:
4
5     !j → f
6     !f → j
7 a xor b => either(a, b), either(~a, ~b)
8 2 * x → x = true
9 2 * x + 1 → x = false
10 struct TwoSat {
11     int n;
12     vector<vector<int>> g;
13     vector<int> values; // 0 = false, 1 = true
14     TwoSat(int n): n(n), g(n << 1){ }
15     int addVar() {
16         g.emplace_back(), g.emplace_back();
17         return n++;
18     }
19     void either(int f, int j){
20         f=max(2*f,-1-2*f), j=max(2*j,-1-2*j);
21         g[f].push_back(j^1);
22         g[j].push_back(f^1);
23     }
24     void setValue(int x){ either(x,x); }
25     void atMostOne(const vector<int> & li){ // (optional)
26         if (li.size()<2) return;
27         int cur= ~li[0];
28         for(int i = 2; i < li.size(); i++) {
29             int next=addVar();
30             either(cur,~li[i]);
31             either(cur,next);
32             either(~li[i],next);
33             cur=~next;
34         }
35         either(cur,~li[1]);
36     }
37     vector<int> in,comp,z; int time=0;
38     int dfs(int i){
39         int low=in[i]=++time,x;z.push_back(i);
40         for(int e:g[i]) if (!comp[e])
41             low=min(low, in[e] ? : dfs(e));
42         if (low == in[i]) do{
43             x=z.back();z.pop_back();
44             comp[x]=low;
45             if (values[x>>1]==-1)
46                 values[x>>1]=x&1;
47         }while (x!=i);
48         return in[i]=low;
49     }
50     bool solve(){
51         values.assign(n,-1);
52         in.assign(n << 1, 0);comp=in;
53         for(int i = 0; i < n << 1; i++) if (!comp[i]) dfs(i);
54         for(int i = 0; i < n; i++) if (comp[i << 1]==comp[i << 1 | 1]) return false;
55         return true;
56     }
57 };
```

0.5 HelpFulMethods

```

1  code fast and init
2  using ll = int64_t;
3  cin.tie(nullptr)->sync_with_stdio(false);
4  #define debug(x) cout << (#x) << " = " << (x) << '\n'
5  cerr << clock() / 1000.0 << " Secs";
6
7  dynamic bitset library
8  #include <tr2/dynamic_bitset>
9
10 tr2::dynamic_bitset<> bit(N);
11
12 Faster unordered map (gp hash table)
13 #include <ext/pb_ds/assoc_container.hpp>
14 using namespace __gnu_pbds;
15 gp_hash_table<int, int> mp;
16
17 better priority queue
18 | Tag           | push      | pop       | modify
19 |-----|-----|-----|-----|
20 | pairing_heap_tag | 0(1)      | 0(log n)  | 0(log n)
21 | thin_heap_tag   | 0(1)      | 0(log n)  | 0(1) if priority increases
22 | binomial_heap_tag | 0(log n)  | 0(log n)  | 0(log n)
23 | rc_binomial_heap_tag | 0(1)      | 0(log n)  | 0(log n)
24 | binary_heap_tag | 0(log n)  | 0(log n)  | not supported
25 #include <ext/pb_ds/priority_queue.hpp>
26 using namespace __gnu_pbds;
27 using PQ = __gnu_pbds::priority_queue<int, less<>, thin_heap_tag>;
28
29 PQ q;
30 PQ::point_iterator it = q.push(5);
31 q.modify(it, 7);
32
33 Randomize
34 mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
35
36 template<typename T>
37 T rng(T l, T r) { return uniform_int_distribution<T>(l, r)(rnd); }
38
39 c++ optimization
40 #pragma GCC optimize("O3,unroll-loops")
41 #pragma GCC target("sse4.2,popcnt")
42 #pragma GCC target("avx2,bmi,bmi2,lzcnt")
43
44 monotonic stack
45 template<typename T, typename F>
46 auto monP(const vector<T> &a, const F &fun) {
47     int n = int(a.size());
48     stack<int> st;
49     vector<int> res(n);
50     for(int i = 0; i < n; i++) {
51         while(!st.empty() && !fun(a[st.top()], a[i])) st.pop();
52         res[i] = st.empty()? -1: st.top();
53         st.push(i);
54     }
55     return res;
56 }
57
58 template<typename T, typename F>
59 auto monS(const vector<T> &a, const F &fun) {
60     auto res = monP(vector(a.rbegin(), a.rend()), fun);
61     reverse(res.begin(), res.end());
62     for(auto &i : res) i = int(a.size()) - i - 1;
63     return res;
64 }
65
66 input line of numbers in vector with unknown size
67 template<class U, typename T>
68 U& operator >> (U &input, vector<T> &x){
69     x.clear();
70     T element;
71     while(input >> element) {
72         x.push_back(element);
73         if(input.peek() == '\n') break;
74     }
75     return input;
76 }
77
78 xor sum (1 ^ 2 ^ 3 ^ ... ^ n)
79 ll getXor(ll x) {

```

```
80     if (x % 4 == 0) return x;  
81     if (x % 4 == 1) return 1;  
82     if (x % 4 == 2) return x + 1;  
83     return 0;  
84 }
```


0.6 Math

0.6.1 Berlekamp-Massey

```
1  ### (Berlekamp-Massey):
2  BM The detective. It looks at your starting sequence s and works backward to find the shortest linear recurrence rule
   ↳ (the multipliers) that generated it. It runs in  $O(n^2)$  time and outputs the cleanly formatted, positive
   ↳ coefficients.
3
4  ### summary
5  you feed kth_term a handful of initial numbers and a massive k, and it does all the math to jump straight to the
   ↳ answer in  $O(n^2 \log k)$  time.
6  const int mod = 998244353;
7
8  int fp(int base, int64_t e = mod - 2) {
9      int res = 1;
10     while(e) {
11         if(e & 1) res = int(res * 1LL * base % mod);
12         base = int(base * 1LL * base % mod);
13         e >>= 1;
14     }
15     return res;
16 }
17
18 // Berlekamp-Massey algorithm to find the shortest linear recurrence
19 vector<int> BM(vector<int> const &s) {
20     vector<int> C = {1}, B = {1};
21     int L = 0, b = 1, m = 1, d;
22     for (int i = 0; i < s.size(); i++) {
23         d = 0;
24         for(int j = 0; j <= L; j++) d = int((d + C[j] * 1LL * s[i - j]) % mod);
25         if (!d) { b++; continue; }
26         auto T = C;
27         int c = int(d * 1LL * fp(m) % mod);
28         if(C.size() <= B.size() + b) C.resize(B.size() + b + 1, 0);
29         for(int j = 0; j < B.size(); j++) {
30             C[j + b] = int((C[j + b] - c * 1LL * B[j]) % mod);
31             if(C[j + b] < 0) C[j + b] += mod;
32         }
33         if(L <= i) L = i + 1 - L, B = T, b = 1, m = d;
34         else b++;
35     }
36     C = vector(C.begin() + 1, C.begin() + L + 1);
37     for(auto &i : C) if(i) i = mod - i;
38     return C;
39 }
40
41 // Polynomial multiplication modulo the recurrence polynomial
42 vector<int> multiply(vector<int> const &a, vector<int> const &b, vector<int> const &tr) {
43     int k = int(tr.size());
44     vector<int> res(a.size() + b.size() - 1, 0);
45     for(int i = 0; i < a.size(); ++i) {
46         for(int j = 0; j < b.size(); ++j) {
47             res[i + j] = int((res[i + j] + a[i] * 1LL * b[j]) % mod);
48         }
49     }
50     for(int i = int(res.size()) - 1; i >= k; --i) {
51         if(!res[i]) continue;
52         for (int j = 1; j <= k; ++j) {
53             res[i - j] = int((res[i - j] + res[i] * 1LL * tr[j - 1]) % mod);
54         }
55     }
56     res.resize(min(int(res.size()), k));
57     return res;
58 }
59 int kth_term(vector<int> const &s, int64_t k) {
60     if (k < s.size()) return s[k];
61     // Step 1: Get the recurrence transition multipliers {c_1, c_2, ...} from BM
62     auto f = BM(s);
63     if(f.empty()) return 0;
64
65     // Step 2: Binary exponentiation to calculate  $x^k \bmod f(x)$ 
66     vector res = {1}, base = {0, 1};
67     int64_t exp = k;
68     while (exp > 0) {
69         if (exp & 1) res = multiply(res, base, f);
70         base = multiply(base, base, f);
71         exp >>= 1;
72     }
73
74     // Step 3: Evaluate the final polynomial using the base cases from 's'
```

```
75     int ans = 0;
76     for (int i = 0; i < res.size(); ++i) {
77         ans = int((ans + res[i] * 1LL * s[i]) % mod);
78     }
79     return ans;
80 }
```

0.6.2 BigInteger

```
1 primality testing and integer factorization for very large integers using Miller-Rabin and Pollard's Rho
2 using u128 = __uint128_t;
3
4 u128 mul(u128 x, u128 y, u128 mod) {
5     if(x <= ULLONG_MAX && y <= ULLONG_MAX) return x * y % mod;
6     if(mod <= ULLONG_MAX) return x % mod * (y % mod) % mod;
7     if(x < y) swap(x, y);
8     u128 res = 0;
9     while(y) {
10         if(y & 1) (res += x) %= mod;
11         (x += x) %= mod;
12         y >>= 1;
13     }
14     return res;
15 }
16 u128 fp(u128 x, u128 e, u128 mod) {
17     u128 res = mod > 1;
18     while(e) {
19         if(e & 1) res = mul(res, x, mod);
20         x = mul(x, x, mod);
21         e >>= 1;
22     }
23     return res;
24 }
25
26 int ctz(u128 x) {
27     int a = __builtin_ctzll(x);
28     return a + (a == 64 ? __builtin_ctzll(x >> 64) : 0);
29 }
30 bool isPrime(u128 n) {
31     if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
32     u128 s = ctz(n-1), d = n >> s;
33     for(u128 a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
34         u128 p = fp(a%n, d, n), i = s;
35         while(p != 1 && p != n - 1 && a % n && i--) p = mul(p, p, n);
36         if (p != n-1 && i != s) return false;
37     }
38     return true;
39 }
40 u128 pollardRho(u128 n) {
41     u128 x = 0, y = 0, t = 30, prd = 2, i = 1, q;
42     auto f = [&](u128 x) { return mul(x, x, n) + i; };
43     while (t++%40 || __gcd(prd, n)!=1) {
44         if(x == y) x = ++i, y = f(x);
45         if((q = mul(prd, x>y?x-y:y-x, n))) prd = q;
46         x = f(x), y = f(f(y));
47     }
48     return __gcd(prd, n);
49 }
50 auto primeFactor(u128 n) {
51     vector<pair<u128,int>> res;
52     if(n==1) return res;
53     vector<u128> f;
54     function<void(u128)> slv=[&](u128 x) {
55         if(isPrime(x)) return f.push_back(x);
56         u128 d = pollardRho(x);
57         slv(d), slv(x/d);
58     };
59     slv(n);
60     sort(f.begin(), f.end());
61     for (u128 p : f) {
62         if(res.empty() || res.back().first != p) res.emplace_back(p, 1);
63         else res.back().second++;
64     }
65     return res;
66 }
```

0.6.3 BinaryTrie

```
1  binary trie (prefix tree) specialized for XOR queries, often called a XOR-trie
2
3  Maximum XOR with num if max = true
4
5  Minimum XOR with num if max = false
6
7  add, find(min, max) [0(log(n)) ]
8  template<int Log = 30, typename T = int>
9  class trie_xor{
10     vector<int> cnt, mp[2];
11     int new_node() {
12         cnt.push_back(0), mp[0].push_back(0), mp[1].push_back(0);
13         return int(cnt.size()) - 1;
14     }
15 public:
16     trie_xor(int EN = 0) {
17         cnt.resize(2), mp[0].resize(2), mp[1].resize(2);
18         cnt.reserve(EN * Log + 2), mp[0].reserve(EN * Log + 2), mp[1].reserve(EN * Log + 2);
19     }
20     void add(T num, int c = 1) {
21         int x = 1; bool b;
22         for(int i = Log - 1; i >= 0; i--) {
23             cnt[x] += c, b = num >> i & 1;
24             if(!mp[b][x]) mp[b][x] = new_node();
25             x = mp[b][x];
26         }
27         cnt[x] += c;
28     }
29
30     T getMax(T num) {
31         if(cnt[1] <= 0) return 0;
32         int x = 1;
33         T ans = 0; bool b;
34         for(int i = Log - 1; i >= 0; i--) {
35             b = !(num >> i & 1);
36             if(cnt[mp[b][x]] > 0) ans |= T(1) << i, x = mp[b][x];
37             else x = mp[!b][x];
38         }
39         return ans;
40     }
41     T getMin(T num) {
42         if(cnt[1] <= 0) return numeric_limits<T>::max();
43         int x = 1;
44         T ans = 0; bool b;
45         for(int i = Log - 1; i >= 0; i--) {
46             b = num >> i & 1;
47             if(cnt[mp[b][x]] > 0) x = mp[b][x];
48             else ans |= T(1) << i, x = mp[!b][x];
49         }
50         return ans;
51     }
52
53     void clear() { cnt.resize(1), mp[0].resize(1), mp[1].resize(1), new_node(); }
54 };
```

0.6.4 combinatorics

```

1 namespace combinatorics {
2     const int mod = 1e9 + 7;
3     int inv(int a) {
4         int x = 1, x1 = 0, q, t, b = mod;
5         while(b) q = a / b, a -= q * b, t = a, a = b, b = t, x -= q * x1, t = x, x = x1, x1 = t;
6         assert(a == 1);
7         return (x + mod) % mod;
8     }
9     struct Z {
10         int v{};
11         Z(int x) : v(x) { }
12         Z(int64_t x) : v(int(x % mod)) { if(v < 0) v += mod; }
13         Z() = default;
14         explicit operator int() const { return v; }
15     };
16     Z operator+(Z const &l, Z const &r) { return {l.v + r.v >= mod? l.v + r.v - mod: l.v + r.v}; }
17     Z operator-(Z const &l, Z const &r) { return {l.v < r.v? l.v + mod - r.v: l.v - r.v}; }
18     Z operator*(Z const &l, Z const &r) { return {int(l.v * 1LL * r.v % mod)}; }
19     Z operator/(Z const &l, Z const &r) { return {int(l.v * 1LL * inv(r.v) % mod)}; }
20     Z operator~(Z const &n) { return {n.v? mod - n.v: 0}; }
21     bool operator==(Z const &l, Z const &r) { return l.v == r.v; }
22     bool operator!=(Z const &l, Z const &r) { return l.v != r.v; }
23     bool operator<(Z const &l, Z const &r) { return l.v < r.v; }
24     ostream& operator<<(ostream& out, Z const &n) { return out << n.v; }
25     istream& operator>>(istream& in, Z &n) {
26         int64_t x; in >> x; n = {x}; return in;
27     }
28     Z operator""_M(uint64_t x) { return {int64_t(x)}; }
29
30     Z fix(int64_t x) {
31         int v = int(x < mod * 2 && x >= -mod? x: x % mod);
32         return v < 0? v + mod: v >= mod? v - mod: v;
33     }
34
35     int MXS_ = 1;
36     vector fac_(1, 1_M), inv_(1, 1_M);
37
38     void up_(int nw) {
39         nw = max(MXS_ << 1, 2 << __lg(nw));
40         fac_.resize(nw), inv_.resize(nw);
41         for(int i = MXS_; i < fac_.size(); i++)
42             fac_[i] = fac_[i - 1] * i;
43
44         inv_.back() = 1 / fac_.back();
45         for(int i = int(inv_.size()) - 2; i >= MXS_; i--)
46             inv_[i] = inv_[i + 1] * (i + 1);
47         MXS_ = nw;
48     }
49     Z fp(Z b, int64_t p) {
50         Z ans = 1;
51         while(p) {
52             if(p & 1) ans = ans * b;
53             b = b * b, p >>= 1;
54         }
55         return ans;
56     }
57
58     inline Z fac(int n) {
59         if(n < 0) return 0;
60         if(n >= MXS_) up_(n);
61         return fac_[n];
62     }
63     inline Z invFac(int n) {
64         if(n < 0) return 0;
65         if(n >= MXS_) up_(n);
66         return inv_[n];
67     }
68
69     inline Z nCr(int n, int r) {
70         if(r < 0 || r > n) return 0;
71         if(n >= MXS_) up_(n);
72         return fac_[n] * inv_[r] * inv_[n - r];
73     }
74     // For large N. In O(min(r, n - r))
75     inline Z nCr(int64_t n, int64_t r) {
76         if(r < 0 || r > n) return 0;
77         r = min<int64_t>(r, n - r);
78         if(r >= MXS_) up_(int(r));
79         Z ans = inv_[r];
80         for(int64_t i = n - r + 1; i <= n; i++) ans = ans * i;

```

```

81     return ans;
82 }
83 inline Z nPr(int n, int r) {
84     if(r < 0 || r > n) return 0;
85     if(n >= MXS_) up_(n);
86     return fac_[n] * inv_[n - r];
87 }
88 inline Z stars_and_bars(int n, int r){
89     return nCr(n + r - 1, r - 1);
90 }
91 inline Z catalan(int n) {
92     if(n < 0) return 0;
93     if(n * 2 >= MXS_) up_(n * 2);
94     return fac_[2 * n] * inv_[n] * inv_[n + 1];
95 }
96 }
97 using namespace combinatorics;

```

0.6.5 Convolutions

```
1  ### GCD Convolution
2
3  template<typename T>
4  void MultipleZetaTransform(vector<T>& v) {
5      const int n = (int)v.size() - 1;
6      for (int p : prm) {
7          if(p > n) break;
8          for (int i = n / p; i; i--) v[i] += v[i * p];
9      }
10 }
11
12 template<typename T>
13 void MultipleMobiusTransform(vector<T>& v) {
14     const int n = (int)v.size() - 1;
15     for (int p : prm) {
16         if(p > n) break;
17         for (int i = 1; i * p <= n; i++) v[i] -= v[i * p];
18     }
19 }
20
21 template<typename T>
22 vector<T> GCDConvolution(vector<T> A, vector<T> B) {
23     MultipleZetaTransform(A);
24     MultipleZetaTransform(B);
25     for (int i = 0; i < A.size(); i++) A[i] *= B[i];
26     MultipleMobiusTransform(A);
27     return A;
28 }
29
30 ### LCM Convolution
31
32 template<typename T>
33 void DivisorZetaTransform(vector<T>& v) {
34     const int n = (int)v.size() - 1;
35     for (int p : prm) {
36         if(p > n) break;
37         for (int i = 1; i * p <= n; i++) v[i * p] += v[i];
38     }
39 }
40
41 template<typename T>
42 void DivisorMobiusTransform(vector<T>& v) {
43     const int n = (int)v.size() - 1;
44     for (int p : prm) {
45         if(p > n) break;
46         for (int i = n / p; i; i--) v[i * p] -= v[i];
47     }
48 }
49
50 template<typename T>
51 vector<T> LCMConvolution(vector<T> A, vector<T> B) {
52     DivisorZetaTransform(A);
53     DivisorZetaTransform(B);
54     for (int i = 0; i < A.size(); i++) A[i] *= B[i];
55     DivisorMobiusTransform(A);
56     return A;
57 }
```

0.6.6 Equation

```

1 Equation to Solve
2  $n_0 * x + n_1 * y = n$ 
3
4 template<typename T = int>
5 struct equation {
6     bool valid;
7     T x, y, n0, n1, n, g;
8     short sign_n0g{}, sign_n1g{};
9     explicit equation(T a, T b, T n) : n0(a), n1(b), n(n), x(1), y(0) { // must a != 0 && b != 0
10         T q, x1 = 0, y1 = 1, t;
11         while(b) {
12             q = a / b;
13             t = b, b = a - q * b, a = t;
14             t = x1, x1 = x - q * x1, x = t;
15             t = y1, y1 = y - q * y1, y = t;
16         }
17         g = a;
18
19         valid = n % g == 0;
20         if(valid){
21             x *= n / g;
22             y *= n / g;
23             stepX = n1 / g;
24             stepY = n0 / g;
25             sign_n0g = (stepY < 0 ? -1 : 1);
26             sign_n1g = (stepX < 0 ? -1 : 1);
27         }
28     }
29     T stepX{}, stepY{};
30     void shift(int64_t cnt) {
31         // n0 * (x + n1 / g) + n1 * (y - n0 / g) == n
32         x += stepX * cnt;
33         y -= stepY * cnt;
34     }
35     void toX(int64_t new_x, bool f = true) {
36         // f == 0? x <= new_x: x >= new_x
37         if(stepX == 0) return;
38         int64_t dif = (new_x - x) / stepX;
39         shift(dif);
40         if(x < new_x && f) {
41             shift(sign_n1g);
42             assert(x >= new_x);
43         }
44         else if(x > new_x && !f) {
45             shift(-sign_n1g);
46             assert(x <= new_x);
47         }
48     }
49     void toY(int64_t new_y, bool f = true) {
50         // f == 0? y <= new_y: y >= new_y
51         if(stepY == 0) return;
52         int64_t dif = (y - new_y) / stepY;
53         shift(dif);
54         if(y < new_y && f) {
55             shift(-sign_n0g);
56             assert(y >= new_y);
57         }
58         else if(y > new_y && !f) {
59             shift(sign_n0g);
60             assert(y <= new_y);
61         }
62     }
63     array<T, 3> count(T lx, T rx, T ly, T ry) { // {cnt, lx, rx}
64         toX(lx);
65         if(x > rx) return {};
66         lx = x;
67         toX(rx, false);
68         rx = x;
69
70         toY(ly);
71         if(y > ry) return {};
72         ly = y;
73         toY(ry, false);
74         ry = y;
75
76         if(ly > ry) swap(ly, ry);
77         lx = max(lx, ly);
78         rx = min(rx, ry);
79         if(lx > rx) return {};

```



```
80     return {(rx - lx) / abs(stepX) + 1, lx, rx};  
81 }  
82 };
```

0.6.7 FFT

```

1 FFT (Fast Fourier Transform) with a clever optimization called complex packing, where two polynomials are packed into
  ↳ the real and imaginary parts of one complex array
2
3 using cd = complex<double>;
4 double pi = acos(-1);
5 vector<cd> roots{0, 1};
6 void prepare_roots(int n) {
7     if (roots.size() >= n) return;
8     int old_size = int(roots.size());
9     roots.resize(n);
10    for (int i = old_size; i < n; i <= 1) {
11        double ang = pi / i;
12        cd w(cos(ang), sin(ang));
13        for (int j = i / 2; j < i; j++) roots[2 * j] = roots[j], roots[2 * j + 1] = roots[j] * w;
14    }
15 }
16 void fft(vector<cd> &a) {
17     int n = int(a.size());
18     static vector<int> rev;
19     if (rev.size() != n) {
20         int k = __builtin_ctz(n); rev.assign(n, 0);
21         for (int i = 0; i < n; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));
22     }
23     prepare_roots(n);
24     for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
25     for (int k = 1; k < n; k <= 1) {
26         for (int i = 0; i < n; i += 2 * k) {
27             for (int j = 0; j < k; j++) {
28                 auto u = a[i + j], v = a[i + j + k] * roots[k + j];
29                 a[i + j] = u + v, a[i + j + k] = u - v;
30             }
31         }
32     }
33 }
34 void ifft(vector<cd> &a) {
35     int n = int(a.size());
36     reverse(a.begin() + 1, a.end());
37     fft(a);
38     for (auto &i : a) i /= n;
39 }
40 vector<int64_t> mul(vector<int> const &a, vector<int> const &b) {
41     if (a.empty() || b.empty()) return {};
42     int N = 1;
43     while (N < a.size() + b.size() - 1) N <= 1;
44     vector<cd> t(N);
45     for (int i = 0; i < a.size(); i++) t[i].real(a[i]);
46     for (int i = 0; i < b.size(); i++) t[i].imag(b[i]);
47     fft(t);
48     for (auto &x : t) x *= x;
49     ifft(t);
50     vector<int64_t> ans(N);
51     for (int i = 0; i < N; i++) ans[i] = (int64_t)round(t[i].imag() / 2.0);
52     return ans;
53 }
54 using ll = int64_t;
55 template<int mod> vector<int> convMod(const vector<int> &a, const vector<int> &b) {
56     if (a.empty() || b.empty()) return {};
57     vector<int> res(a.size() + b.size() - 1);
58     int B = 32 - __builtin_clz(res.size()), n = 1 << B, cut = int(sqrt(mod));
59     vector<cd> L(n), R(n), outs(n), outl(n);
60     for (int i = 0; i < a.size(); i++) L[i] = cd(a[i] / cut, a[i] % cut);
61     for (int i = 0; i < b.size(); i++) R[i] = cd(b[i] / cut, b[i] % cut);
62     fft(L), fft(R);
63     for (int i = 0; i < n; i++) {
64         int j = -i & (n - 1);
65         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.01 * n);
66         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.01 * n) / (cd)1i;
67     }
68     fft(outl), fft(outs);
69     for (int i = 0; i < res.size(); i++) {
70         ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) + .5);
71         ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
72         res[i] = ((av % mod * cut + bv) % mod * cut + cv) % mod;
73     }
74     return res;
75 }
76 string matching with wildcards using polynomial multiplication (convolution)
77
78 ### We want to find all occurrences of string t in string s, where either string can contain a wildcard ? that matches
  ↳ any character.

```

```

79 vector<int> string_matching(string &s, string &t) {
80     if (t.size() > s.size()) return {};
81     int n = s.size(), m = t.size();
82     vector<int> s1(n), s2(n), s3(n);
83     for(int i = 0; i < n; i++) {
84         s1[i] = s[i] == '?' ? 0 : s[i] - 'a' + 1; // assign any non-zero number for non '?'s
85         s2[i] = s1[i] * s1[i];
86         s3[i] = s1[i] * s2[i];
87     }
88     vector<int> t1(m), t2(m), t3(m);
89     for(int i = 0; i < m; i++) {
90         t1[i] = t[m - i - 1] == '?' ? 0 : t[m - i - 1] - 'a' + 1;
91         t2[i] = t1[i] * t1[i];
92         t3[i] = t1[i] * t2[i];
93     }
94     auto s1t3 = mul(s1, t3);
95     auto s2t2 = mul(s2, t2);
96     auto s3t1 = mul(s3, t1);
97     vector<int> oc;
98     for(int i = m - 1; i < n; i++) if(s1t3[i] - s2t2[i] * 2 + s3t1[i] == 0) oc.push_back(i - m + 1);
99     return oc;
100 }
101
102 ### Consider all the possible sums of three integers from the sequence at three different indices
103 int n;
104 cin >> n;
105 vector<int64_t> f(N2);
106 for(int i = 0, x; i < n; i++) {
107     cin >> x;
108     f[x + N]++;
109 }
110 auto f2 = mul(f, f);
111 for(int i = 0; i < N2; i++) f2[i + i] -= f[i];
112 for(auto &i : f2) i >>= 1;
113
114 vector<int64_t> fii(N2 + N2 - 1);
115 for(int i = 0; i < N2; i++) fii[i + i] = f[i];
116
117 auto fii_j = mul(fii, f);
118 for(int i = 0; i < N2; i++) fii_j[i + i + i] -= f[i];
119
120 auto f3 = mul(f, f2);
121 for(int i = 0; i < min(fii_j.size(), f3.size()); i++) f3[i] -= fii_j[i];
122 for(auto &i : f3) i /= 3;

```

0.6.8 FWHT

```
1 Fast Walsh-Hadamard Transform (FWHT) for AND, OR, and XOR convolutions modulo mod
2
3 void fwht_and(vector<ll>& a, bool invert) {
4     int n = a.size();
5     for (int len = 1; 2 * len <= n; len <= 1) {
6         for (int i = 0; i < n; i += 2 * len) {
7             for (int j = 0; j < len; ++j) {
8                 a[i + j] = (a[i + j] + (invert? -1: 1) * a[i + j + len] + mod) % mod;
9             }
10        }
11    }
12 }
13 void fwht_or(vector<ll>& a, bool invert) {
14     int n = a.size();
15     for (int len = 1; 2 * len <= n; len <= 1) {
16         for (int i = 0; i < n; i += 2 * len) {
17             for (int j = 0; j < len; ++j) {
18                 a[i + j + len] = (a[i + j + len] + (invert? -1: 1) * a[i + j] + mod) % mod;
19             }
20        }
21    }
22 }
23 void fwht_xor(vector<ll>& a, bool invert) {
24     int n = a.size();
25     for (int len = 1; 2 * len <= n; len <= 1) {
26         for (int i = 0; i < n; i += 2 * len) {
27             for (int j = 0; j < len; ++j) {
28                 ll u = a[i + j], v = a[i + j + len];
29                 a[i + j] = (u + v) % mod;
30                 a[i + j + len] = (u - v + mod) % mod;
31             }
32        }
33    }
34    if (invert) {
35        ll inv2 = (mod + 1) / 2;
36        ll inv_n = 1;
37        for(int i = 1; i < n; i <= 1)
38            inv_n = inv_n * inv2 % mod;
39        for (ll &x : a) x = x * inv_n % mod;
40    }
41 }
42 template<typename F>
43 vector<ll> convolution(vector<ll> a, vector<ll> b, F const &fun) {
44     int n = 1;
45     while (n < max(a.size(), b.size())) n <= 1;
46     a.resize(n), b.resize(n);
47     fun(a, false);
48     fun(b, false);
49     for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
50     fun(a, true);
51     return a;
52 }
```

0.6.9 Gauss

```

1 Gaussian elimination over a finite field modulo mod to solve a system of linear equations
2 vector<int> gauss(vector<vector<int>>> a) {
3     // a[i][0] * x1 + a[i][1] * x2 + ... = a[i].back()
4     int n = int(a.size()), m = int(a[0].size()) - 1;
5     vector<int> ia(m, -1);
6     for (int c = 0, r = 0; c < m && r < n; ++c) {
7         for(int i = r; i < n; i++) {
8             swap(a[i], a[r]);
9             if(a[r][c]) break; // a[col][col] != 0
10        }
11        if(!a[r][c]) continue; // infinity solution of a[col]
12        ia[c] = r;
13        int I = inv(a[r][c]);
14        for(int &j : a[r]) j = mul(j, I);
15        for(int i = 0; i < n; i++) {
16            if (i != r) {
17                int mm = a[i][c];
18                for(int j = 0; j <= m; j++) {
19                    a[i][j] = add(a[i][j], mod - mul(mm, a[r][j]));
20                }
21            }
22        }
23        r++;
24    }
25    vector<int> ans(m);
26    for(int i = 0; i < m; i++) if(ia[i] != -1) ans[i] = a[ia[i]][m];
27    for (int i = 0; i < n; i++) {
28        int sum = 0;
29        for(int j = 0; j < m; j++) sum = add(sum, mul(ans[j], a[i][j]));
30        if(sum != a[i][m]) return {}; // no solution
31    }
32    return ans;
33 }
34 Gaussian elimination over a finite field to solve a system of linear equations
35 vector<ld> gauss(vector<vector<ld>>> a) {
36     // a[i][0] * x + a[i][1] * y + ... = a[i].back()
37     int n = int(a.size()), m = int(a[0].size()) - 1;
38     vector<int> ia(m, -1);
39     for (int c = 0, r = 0; c < m && r < n; ++c) {
40         for (int i = r; i < n; i++) {
41             swap(a[i], a[r]);
42             if(fabs(a[r][c]) > EPS) break; // a[col][col] != 0
43         }
44         if(fabs(a[r][c]) < EPS) continue; // infinity solution of a[col]
45         ia[c] = r;
46         ld I = 1.0 / a[r][c];
47         for(auto &j : a[r]) j *= I;
48         for(int i = 0; i < n; i++) {
49             if (i != r) {
50                 ld mm = a[i][c];
51                 for(int j = 0; j <= m; j++) {
52                     a[i][j] -= mm * a[r][j];
53                 }
54             }
55         }
56         r++;
57     }
58     vector<ld> ans(m);
59     for(int i = 0; i < m; i++) if(ia[i] != -1) ans[i] = a[ia[i]][m];
60     for (int i = 0; i < n; i++) {
61         ld sum = 0;
62         for(int j = 0; j < m; j++) sum += ans[j] * a[i][j];
63         if(fabs(sum - a[i][m]) > EPS) return {}; // no solution
64     }
65     return ans;
66 }
67
68 Gauss XOR
69 like
70  $x_1 \wedge (\text{not } x_2) \wedge (x_3) = 1$ 
71 const int N = 100;
72 class Gauss {
73     const int INF = 2;
74     const double EPS = 1E-9;
75 public:
76     int gauss(vector<bitset<N>> a, int n, int m, bitset<N> &ans) {
77         vector<int> where(m, -1);
78         for (int col = m - 1, row = 0; col >= 0 && row < n; col--) {
79             for (int i = row; i < n; ++i) {
80                 if (a[i][col]) {

```

```

81         swap(a[i], a[row]);
82         break;
83     }
84 }
85 if (!a[row][col])
86     continue;
87 where[col] = row;
88 for (int i = 0; i < n; ++i) {
89     if (i != row && a[i][col])
90         a[i] ^= a[row];
91 }
92 ++row;
93 }
94 ans.reset();
95 for (int i = 0; i < m; ++i) {
96     if (where[i] != -1) {
97         int r = where[i];
98         bool sum = false;
99         for (int j = 0; j < i; ++j) {
100             sum ^= (a[r][j] & ans[j]);
101         }
102         ans[i] = (a[r][m] ^ sum);
103     } else {
104         ans[i] = true;
105     }
106 }
107 for (int i = 0; i < n; ++i) {
108     bool sum = false;
109     for (int j = 0; j < m; ++j)
110         sum ^= (ans[j] & a[i][j]);
111     if (sum != a[i][m])
112         return 0;
113 }
114 for (int i = 0; i < m; ++i)
115     if (where[i] == -1)
116         return INF;
117 return 1;
118 }
119 };

```

0.6.10 Generating-Functions

```
1  ### number of subsets of an array of n elements having sum equal to k for each k from 1 to m
2
3  https://judge.yosupo.jp/problem/sharp\_p\_subset\_sum`
4
5  int n, m;
6  cin >> n >> m;
7  vector<int> a(m + 1, 0);
8  for (int i = 0, x; i < n; i++) {
9      cin >> x; // x >= 1, handle [x = 0] separately
10     a[x]++;
11 }
12 poly p(m + 1);
13 precompute_inv(m);
14 for (int i = 1; i <= m; i++) {
15     for (int j = 1; i * j <= m; j++) {
16         if (j & 1) p[i * j] = add<mod>(p[i * j], mul<mod>(a[i], inv[j]));
17         else p[i * j] = add<mod>(p[i * j], mod - mul<mod>(a[i], inv[j]));
18     }
19 }
20 p = p.exp(m + 1);
21 // check for m = 0
22 for(int i = 1; i <= m; i++) cout << p[i] << ' ';
23 cout << '\n';
24
25 ### transform array to prefix sum m times
26
27 auto slv(vector<int> &a, int m) {
28     vector<int> b(a.size() + 1);
29     for(int i = 0; i < a.size(); i++)
30         b[i] = nCr(m + i - 1, i);
31     return ntt.mulMod(a, b);
32     while(m--) {
33         for(int i = 1; i < a.size(); i++) {
34             a[i] = add<mod>(a[i], a[i - 1]);
35         }
36     }
37     return a;
38 }
```

0.6.11 HelpfulFunctions

```
1 bool isPrime(ll n) {
2     if(n < 4) return n > 1;
3     if(n % 2 == 0 || n % 3 == 0) return false;
4     for (ll i = 5; i * i <= n; i += 6)
5         if (n % i == 0 || n % (i + 2) == 0)
6             return false;
7     return true;
8 }
9 ll phi(ll x) {
10     ll ans = x;
11     for(ll i = 2; i * i <= x; i++) {
12         if(x % i == 0) {
13             while(x % i == 0) x /= i;
14             ans -= ans / i;
15         }
16     }
17     if(x > 1) ans -= ans / x;
18     return ans;
19 }
20
21 template<int mod>
22 int fp(int b, int p) {
23     int res = 1;
24     while(p) {
25         if(p & 1) res = int(res * 1LL * b % mod);
26         b = int(b * 1LL * b % mod), p >>= 1;
27     }
28     return res;
29 }
30 template<int mod>
31 inline int add(int x, int y) { return x + y >= mod? x + y - mod: x + y; }
32 template<int mod>
33 inline int mul(int x, int y) { return int((x * 1LL * y) % mod); }
34
35 array<ll, 3> eGcd(ll a, ll b) {
36     if (b == 0) return {a, 1, 0};
37     auto [g, x1, y1] = eGcd(b, a % b);
38     return {g, y1, x1 - (a / b) * y1};
39 }
40 array<ll, 2> CRT(ll a1, ll m1, ll a2, ll m2) {
41     a1 = (a1 % m1 + m1) % m1, a2 = (a2 % m2 + m2) % m2;
42     auto [g, x, y] = eGcd(m1, m2);
43     if ((a2 - a1) % g) return {-1, -1};
44     ll lcm = m1 / g * m2;
45     ll m = m2 / g;
46     ll k = ((a2 - a1) / g % m + m) % m;
47     k = (k * (x % m + m) % m) % m;
48     ll res = (a1 + m1 * k) % lcm;
49     if (res < 0) res += lcm;
50     return {res, lcm};
51 }
52 ll inv(ll a, ll m = mod) {
53     ll x = 1, x1 = 0, q, t, b = m;
54     while(b) {
55         q = a / b;
56         a -= q * b, t = a, a = b, b = t;
57         x -= q * x1, t = x, x = x1, x1 = t;
58     }
59     assert(a == 1);
60     return (x + m) % m;
61 }
62 BSGS (Baby-Step Giant-Step) for discrete logarithm, with support for non-coprime cases. Let's break it down carefully
63 ll BSGS(ll a, ll b, ll p) {
64     a %= p, b %= p;
65     if(b == 1 || p == 1) return 0;
66     if(a == 0) return b == 0? 1: -1;
67     int add = 0;
68     ll g, tmp = 1;
69     while ((g = gcd(a, p)) > 1) {
70         if(b % g) return -1;
71         p /= g, b /= g, tmp = tmp * (a / g) % p, ++add;
72         if(tmp == b) return add;
73     }
74     b = b * inv(tmp, p) % p;
75     int n = (int)sqrtl(p) + 1;
76     unordered_map<ll, int> mp;
77     for (ll q = 0, cur = 1; q <= n; ++q)
78         mp.emplace(cur, q), cur = cur * a % p;
79     ll an = 1;
```



```

80     for (ll i = 0; i < n; ++i) an = an * a % p;
81     an = inv(an, p);
82     for (ll i = 0, cur = b; i <= n; ++i) {
83         auto it = mp.find(cur);
84         if(it != mp.end()) return i * n + it->second + add;
85         cur = cur * an % p;
86     }
87     return -1;
88 }
89 SUMm(n) = (1^m + 2^m ... n^m) % mod
90 int sumNPowerM(int n, int m) {
91     int k = m + 3;
92     vector<int> res(k);
93     for(int i = 1; i < k; i++) res[i] = (res[i - 1] + fp(i, m)) % mod;
94     if(n < k) return res[n];
95     int facK = k;
96     vector<int> p(k); p[0] = 1;
97     for(int i = 1; i < k; i++) {
98         p[i] = int(p[i - 1] * 1LL * (n - i) % mod);
99         facK = int(facK * 1LL * i % mod);
100    }
101    vector<int> inv(k + 1), s(k + 1);
102    inv[k] = fp(facK, mod - 2), s[k] = 1;
103    for(int i = k - 1; i >= 0; i--) {
104        s[i] = int(s[i + 1] * 1LL * (n - i) % mod);
105        inv[i] = int(inv[i + 1] * 1LL * (i + 1) % mod);
106    }
107    int ans = 0;
108    for(int i = 1; i < k; i++) {
109        // int cur = res[i];
110        // for(int j = 1; j < k; j++) {
111        //     if(i == j) continue;
112        //     cur = int(cur * 1LL * (n - j) % mod);
113        //     cur = int(cur * 1LL * fp(abs(i - j), mod - 2) % mod);
114        // }
115        // if((k - i + 1) & 1) cur = (mod - cur) % mod;
116        int cur = int(res[i] * 1LL * p[i - 1] % mod * s[i + 1] % mod * inv[i - 1] % mod * inv[k - i - 1] % mod);
117        if((k - i + 1) & 1) cur = (mod - cur) % mod;
118        ans = (ans + cur) % mod;
119    }
120    return ans;
121 }
122 compute primitive root
123 void primitiveRoot() {
124     int phi = mod - 1;
125     vector<int> fac;
126     for(int i = 2; i * 1LL * i < phi; i++) {
127         if(phi % i == 0) {
128             fac.push_back(i);
129             while(phi % i == 0) phi /= i;
130         }
131     }
132     if(phi > 1) fac.push_back(phi);
133
134     for(int g = 2; g < mod; g++) {
135         for(int pr : fac) if(fp(g, (mod - 1) / pr) == 1)
136             goto bad;
137         cout << "const int root = " << g << ";\n";
138         cout << "const int invRoot = " << fp(g, mod - 2) << ";\n";
139         return;
140     bad:;
141     }
142 }
143
144 ## kth_permutation
145 Given n and k, find the k-th permutation (0-based) of numbers 1..n in lexicographical order.
146
147 ## permutation_index
148 Given a permutation p of size n, return its 0-based index in lexicographical order.
149 ll fac[21];
150 void preFac() {
151     fac[0] = 1;
152     for(int i = 1; i <= 20; ++i)
153         fac[i] = fac[i - 1] * i;
154 }
155
156 vector<int> kth_permutation(int n, ll k) {
157     vector<int> a(n), ans;
158     iota(a.begin(), a.end(), 1);
159     for (int i = n; i >= 1; i--) {
160         ll f = fac[i - 1];

```

```

161         ans.push_back(a[k / f]);
162         a.erase(a.begin() + k / f);
163         k %= f;
164     }
165     return ans;
166 }
167
168 ll permutation_index(vector<int>& p) {
169     int n = int(p.size());
170     vector<int> a(n);
171     iota(a.begin(), a.end(), 1);
172
173     ll k = 0;
174     for (int i = 0; i < n; ++i) {
175         int j = int(find(a.begin(), a.end(), p[i]) - a.begin());
176         k += j * fac[n - 1 - i];
177         a.erase(a.begin() + j);
178     }
179     return k;
180 }

```

0.6.12 Lehmer

```

1  Number of prime numbers | n, Used when n | 1e12
2
3  Run in  $O(n^{2/3})$ 
4  const int N = 2e7 + 10, nPrimes = 1270608, phiN = 100000, phiK = 100;
5  int pc, prm[nPrimes], pref[N], dpPhi[phiN][phiK];
6
7  auto auto_ = [] {
8      uint64_t f[(N >> 6) + 1]{};
9      for(int i = 2; i < N; i++) {
10         if(!(f[i >> 6] >> (i & 63) & 1)) prm[pc++] = i;
11         pref[i] = pc;
12         for(int j = 0, k; (k = i * prm[j]) < N; j++) {
13             f[k >> 6] |= 1ULL << (k & 63);
14             if(i % prm[j] == 0) break;
15         }
16     }
17     for(int n = 0; n < phiN; n++) dpPhi[n][0] = n;
18     for(int n = 0; n < phiN; n++) {
19         for(int k = 1; k < phiK; k++) {
20             dpPhi[n][k] = dpPhi[n][k - 1] - dpPhi[n / prm[k - 1]][k - 1];
21         }
22     }
23     return 0;
24 }();
25
26 // Legendre's Phi Function
27 // Number of positive integers n that are not divisible by any of the first k primes
28 ll lPhi(ll n, int k) {
29     if(n < phiN && k < phiK) return dpPhi[n][k];
30     if(k == 1) return (n + 1) >> 1;
31     if(prm[k - 1] >= n) return 1;
32     return lPhi(n, k - 1) - lPhi(n / prm[k - 1], k - 1);
33 }
34
35 // Number of prime numbers n, Used when n 1e12
36 ll lehmer(ll n) {
37     if(n < N) return pref[n];
38     ll w, res;
39     int b = int(sqrtl(n)), c = int(lehmer( ll(cbrtl(n)) )), a = int(lehmer( ll(sqrtl(b)) ));
40     b = int(lehmer(b));
41     res = lPhi(n, a) + ((b + a - 2LL) * (b - a + 1) >> 1);
42     for (int i = a, j, lim; i < b; i++) {
43         w = n / prm[i];
44         lim = int(lehmer( int(sqrtl(w)) ));
45         res -= lehmer(w);
46         if(i <= c) for(j = i; j < lim; j++) res += j - lehmer(w / prm[j]);
47     }
48     return res;
49 }

```

0.6.13 Matrices

```
1  const int mod = 1'000'000'007;
2
3  template <class T>
4  struct matrix {
5  private:
6      vector<T> a;
7      int n, m;
8      static T modInverse(T a) {
9          T x = 1, x1 = 0, q, t, b = mod;
10         while(b) q = a / b, a -= q * b, t = a, a = b, b = t, x -= q * x1, t = x, x = x1, x1 = t;
11         assert(a == 1);
12         return (x + mod) % mod;
13     }
14 public:
15     matrix(int n, int m) : n(n), m(m), a(n * m, T(0)) {}
16
17     static matrix identity(int n) {
18         matrix res(n, n);
19         for (int i = 0; i < n; i++) res[i][i] = 1;
20         return res;
21     }
22
23     T* operator[](int i) { return a.data() + i * m; }
24     const T* operator[](int i) const { return a.data() + i * m; }
25
26     matrix operator+(const matrix& B) const {
27         assert(n == B.n && m == B.m);
28         auto res = *this;
29         for (int i = 0; i < n * m; i++) {
30             res.a[i] += B.a[i];
31             if(res.a[i] >= mod) res.a[i] -= mod;
32         }
33         return res;
34     }
35     matrix operator-(const matrix& B) const {
36         assert(n == B.n && m == B.m);
37         auto res = *this;
38         for (int i = 0; i < n * m; i++) {
39             res.a[i] -= B.a[i];
40             if(res.a[i] < 0) res.a[i] += mod;
41         }
42         return res;
43     }
44
45     matrix operator*(const matrix& B) const {
46         assert(m == B.n);
47         matrix<T> C(n, B.m);
48         for (int i = 0; i < n; i++) {
49             vector<uint64_t> Ci(B.m);
50             for (int k = 0; k < m; k++) {
51                 T aik = (*this)[i][k];
52                 if (!aik) continue;
53                 const auto &Bk = B[k];
54                 for (int j = 0; j < B.m; j++) Ci[j] += static_cast<uint64_t>(aik) * Bk[j];
55                 if(!(k & 15)) for(int j = 0; j < B.m; j++) Ci[j] %= mod;
56             }
57             for(int j = 0; j < B.m; j++) C[i][j] = Ci[j] % mod;
58         }
59         return C;
60     }
61
62     matrix operator^(int64_t exp) const {
63         assert(n == m);
64         auto res = identity(n), base = *this;
65         while (exp > 0) {
66             if (exp & 1) res = res * base;
67             base = base * base;
68             exp >>= 1;
69         }
70         return res;
71     }
72     [[nodiscard]] matrix inverse() const {
73         assert(n == m);
74         auto res = identity(n), tmp = *this;
75
76         for (int i = 0; i < n; i++) {
77             int pivot = i;
78             for (int j = i; j < n; j++) {
79                 if(tmp[j][i]) {
80                     pivot = j;
```

```

81         break;
82     }
83 }
84 assert(tmp[pivot][i]);
85 if (i ^ pivot) {
86     for (int j = 0; j < n; j++) {
87         swap(tmp[i][j], tmp[pivot][j]);
88         swap(res[i][j], res[pivot][j]);
89     }
90 }
91 T invP = modInverse(tmp[i][i]);
92 for (int j = 0; j < n; j++) {
93     tmp[i][j] = static_cast<T>((1LL * tmp[i][j] * invP) % mod);
94     res[i][j] = static_cast<T>((1LL * res[i][j] * invP) % mod);
95 }
96 for (int j = 0; j < n; j++) {
97     if (i != j && tmp[j][i] != 0) {
98         T factor = tmp[j][i];
99         for (int k = 0; k < n; k++) {
100             tmp[j][k] = static_cast<T>((tmp[j][k] - 1LL * factor * tmp[i][k]) % mod);
101             if (tmp[j][k] < 0) tmp[j][k] += mod;
102             res[j][k] = static_cast<T>((res[j][k] - 1LL * factor * res[i][k]) % mod);
103             if (res[j][k] < 0) res[j][k] += mod;
104         }
105     }
106 }
107 }
108 return res;
109 }
110 matrix operator/(const matrix& B) const {
111     return *this * B.inverse();
112 }
113 };

```

0.6.14 Notes

```

1  ## mod rule
2  a % m == b
3  a and m not coprime
4  g = gcd(a, m)
5  (a / g) % (m / g) = b / g
6  a^x % m == b
7  a and m not coprime
8  g = gcd(a, m)
9  (a^(x-1) * (a / g)) % (m / g) = b / g
10 ## be careful if a and m not co prime you should take mod like that
11  $a^{(\text{power} \% \phi(m))} \% m$ 
12 void MOD(int x, int mod) { return x < mod? x: x % mod + mod; } or
13 void MOD(int x, int mod) { return x < mod? x: (x - log2(mod)) % mod + log2(mod); }
14 ## phi methods
15 for all 1 <= i <= n, sum[n] += gcd(i, n)
16 sum[a * b] = sum[a] * sum[b] if and only if a and b are coprime
17 sum of numbers 1 <= i <= n | if(gcd(n, i) == d): sum += i, cnt++
18 sum = n * phi(n / d) / 2
19 cnt = phi(n / d)
20 ## Counting
21 ### 1
22 for(int i = 0; i < (1 << n); i++) sum += 1 << popcnt(i); assert(sum == pow(3, n))
23
24 ### 2 count balanced brackets
25 r=n/2 || or r = number of opened brackets
26 nCr(n, r) - nCr(n, r-1)
27 ### 3 different n*n grids whose each square have m colors, if possible to rotate one of them so that they look the same
28  $\hookrightarrow$  then they same
29 t = n * n;
30 total = (fp(m, t)
31 + fp(m, (t + 1) / 2)
32 + 2 * fp(m, (t / 4) + (n % 2))) % mod;
33 total = mul(total, fp(4, mod - 2));
34 #### 4 sum of divisors
35 prime^power * prime2^power2 * ...
36 ((prime^(power + 1) - 1) / (prime - 1)) * ((prime2^(power2 + 1) - 1) / (prime2 - 1)) * ...
37 #### 5
38 sum from 1 to n: i * nCr(n, i) = n * (1LL << (n - 1))
39 #### 6 count the number of different necklaces that consist of n pearls and each pearl has m possible colors.
40 #### Two necklaces are considered to be different if it is not possible to rotate one of them so that they look the
41  $\hookrightarrow$  same.
42 for (int i = 0; i < n; ++i)
43     ans = add(ans, fp(m, __gcd(n, i)));
44 ans = mul(ans, fp(n, mod - 2));
45
46 ---
47 biggest divisors
48 735134400 1344 => 2^6 3^3 5^2 7 11 13 17
49 73513440 768
50
51 ### 6 masks in increasing order with exactly x bits.
52
53 for (int mask = (1 << x) - 1; mask <= n; ) {
54
55     // use mask here
56     cout << mask << '\n';
57
58     // Gosper's hack (next mask with same popcount)
59     int c = mask & -mask;
60     int r = mask + c;
61     mask = ((r ^ mask) >> 2) / c | r;
62
63     if (mask > n) break;
64 }
65
66 ### 7 some probability rules
67
68 #### sum inf E[x]
69 0 <= p <= 1
70 sum i = 1 to inf
71     power(1 - p, i - 1) * p * i
72 equal (1 / p)
73 #### sum inf E[x^2]
74 0 <= p <= 1
75 sum i = 1 to inf
76     power(1 - p, i - 1) * p * i^2
77 equal (2 - p) / p^2
78 ### global E[X(X+1)(X+2)...(X+k-1)]
79 k! / power(p, k)

```

0.6.15 NTT

```

1  Number Theoretic Transform
2  template<int mod>
3  int fp(int b, int p) {
4      int res = 1;
5      while(p) {
6          if(p & 1) res = int(res * 1LL * b % mod);
7          b = int(b * 1LL * b % mod), p >>= 1;
8      }
9      return res;
10 }
11 template<int mod>
12 inline int add(int x, int y) { return x + y >= mod? x + y - mod: x + y; }
13 template<int mod>
14 inline int mul(int x, int y) { return int((x * 1LL * y) % mod); }
15
16 template<const int mod, const int root>
17 struct NTT {
18     vector<int> mulMod(vector<int> a, vector<int> b) {
19         int n = 1;
20         while(n < a.size() + b.size() - 1) n <<= 1;
21         a.resize(n), b.resize(n), ntt(a), ntt(b);
22         for(int i = 0; i < n; i++) a[i] = mul<mod>(a[i], b[i]);
23         intt(a);
24         return a;
25     }
26     void ntt(vector<int> &a) {
27         int n = int(a.size());
28         static vector<int> rev;
29         if(rev.size() != n) {
30             int k = __builtin_ctz(n); rev.assign(n, 0);
31             for(int i = 0; i < n; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));
32         }
33         prepare_roots(n);
34         for(int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
35         for (int k = 1; k < n; k <<= 1) {
36             for (int i = 0; i < n; i += 2 * k) {
37                 for (int j = 0; j < k; j++) {
38                     int u = a[i + j], v = mul<mod>(a[i + j + k], roots[k + j]);
39                     a[i + j] = add<mod>(u, v);
40                     a[i + j + k] = add<mod>(u, mod - v);
41                 }
42             }
43         }
44     }
45     void intt(vector<int> &a) {
46         reverse(a.begin() + 1, a.end());
47         ntt(a);
48         int iv = fp<mod>(int(a.size()), mod - 2);
49         for (int &x : a) x = mul<mod>(x, iv);
50     }
51 private:
52     vector<int> roots{0, 1};
53     void prepare_roots(int n) {
54         if (roots.size() >= n) return;
55         int old_size = roots.size();
56         roots.resize(n);
57         for (int i = old_size; i < n; i <<= 1) {
58             int w = fp<mod>(root, (mod - 1) / (i << 1));
59             for (int j = i / 2; j < i; j++) {
60                 roots[2 * j] = roots[j];
61                 roots[2 * j + 1] = mul<mod>(roots[j], w);
62             }
63         }
64     }
65 };
66
67 mul Two polynomial With bad mod like  $10^9 + 7$ 
68
69 vector<int> mulBadMod(vector<int> const &a, vector<int> const &b) {
70     using i128 = __int128_t;
71     static const int m1 = 998244353, m2 = 754974721, m3 = 167772161;
72     static NTT<m1, 3> ntt0;
73     static NTT<m2, 11> ntt1;
74     static NTT<m3, 3> ntt2;
75     static auto CRT = [](i128 a1, i128 m1, i128 a2, i128 m2, i128 lcm, i128 x) {
76         return (a1 + m1 * (((a2 - a1) * x % m2 + m2) % m2)) % lcm;
77     };
78     vector<int> res[3] = {ntt0.mulMod(a, b), ntt1.mulMod(a, b), ntt2.mulMod(a, b)};
79     const i128 m12 = m1 * 1LL * m2, m123 = m12 * m3;
80     static const i128 inv1 = fp<m2>(m1 % m2, m2 - 2);

```

```

81     static const i128 inv2 = fp<m3>(int(m12 % m3), m3 - 2);
82     vector<int> ans(res[0].size());
83     for(int i = 0; i < ans.size(); i++) {
84         ans[i] = int(CRT(
85             CRT(res[0][i], m1, res[1][i], m2, m12, inv1), m12,
86             res[2][i], m3,
87             m123, inv2) % mod);
88     }
89     return ans;
90 }

```


0.6.16 polynomial

```

1  const int mod = 998244353, root = 3;
2  NTT<mod, root> ntt;
3
4  vector<int> inv{0, 1};
5  void precompute_inv(int n) {
6      int old_sz = int(inv.size());
7      if (n <= old_sz) return;
8      inv.resize(n + 1);
9      for(int i = max(2, old_sz); i <= n; i++) inv[i] = mul<mod>(mod - mod / i, inv[mod % i]);
10 }
11
12 struct poly : vector<int> {
13     void norm() { while(!empty() && !back()) pop_back(); }
14     poly(int n = 0) : vector(n) { }
15     template<class...Args> poly(Args...args): vector(std::move(args...)) { }
16     poly(const initializer_list<int> &x): vector(x.begin(), x.end()) { }
17
18     inline int coef(const int i) const { return i < size() && i >= 0? at(i): 0; }
19     bool is_zero() const { return count(begin(), end(), 0) == size(); }
20
21     poly operator+(const poly &x) const {
22         auto n = max(size(), x.size());
23         poly ans(n);
24         for(int i = 0; i < n; i++) ans[i] = add<mod>(coef(i), x.coef(i));
25         return ans.norm(), ans;
26     }
27     poly operator-(const poly &x) const {
28         auto n = max(size(), x.size());
29         poly ans(n);
30         for(int i = 0; i < n; i++) ans[i] = add<mod>(coef(i), mod - x.coef(i));
31         return ans.norm(), ans;
32     }
33     poly operator*(const poly &b) const {
34         if(is_zero() || b.is_zero()) return {};
35         auto ans = poly(ntt.mulMod(*this, b));
36         return ans.norm(), ans;
37     }
38     poly operator*(const int &x) const {
39         poly ans(size());
40         for(int i = 0; i < size(); i++) ans[i] = mul<mod>(coef(i), x);
41         return ans.norm(), ans;
42     }
43     poly operator/(const int &x) const { return (*this) * fp<mod>(x, mod - 2); }
44     poly mod_xk(int k) const { return vector(begin(), begin() + min(k, int(size()))); } // modulo by x^k
45     poly mul_xk(int k) const { // multiply by x^k
46         poly ans(*this);
47         return ans.insert(ans.begin(), k, 0), ans;
48     }
49     poly div_xk(int k) const { return vector(begin() + min(k, int(size())), end()); } // divide by x^k
50     poly substr(int l, int r) const { return vector(begin() + min(l, int(size())), begin() + min(r, int(size()))); }
51     // return mod_xk(r).div_xk(l)
52     poly derivative() const {
53         auto n = size();
54         if(n < 2) return {};
55         poly ans(n - 1);
56         for(int i = 1; i < n; i++) ans[i - 1] = mul<mod>(coef(i), i);
57         return ans.norm(), ans;
58     }
59     poly integrate() const {
60         precompute_inv(int(size()));
61         poly ans(size() + 1);
62         for(int i = 0; i < size(); i++) ans[i + 1] = mul<mod>(coef(i), inv[i + 1]);
63         return ans.norm(), ans;
64     }
65     poly inverse(int n) const { // 1 / p(x) % x^n, O(n log(n))
66         assert(!is_zero() && at(0) != 0);
67         poly ans({fp<mod>(at(0), mod - 2)});
68         for(int i = 1; i < n; i += 2) ans = (ans * 2 - ans * ans * mod_xk(2 * i)).mod_xk(2 * i);
69         return ans.mod_xk(n);
70     }
71     poly log(int n) const { // ln p(x) mod x^n
72         assert(coef(0) == 1);
73         return (derivative().mod_xk(n) * inverse(n)).integrate().mod_xk(n);
74     }
75     poly exp(int n) const { // e ^ p(x) mod x^n
76         if(is_zero()) return vector{1};
77         assert(coef(0) == 0);
78         poly res{1};
79         for(int k = 2; k <= (n << 1); k <= 1) res = (res * (poly{1} - res.log(k) + mod_xk(k))).mod_xk(k);

```

```

79     return res.mod_xk(n);
80 }
81 poly pow(int64_t e, int n) const { // slow with small power
82     if (e == 0) return poly({1}).mod_xk(n);
83     int p = 0;
84     while(p < size() && at(p) == 0) p++;
85     if (p == size() || (p > 0 && e >= n / p)) return {};
86     int shift = p * e;
87     int a0 = at(p);
88     int inv_a0 = fp<mod>(a0, mod - 2);
89     poly b = div_xk(p) * inv_a0;
90     b = (b.log(n - shift) * (int(e % mod))).exp(n - shift);
91     poly res = b * fp<mod>(a0, e % (mod - 1));
92     return res.mul_xk(shift).mod_xk(n);
93 }
94 poly power(int64_t e, int lim) const { // slow with big power
95     poly res({1}), b = *this;
96     while(e) {
97         if(e & 1) res = (res * b).mod_xk(lim);
98         b = (b * b).mod_xk(lim), e >>= 1;
99     }
100     return res;
101 }
102 };

```

0.6.17 Sieve

```

1 linear sieve Smallest Prime Factor
2 Computes (i) for all i | N in  $O(N)$ 
3 Linear Sieve for Möbius (n) in  $O(n)$ 
4 const int N = 1e5 + 1, EN = max(0, int(N / (log(N) - 1.1))) + 10;
5 int pc, prm[EN];
6 int spf[N];
7 int phi[N];
8 int8_t mob[N];
9
10 auto pre_Sieve = []() {
11     mob[1] = 1;
12     for (int i = 2; i < N; i++) {
13         if (!spf[i]) spf[i] = prm[pc++] = i, mob[i] = -1, phi[i] = i - 1;
14         for (int j = 0; i * prm[j] < N; j++) {
15             spf[i * prm[j]] = prm[j];
16             if (spf[i] == prm[j]) {
17                 phi[i * prm[j]] = phi[i] * prm[j];
18                 break;
19             }
20             mob[i * prm[j]] = int8_t(-mob[i]);
21             phi[i * prm[j]] = phi[i] * (prm[j] - 1);
22         }
23     }
24     return 0;
25 }();
26
27 Prime Factorization using SPF  $O(\log(n))$ 
28
29 Generate All Divisors  $O(\text{number of divisors})$ 
30 auto factors(int n) {
31     vector<array<int, 2>> res;
32     if(n < 2) return res;
33     int p = spf[n];
34     while(p > 1) {
35         res.push_back({p, 0});
36         while(n % p == 0) n /= p, res.back()[1]++;
37         p = spf[n];
38     }
39     return res;
40 }
41 auto getDivisors(int _n) {
42     auto _fac = factors(_n);
43     int cnt = 1;
44     for(auto [pr, pw] : _fac) cnt *= pw + 1;
45     vector<int> res(1, 1); res.reserve(cnt);
46     for(auto [pr, pw] : _fac)
47         for(int i = int(res.size()) - 1; i >= 0; i--)
48             for(int b = pr, j = 0; j < pw; j++, b *= pr)
49                 res.push_back(res[i] * b);
50     sort(res.begin(), res.end());
51     return res;
52 }
53
54 get Divisors in  $O(n \log(n))$ 
55 const int N = 1e6 + 1, sumD = 13970034;
56 int cnt[N], start[N], all[sumD];
57
58 auto pre_compute = []{
59     for(int i = 1; i < N; ++i)
60         for(int j = i; j < N; j += i)
61             cnt[j]++;
62     for(int i = 1; i < N; ++i)
63         start[i] = start[i - 1] + cnt[i], cnt[i] = 0;
64     for(int i = 1; i < N; ++i)
65         for(int j = i; j < N; j += i)
66             all[start[j - 1] + cnt[j]++] = i;
67     // divisors of x
68     // L = start[x - 1]
69     // R = start[x]
70     // for(int i = L; i < R; i++) d = all[i]
71     return 0;
72 }();
73
74 30-wheel sieve optimization
75
76 can compute primes up to 1e9 in 500ms
77
78 vector<int> sieve(const int BN, const int Q = 17, const int L = 1 << 15) {
79     static const int rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
80     struct P {

```

```

81     P(int p) : p(p) {}
82
83     int p;
84     int pos[8];
85 };
86 auto approx_prime_count = [](const int N) -> int {
87     return N > 60184 ? N / (log(N) - 1.1)
88         : max(1., N / (log(N) - 1.11)) + 1;
89 };
90
91 const int v = sqrt(BN), vv = sqrt(v);
92 vector<bool> isp(v + 1, true);
93 for (int i = 2; i <= vv; ++i)
94     if (isp[i]) {
95         for (int j = i * i; j <= v; j += i) isp[j] = false;
96     }
97
98 const int rsize = approx_prime_count(BN + 30);
99 vector<int> primes = {2, 3, 5};
100 int psize = 3;
101 primes.resize(rsize);
102
103 vector<P> sprimes;
104 size_t pbeg = 0;
105 int prod = 1;
106 for (int p = 7; p <= v; ++p) {
107     if (!isp[p]) continue;
108     if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
109     auto pp = P(p);
110     for (int t = 0; t < 8; ++t) {
111         int j = (p <= Q) ? p : p * p;
112         while (j % 30 != rs[t]) j += p << 1;
113         pp.pos[t] = j / 30;
114     }
115     sprimes.push_back(pp);
116 }
117
118 vector<unsigned char> pre(prod, 0xFF);
119 for (size_t pi = 0; pi < pbeg; ++pi) {
120     auto pp = sprimes[pi];
121     const int p = pp.p;
122     for (int t = 0; t < 8; ++t) {
123         const unsigned char m = ~(1 << t);
124         for (int i = pp.pos[t]; i < prod; i += p) pre[i] &= m;
125     }
126 }
127
128 const int block_size = (L + prod - 1) / prod * prod;
129 vector<unsigned char> block(block_size);
130 unsigned char *pblock = block.data();
131 const int M = (BN + 29) / 30;
132 for (int beg = 0; beg < M; beg += block_size, pblock += block_size) {
133     int end = min(M, beg + block_size);
134     for (int i = beg; i < end; i += prod) {
135         copy(pre.begin(), pre.end(), pblock + i);
136     }
137     if (beg == 0) pblock[0] &= 0xFE;
138     for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
139         auto &pp = sprimes[pi];
140         const int p = pp.p;
141         for (int t = 0; t < 8; ++t) {
142             int i = pp.pos[t];
143             const unsigned char m = ~(1 << t);
144             for (; i < end; i += p) pblock[i] &= m;
145             pp.pos[t] = i;
146         }
147     }
148     for (int i = beg; i < end; ++i) {
149         for (int m = pblock[i]; m > 0; m &= m - 1) {
150             primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
151         }
152     }
153 }
154 assert(psize <= rsize);
155 while (psize > 0 && primes[psize - 1] > BN) --psize;
156 primes.resize(psize);
157 return primes;
158 }

```

0.6.18 XorBasis

linear XOR basis (or binary linear basis) implementation. It's used to represent a set of integers so that any number that can be formed by XORing some subset of the set can be efficiently queried

```
template<const int Log = 30, typename T = int>
struct basis {
    int sz = 0;
    array<T, Log> a{};
    void add(T x) {
        if(sz == Log) return;
        int i;
        while(x) {
            if(!a[i = __lg(x)]) return sz++, void(a[i] = x);
            x ^= a[i];
        }
    }
    T reduce(T x) {
        if(sz == Log) return 0;
        T res = 0;
        int i;
        while(x) {
            if(a[i = __lg(x)]) x ^= a[i];
            else res |= T(1) << i, x ^= T(1) << i;
        }
        return res;
    }
    bool find(T x) {
        if(sz == Log) return true;
        int i;
        while(x) {
            if(a[i = __lg(x)]) x ^= a[i];
            else return false;
        }
        return true;
    }
    void clear() {
        if(sz) a.fill(0), sz = 0;
    }
    T getMax(T r = 0) {
        for(int i = Log - 1; i >= 0; i--) r = max(r ^ a[i], r);
        return r;
    }
    T find_k(size_t k) { // index-0
        assert(k < 1ULL << sz);
        T curr = 0;
        for(int i = Log - 1, b = sz - 1; i >= 0; i--) {
            if(a[i]) {
                if((k >> b & 1) ^ (curr >> i & 1)) curr ^= a[i];
                b--;
            }
        }
        return curr;
    }
    friend basis operator+(basis const &l, basis const &r) {
        if(l.sz == Log) return l;
        if(r.sz == Log) return r;
        auto res = l;
        for(int i = 0; i < Log; i++) if(r.a[i]) res.add(r.a[i]);
        return res;
    }
};
```

Prefix Linear Basis

```
template<const int Log = 30, typename T = int>
struct prefix_basis {
    array<T, Log> a{};
    array<int, Log> pos{};

    prefix_basis() { pos.fill(-1); }

    void add(T x, int idx) {
        int i;
        while(x) {
            if(!a[i = __lg(x)]) return a[i] = x, void(pos[i] = idx);
            if(pos[i] < idx) swap(pos[i], idx), swap(a[i], x);
            x ^= a[i];
        }
    }
    void clear() { a.fill(0), pos.fill(-1); }

    int size(int L = 0) {
```

```

80     return count_if(pos.begin(), pos.end(), [&L](int x) { return x >= L; });
81 }
82
83 bool find(T x, int L = 0) {
84     int i;
85     while(x) {
86         if(pos[i = __lg(x)] >= L) x ^= a[i];
87         else return false;
88     }
89     return true;
90 }
91
92 T getMax(T r = 0, int L = 0) {
93     for(int i = Log - 1; i >= 0; i--)
94         if(pos[i] >= L) r = max(r, r ^ a[i]);
95     return r;
96 }
97
98 T reduce(T x, int L = 0) {
99     T res = 0;
100    int i;
101    while(x) {
102        if(pos[i = __lg(x)] >= L) x ^= a[i];
103        else res |= T(1) << i, x ^= T(1) << i;
104    }
105    return res;
106 }
107
108 T find_k(int L, size_t k) {
109     int sz = size(L);
110     assert(k < 1ULL << sz);
111     T curr = 0;
112     for(int i = Log - 1, b = sz - 1; i >= 0; i--) {
113         if(pos[i] >= L) {
114             if(((k >> b & 1) ^ (curr >> i & 1))) curr ^= a[i];
115             b--;
116         }
117     }
118     return curr;
119 }
120
121 friend prefix_basis operator+(prefix_basis const &l, prefix_basis const &r) {
122     auto res = l;
123     for(int i = 0; i < Log; i++) {
124         if(r.pos[i] != -1) res.add(r.a[i], r.pos[i]);
125     }
126     return res;
127 }
128 };

```

0.7 Strings

0.7.1 Corasick

```
1 struct corasick {
2     struct node {
3         int fail = -1;
4         array<int, 26> nxt{};
5         vector<int> idx;
6         explicit node() { nxt.fill(-1); }
7     };
8     vector<node> tr;
9     explicit corasick() : tr(1) { }
10    int add(const string &s, int id) {
11        int x = 0;
12        for (char ch: s) {
13            if(tr[x].nxt[ch - 'a'] == -1)
14                tr[x].nxt[ch - 'a'] = int(tr.size()), tr.emplace_back();
15            x = tr[x].nxt[ch - 'a'];
16        }
17        if(tr[x].idx.empty()) tr[x].idx.push_back(id);
18        return tr[x].idx.back();
19    }
20    void build() {
21        queue<int> q;
22        tr[0].fail = 0;
23        for (int c = 0; c < 26; c++) {
24            int u = tr[0].nxt[c];
25            if(u != -1) tr[u].fail = 0, q.push(u);
26            else tr[0].nxt[c] = 0;
27        }
28        while (!q.empty()) {
29            int v = q.front();
30            q.pop();
31            for (int c = 0; c < 26; c++) {
32                int u = tr[v].nxt[c];
33                if (u != -1) {
34                    tr[u].fail = tr[tr[v].fail].nxt[c];
35                    tr[u].idx.reserve(tr[u].idx.size() + tr[tr[u].fail].idx.size());
36                    for(int x : tr[tr[u].fail].idx) tr[u].idx.push_back(x);
37                    q.push(u);
38                }
39                else tr[v].nxt[c] = tr[tr[v].fail].nxt[c];
40            }
41        }
42    }
43 };
44
45 Dynamic corasick
46 struct dynamic_corasick {
47     int Log;
48     vector<vector<string>> li;
49     vector<corasick> ac;
50     dynamic_corasick(int Log = 20) : Log(Log), li(Log), ac(Log) { }
51
52     void add(string const &s) {
53         vector<string> a{s};
54         for(int ip = 0; ip < Log; ip++) {
55             if(!li[ip].empty()) {
56                 a.insert(a.end(), li[ip].begin(), li[ip].end());
57                 li[ip].clear(), ac[ip] = corasick();
58             }
59             else {
60                 swap(a, li[ip]);
61                 for(auto &st : li[ip]) ac[ip].add(st);
62                 ac[ip].build();
63                 break;
64             }
65         }
66     }
67     int64_t count(string &s) {
68         int64_t ans = 0;
69         for(int ip = 0; ip < Log; ip++) {
70             if(li[ip].empty()) continue;
71             int x = 0;
72             for(char c : s) {
73                 x = ac[ip].tr[x].nxt[c - 'a'];
74                 ans += ac[ip].tr[x].cnt;
75             }
76         }
77         return ans;
78     }
79 }
```


0.7.2 DynamicSuffixArray

```
1 DynamicSuffixArray
2 ##### Time Complexity:  $O(\log^2 N)$  per modification (push/pop) because the  $O(\log N)$  Treap traversal calls  $O(\log N)$ 
   ↳ getIndex for comparisons.
3 #pragma GCC optimize("O3,unroll-loops")
4 uint64_t fast_rand() {
5     static uint64_t x = 88172645463325252ULL;
6     x ^= x << 13; x ^= x >> 7; x ^= x << 17;
7     return x;
8 }
9
10 class DynamicSuffixArray {
11     struct Node {
12         int l = 0, r = 0, p = 0, sz = 1;
13         uint64_t y = fast_rand();
14     };
15     vector<char> s;
16     vector<Node> tr;
17     int root_ = 0;
18
19     void upd(int x) { if(x) tr[x].sz = 1 + tr[tr[x].l].sz + tr[tr[x].r].sz; }
20     int setP(int x, int p) {
21         if(x) tr[x].p = p;
22         return x;
23     }
24     int merge(int L, int R) {
25         if (!L || !R) return L + R;
26         if (tr[L].y < tr[R].y) return setP(tr[R].l = merge(L, tr[R].l), R), upd(R), R;
27         setP(tr[L].r = merge(tr[L].r, R), L);
28         return upd(L), L;
29     }
30     int getIndex(int x) {
31         if (!x) return 0;
32         int i = tr[tr[x].l].sz + 1;
33         while (tr[x].p) {
34             int p = tr[x].p;
35             if (x == tr[p].r) i += tr[tr[p].l].sz + 1;
36             x = p;
37         }
38         return i;
39     }
40     int compare(int id1, int id2, int i) {
41         int cmp = s[id1] - s[id2];
42         return cmp == 0 ? i - getIndex(id2 - 1) : cmp;
43     }
44     void split(int x, int id, int i, int& L, int& R) {
45         if (!x) { L = R = 0; return; }
46         if (compare(id, x, i) < 0) {
47             split(tr[x].l, id, i, L, tr[x].l);
48             setP(tr[x].l, x); R = x;
49         } else {
50             split(tr[x].r, id, i, tr[x].r, R);
51             setP(tr[x].r, x); L = x;
52         }
53         upd(x);
54     }
55     int insert(int x, int id, int i1) {
56         if (!x) return id;
57         if (tr[x].y < tr[id].y) {
58             split(x, id, i1, tr[id].l, tr[id].r);
59             setP(tr[id].l, id); setP(tr[id].r, id);
60             return upd(id), id;
61         }
62         if (compare(id, x, i1) < 0) setP(tr[x].l = insert(tr[x].l, id, i1), x);
63         else setP(tr[x].r = insert(tr[x].r, id, i1), x);
64         tr[x].sz++;
65         return x;
66     }
67 public:
68     DynamicSuffixArray(int max_capacity = 100000) {
69         s.reserve(max_capacity + 10);
70         tr.reserve(max_capacity + 10);
71         s.push_back(0);
72         tr.emplace_back();
73         tr[0].sz = 0;
74     }
75     void push_front(char c) {
76         s.push_back(c);
77         tr.emplace_back();
78         int id = size();
```

```

79     root_ = insert(root_, id, getIndex(id - 1));
80 }
81 char pop_front() {
82     if (size() <= 0) return 0;
83     int id = size(), p = tr[id].p, tmp = merge(tr[id].l, tr[id].r);
84     setP(tmp, p);
85     if (p) {
86         (tr[p].l == id ? tr[p].l : tr[p].r) = tmp;
87         while (p) tr[p].sz--, p = tr[p].p;
88     } else root_ = tmp;
89
90     tr.pop_back();
91     char c = s.back();
92     s.pop_back();
93     return c;
94 }
95 int match(const string& q, int x = -1, bool L = false, bool R = false) {
96     if (x == -1) x = root_;
97     if (!x) return 0;
98     if (L && R) return tr[x].sz;
99     for (size_t i = 0; i < q.size(); ++i) {
100         if (q[i] > s[x - i]) return match(q, tr[x].r, false, R);
101         if (q[i] < s[x - i]) return match(q, tr[x].l, L, false);
102     }
103     return 1 + match(q, tr[x].l, L, true) + match(q, tr[x].r, true, R);
104 }
105 int size() { return (int)s.size() - 1; }
106 char at(int i) { return s[size() - i]; }
107 };

```

0.7.3 Eertree

```

1  highly optimized implementation of a palindromic tree (eertree) with support for adding/removing letters from both
   ↪ ends of a string
2
3  template<int sigma = 26, char mch = 'a'>
4  struct eertree {
5      eertree(size_t q) {
6          q += 2;
7          cnt = len = par = qlink = vector(q, 0);
8          to.resize(q);
9          link.resize(q);
10         ranges::fill(link[0], 1);
11         qlink[0] = 1;
12         len[1] = -1;
13     }
14
15     template<bool back = 1>
16     static int get(auto const &d, size_t idx) {
17         if (idx >= size(d)) { return -1; } else { return back ? rbegin(d)[idx] : d[idx]; }
18     }
19
20     template<bool back = 1>
21     static void push(auto &d, auto c) { back ? d.push_back(c) : d.push_front(c); }
22
23     template<bool back = 1>
24     static void pop(auto &d) { back ? d.pop_back() : d.pop_front(); }
25
26     template<bool back = 1>
27     void add_letter(char c) {
28         c -= mch;
29         push<back>(s, c);
30         int pre = get<back>(states, 0);
31         int last = make_to<back>(pre, c);
32         active += !(cnt[last]++);
33         int D = 2 + len[pre] - len[last];
34         while (D + len[pre] <= len[last]) {
35             pop<back>(states);
36             if (!empty(states)) {
37                 pre = get<back>(states, 0);
38                 D += get<back>(diffs, 0);
39                 pop<back>(diffs);
40             } else { break; }
41         }
42         if (!empty(states)) { push<back>(diffs, D); }
43         push<back>(states, last);
44     }
45
46     template<bool back = 1>
47     void pop_letter() {
48         int last = get<back>(states, 0);
49         active -= (--cnt[last]);
50         pop<back>(states);
51         pop<back>(s);
52         array cands = {pair{qlink[last], len[last] - len[qlink[last]]}, pair{par[last], 0}};
53         for (auto [state, diff] : cands) {
54             if (empty(states)) {
55                 states = {state};
56                 diffs = {diff};
57             } else {
58                 int D = get<back>(diffs, 0) - diff;
59                 int pre = get<back>(states, 0);
60                 if (D + len[state] > len[pre]) {
61                     push<back>(states, state);
62                     pop<back>(diffs);
63                     push<back>(diffs, D);
64                     push<back>(diffs, diff);
65                 }
66             }
67         }
68         pop<back>(diffs);
69     }
70
71     void add_letter(char c, bool back) { back ? add_letter<1>(c) : add_letter<0>(c); }
72     void pop_letter(bool back) { back ? pop_letter<1>() : pop_letter<0>(); }
73     int distinct() { return active; }
74
75     template<bool back = 1>
76     int maxlen() { return len[get<back>(states, 0)]; }
77
78 private:
79     vector<array<int, sigma>> to, link;

```

```

80     vector<int> len, qlink, par, cnt;
81     deque<char> s;
82     deque<int> states = {0}, diffs;
83     int sz = 2, active = 0;
84
85     template<bool back = 1>
86     int make_to(int last, int c) {
87         if (c != get<back>(s, len[last] + 1)) { last = link[last][c]; }
88         if (!to[last][c]) {
89             int u = to[link[last][c]][c];
90             qlink[sz] = u;
91             link[sz] = link[u];
92             link[sz][get<back>(s, len[u])] = u;
93             len[sz] = len[last] + 2;
94             par[sz] = last;
95             to[last][c] = sz++;
96         }
97         return to[last][c];
98     }
99 };

```

0.7.4 KMP

```
1 auto KMP(string const &s) {  
2     vector<int> p(s.size());  
3     for(int i = 1, j = 0; i < s.size(); i++) {  
4         while(j && s[i] ^ s[j]) j = p[j - 1];  
5         p[i] = j += s[i] == s[j];  
6     }  
7     return p;  
8 }
```

0.7.5 LargestLexSubstring

```
1 string largestLexSubstring(const string &s) {
2     int n = int(s.size());
3     int i = 0, j = 1, k = 0;
4
5     while (j + k < n) {
6         if (s[i + k] == s[j + k]) k++;
7         else if (s[i + k] < s[j + k])
8             i = max(i + k + 1, j), j = i + 1, k = 0; /* change it to > if you want lowest */
9         else j = j + k + 1, k = 0;
10    }
11
12    return s.substr(i);
13 }
```

0.7.6 Manacher

```
1 get max odd len = res[2 * i]; aba -> i = b
2
3 get max even len = res[2 * i + 1]; abba -> i = first b
4 auto manacher(const string &t) {
5     string s = "%#";
6     s.reserve(t.size() * 2 + 3);
7     for(char c : t) s += c + "#";
8     s += '$';
9     // t = aabaacaabaa -> s = #a#a#b#a#a#c#a#a#b#a#a#f
10    vector<int> res(s.size());
11    for(int i = 1, l = 1, r = 1; i < s.size(); i++) {
12        res[i] = max(0, min(r - i, res[l + r - i]));
13        while(s[i + res[i]] == s[i - res[i]]) res[i]++;
14        if(i + res[i] > r) {
15            l = i - res[i];
16            r = i + res[i];
17        }
18    }
19    for(auto &i : res) i--;
20    return vector(res.begin() + 2, res.end() - 2); // a#a#b#a#a#c#a#a#b#a#a
21 }
22
23 check palindromic substring
24
25 auto man = manacher(s);
26 auto check = [&](int l, int r) {
27     return man[l + r] >= r - l + 1;
28 };
```

0.7.7 RollingHash

```
1 Highly optimized rolling hash implementation using 64-bit modular arithmetic for strings
2 using u64 = uint64_t;
3 using u32 = uint32_t;
4 const int N = 2000'001;
5 const u64 md = (1LL << 61) - 1, B = 788294215192145664ULL, IB = 285196104568472234ULL;
6 inline u64 add(u64 a, u64 b) { return a + b >= md ? a + b - md : a + b; }
7 inline u64 mul(u64 a, u64 b) {
8     u64 l1 = u32(a), h1 = a >> 32, l2 = u32(b), h2 = b >> 32;
9     u64 l = l1 * l2, m = l1 * h2 + l2 * h1, h = h1 * h2;
10    u64 ret = (l & md) + (l >> 61) + (h << 3) + (m >> 29) + (m << 35 >> 3) + 1;
11    ret = (ret & md) + (ret >> 61);
12    ret = (ret & md) + (ret >> 61);
13    return ret - 1;
14 }
15 vector<u64> PB(N), PIB(N);
16 char init = []() {
17     PB[0] = PIB[0] = 1;
18     for (int i = 1; i < N; i++) {
19         PB[i] = mul(PB[i - 1], B);
20         PIB[i] = mul(PIB[i - 1], IB);
21     }
22     return char();
23 }();
24 struct Hash : pair<u64, int> { // {code, size}
25     Hash() { }
26     Hash(const pair<u64, int> &x) : pair(x) { }
27     Hash(int x) : pair(x, 1) { }
28     Hash(const string &x) {
29         for(char c : x) *this = *(this) + c;
30     }
31     void pop_front(int x) {
32         first = add(first, md - mul(PB[--second], x));
33     }
34     void pop_back(int x) {
35         first = mul(add(first, md - x), IB), --second;
36     }
37     friend Hash operator+(const Hash &f, const Hash &o) {
38         return pair{add(mul(f.first, PB[o.second]), o.first), f.second + o.second};
39     }
40 };
41 Both forward and reverse hashing
42 struct HashRange {
43     vector<Hash> p, s;
44     HashRange(const string &t) : p(t.size()), s(t.size()) {
45         p.front() = t.front();
46         for(int i = 1; i < t.size(); i++) p[i] = p[i - 1] + t[i];
47         s.back() = t.back();
48         for(int i = int(t.size()) - 2; i >= 0; i--) s[i] = s[i + 1] + t[i];
49     }
50     Hash get(int l, int r) {
51         if(l > r) return {};
52         if(!l) return p[r];
53         return pair{add(p[r].first, md - mul(p[l - 1].first, PB[r - l + 1])), r - l + 1};
54     }
55     Hash inv(int l, int r) {
56         if(l > r) return {};
57         if(r + 1 == s.size()) return s[l];
58         return pair{add(s[l].first, md - mul(s[r + 1].first, PB[r - l + 1])), r - l + 1};
59     }
60 };
```


0.7.8 SuffixArray

```

1  O(n log(n))
2  struct suffix {
3      int n;
4      vector<int> p, c, lcp;
5      string s;
6
7      explicit suffix(string _s) : n(int(_s.size()) + 1), s(std::move(_s)), p(n), c(n), lcp(n - 1) {
8          s += char(0);
9          iota(p.begin(), p.end(), 0);
10         sort(p.begin(), p.end(), [&](int i, int j) { return s[i] < s[j]; });
11         for (int i = 1; i < n; i++) c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
12         vector<int> nc(n), np(n);
13         int k = 1;
14         while (k < n) {
15             vector<int> f(n + 1);
16             for (int i = 0; i < n; i++) p[i] = (p[i] - k + n) % n, f[c[i] + 1]++;
17             for (int i = 1; i <= n; i++) f[i] += f[i - 1];
18             for (int i = 0; i < n; i++) np[f[c[p[i]]]++] = p[i];
19             swap(p, np), nc[p[0]] = 0;
20             for (int i = 1; i < n; i++)
21                 nc[p[i]] = nc[p[i - 1]] +
22                     (c[p[i]] != c[p[i - 1]]
23                      || c[(p[i] + k) % n] != c[(p[i - 1] + k) % n]);
24             swap(c, nc), k <= 1;
25         }
26         for (int i = k = 0; i < n - 1; i++) {
27             int j = p[c[i] - 1];
28             for (; s[i + k] == s[j + k]; k++);
29             if (c[i]) lcp[c[i] - 1] = k;
30             k = max(k - 1, 0);
31         }
32         s.pop_back(), c.pop_back(), n--;
33         p.erase(p.begin()), lcp.erase(lcp.begin());
34         for (auto &i : c) i--;
35     }
36     vector<vector<int>> table;
37     void buildLcp() {
38         int LOG = __lg(n) + 1;
39         table.resize(LOG, vector<int>(n));
40         table[0] = lcp;
41         for (int l = 1; l < LOG; l++) {
42             for (int i = 0; i + (1 << (l - 1)) < n; i++) {
43                 table[l][i] = min(table[l - 1][i], table[l - 1][i + (1 << (l - 1))]);
44             }
45         }
46     }
47     int query(int l, int r) { // 0-based
48         if (l == r) return n - 1;
49         l = c[l], r = c[r];
50         if (l > r) swap(l, r);
51         r--;
52         int len = __lg(r - l + 1);
53         return min(table[len][l], table[len][r - (1 << len) + 1]);
54     }
55     /*
56     * -1 if s[l1...r1] < s[l2...r2]
57     * 0 if s[l1...r1] == s[l2...r2]
58     * 1 if s[l1...r1] > s[l2...r2]
59     */
60     int comp(int l1, int r1, int l2, int r2) {
61         int len1 = r1 - l1 + 1;
62         int len2 = r2 - l2 + 1;
63         int common = query(l1, l2);
64         if (common >= min(len1, len2)) {
65             if (len1 == len2) return 0;
66             return (len1 < len2) ? -1 : 1;
67         }
68         return c[l1] < c[l2] ? -1 : 1;
69     }
70     auto count(string const &t) { // O(log(n) * size(t))
71         int l = int(lower_bound(p.begin(), p.end(), -1, [&](int i, int j) {
72             return s.substr(i, min<size_t>(t.size(), n - i)) < t;
73         }) - p.begin());
74         int r = int(lower_bound(p.begin() + 1, p.end(), -1, [&](int i, int j) {
75             return s.substr(i, min<size_t>(t.size(), n - i)) <= t;
76         }) - p.begin()) - 1;
77         return pair{l, r};
78     }
79 };

```

```

80
81 O(n)
82 struct SuffixArray {
83     int n;
84     string s;
85     vector<int> p, c, lcp;
86     vector<vector<int>> st;
87     SuffixArray(const string &s) : s(s), n((int)s.size()) {
88         if (n == 0) return;
89         vector<int> _s(n + 1);
90         int max_char = 0;
91         for (int i = 0; i < n; i++) {
92             _s[i] = (unsigned char)s[i] + 1;
93             max_char = max(max_char, _s[i]);
94         }
95         _s[n] = 0;
96         vector<int> sa_raw(n + 1);
97         sa_is(_s, sa_raw, max_char + 1);
98         p.assign(sa_raw.begin() + 1, sa_raw.end());
99         c.resize(n);
100         lcp.assign(n - 1, 0);
101         for (int i = 0; i < n; i++) c[p[i]] = i;
102         for (int i = 0, k = 0; i < n; i++) {
103             if (c[i] == n - 1) { k = 0; continue; }
104             int j = p[c[i] + 1];
105             while (i + k < n && j + k < n && _s[i + k] == _s[j + k]) k++;
106             lcp[c[i]] = k;
107             if (k > 0) k--;
108         }
109     }
110     void buildLcp() {
111         if (n > 1) {
112             int LOG = 32 - __builtin_clz(n);
113             st.assign(LOG, vector<int>(n - 1));
114             st[0] = lcp;
115             for (int j = 1; j < LOG; j++) {
116                 for (int i = 0; i + (1 << j) <= n - 1; i++) {
117                     st[j][i] = min(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
118                 }
119             }
120         }
121     }
122     /*
123     * -1 if s[l1...r1] < s[l2...r2]
124     * 0 if s[l1...r1] == s[l2...r2]
125     * 1 if s[l1...r1] > s[l2...r2]
126     */
127     int comp(int l1, int r1, int l2, int r2) {
128         int len1 = r1 - l1 + 1;
129         int len2 = r2 - l2 + 1;
130         int common = query(l1, l2);
131         if (common >= min(len1, len2)) {
132             if (len1 == len2) return 0;
133             return (len1 < len2) ? -1 : 1;
134         }
135         return c[l1] < c[l2] ? -1 : 1;
136     }
137     int query(int i, int j) {
138         if (i == j) return n - i;
139         int l = c[i], r = c[j];
140         if (l > r) swap(l, r);
141         r--;
142         int len = 31 - __builtin_clz(r - l + 1);
143         return min(st[len][l], st[len][r - (1 << len) + 1]);
144     }
145 private:
146     void sa_is(const vector<int> &_s, vector<int> &sa, int alphabet) {
147         int _n = (int)_s.size();
148         vector<bool> type(_n);
149         type[_n - 1] = true;
150         for (int i = _n - 2; i >= 0; i--) {
151             if (_s[i] < _s[i + 1]) type[i] = true;
152             else if (_s[i] > _s[i + 1]) type[i] = false;
153             else type[i] = type[i + 1];
154         }
155         auto isLMS = [&](int i) { return i > 0 && type[i] && !type[i - 1]; };
156         vector<int> bucket(alphabet, 0);
157         for (int x : _s) bucket[x]++;
158         vector<int> head(alphabet), tail(alphabet);
159         for (int i = 0, sum = 0; i < alphabet; i++) {
160             head[i] = sum;

```

```

161         sum += bucket[i];
162         tail[i] = sum - 1;
163     }
164     auto induce = [&](const vector<int> &lms) {
165         fill(sa.begin(), sa.end(), -1);
166         vector<int> cur_tail = tail;
167         for (int i = (int)lms.size() - 1; i >= 0; i--) {
168             sa[cur_tail[_s[lms[i]]]--] = lms[i];
169         }
170         vector<int> cur_head = head;
171         for (int i = 0; i < _n; i++) {
172             if (sa[i] > 0 && !type[sa[i] - 1])
173                 sa[cur_head[_s[sa[i] - 1]]++] = sa[i] - 1;
174         }
175         cur_tail = tail;
176         for (int i = _n - 1; i >= 0; i--) {
177             if (sa[i] > 0 && type[sa[i] - 1])
178                 sa[cur_tail[_s[sa[i] - 1]]--] = sa[i] - 1;
179         }
180     };
181     vector<int> lms_idx;
182     for (int i = 1; i < _n; i++) if (isLMS(i)) lms_idx.push_back(i);
183     induce(lms_idx);
184     vector<int> sorted_lms;
185     for (int x : sa) if (isLMS(x)) sorted_lms.push_back(x);
186     vector<int> rec_s(lms_idx.size());
187     int name = 0;
188     rec_s[0] = 0;
189     for (int i = 1; i < (int)sorted_lms.size(); i++) {
190         int u = sorted_lms[i - 1], v = sorted_lms[i];
191         bool diff = false;
192         for (int d = 0; ; d++) {
193             if (_s[u + d] != _s[v + d] || type[u + d] != type[v + d]) {
194                 diff = true; break;
195             }
196             if (d > 0 && (isLMS(u + d) || isLMS(v + d))) break;
197         }
198         if (diff) name++;
199         auto it = lower_bound(lms_idx.begin(), lms_idx.end(), v);
200         rec_s[distance(lms_idx.begin(), it)] = name;
201     }
202     vector<int> rec_sa(rec_s.size());
203     if (name < (int)rec_s.size() - 1) {
204         sa_is(rec_s, rec_sa, name + 1);
205     } else {
206         for (int i = 0; i < (int)rec_s.size(); i++) rec_sa[rec_s[i]] = i;
207     }
208     vector<int> sorted_lms_final;
209     for (int x : rec_sa) sorted_lms_final.push_back(lms_idx[x]);
210     induce(sorted_lms_final);
211 }
212 };

```

0.7.9 SuffixAutomaton

```

1 Suffix Automaton (SAM)
2
3 state = substring of size from [state.fail.len + 1] to [state.len]
4
5 count of each of them = state.cnt
6
7 struct suffix_automaton {
8     struct state : map<int, int> {
9         int fail = -1, len{}, cnt = 1;
10        bool ed = false;
11    };
12    int lst, n{};
13    vector<state> tr;
14    explicit suffix_automaton(const string& s = ""s) : tr(1), lst(0) {
15        tr.reserve(s.size() * 2);
16        for(auto i : s) add(i);
17    }
18    void add(int c) {
19        int x = int(tr.size());
20        tr.emplace_back(), n++;
21        tr[x].len = tr[lst].len + 1;
22        int p = lst, q;
23        while(~p && (q = tr[p].emplace(c, x).first->second) == x) p = tr[p].fail;
24        if(p == -1) tr[x].fail = 0;
25        else {
26            if(tr[p].len + 1 == tr[q].len) tr[x].fail = q;
27            else {
28                int y = int(tr.size()); tr.emplace_back(tr[q]);
29                tr[y].cnt = 0, tr[y].len = tr[p].len + 1;
30                while(~p && tr[p][c] == q) tr[p][c] = y, p = tr[p].fail;
31                tr[x].fail = tr[q].fail = y;
32            }
33        }
34        lst = x;
35    }
36    void init() { // to build cnt, end
37        for(int p = lst; ~p; tr[p].ed = true, p = tr[p].fail);
38        vector b(n + 1, vector(0, 0));
39        for(int i = 0; i < tr.size(); ++i) b[tr[i].len].push_back(i);
40        for(int l = n; l >= 1; --l)
41            for(int u : b[l])
42                tr[tr[u].fail].cnt += tr[u].cnt;
43    }
44 };
45 Generalized Suffix Automaton
46 struct generalized_sam {
47     struct state : map<int, int> {
48         int fail = -1, len{}, cnt = 0;
49         bool ed = false;
50     };
51     vector<state> tr;
52     int max_len = 0;
53
54     explicit generalized_sam() : tr(1) {}
55
56     int add(int c, int last) {
57         if (tr[last].count(c)) {
58             int q = tr[last][c];
59             if (tr[last].len + 1 == tr[q].len) return q;
60             int clone = int(tr.size());
61             tr.emplace_back(tr[q]);
62             tr[clone].cnt = 0, tr[clone].len = tr[last].len + 1;
63             max_len = max(max_len, tr[clone].len);
64             int p = last;
65             while (~p && tr[p][c] == q) tr[p][c] = clone, p = tr[p].fail;
66             tr[q].fail = clone;
67             return clone;
68         }
69
70         int x = int(tr.size());
71         tr.emplace_back();
72         tr[x].len = tr[last].len + 1;
73         max_len = max(max_len, tr[x].len);
74         int p = last;
75         while (~p && !tr[p].count(c)) tr[p][c] = x, p = tr[p].fail;
76         if (p == -1) tr[x].fail = 0;
77         else {
78             int q = tr[p][c];
79             if (tr[p].len + 1 == tr[q].len) tr[x].fail = q;
80             else {

```

```

81         int y = int(tr.size());
82         tr.emplace_back(tr[q]);
83         tr[y].cnt = 0, tr[y].len = tr[p].len + 1;
84         max_len = max(max_len, tr[y].len);
85
86         while (~p && tr[p][c] == q) tr[p][c] = y, p = tr[p].fail;
87         tr[x].fail = tr[q].fail = y;
88     }
89 }
90 return x;
91 }
92
93 void insert(const string& s) {
94     int last = 0;
95     for (auto i : s) {
96         last = add(i, last);
97         tr[last].cnt++;
98     }
99     tr[last].ed = true;
100 }
101
102 void init() {
103     vector b(max_len + 1, vector(0, 0));
104     for (int i = 0; i < tr.size(); ++i) b[tr[i].len].push_back(i);
105     for (int l = max_len; l >= 1; --l) {
106         for (int u : b[l]) {
107             if (~tr[u].fail) {
108                 tr[tr[u].fail].cnt += tr[u].cnt;
109                 if (tr[u].ed) tr[tr[u].fail].ed = true;
110             }
111         }
112     }
113 }
114 };

```

0.7.10 $Z\backslash_{A}lgo$

```
1 vector<int> z_algo(string s) {
2     vector<int> z(s.size());
3     for(int i = 1, l = 0, r = 0; i < s.size(); i++) {
4         if(i < r) z[i] = min(r - i, z[i - l]);
5         while(i + z[i] < s.size() && s[z[i]] == s[z[i] + i]) z[i]++;
6         if(i + z[i] > r) r = i + z[i], l = i;
7     }
8     return z;
9 }
```

0.8 Trees

0.8.1 Centroid

```
1 struct Centroid {
2     vector<vector<int>> g;
3     vector<int> siz, prevCen;
4     vector<bool> removed;
5     void add(int u, int v) {
6         g[u].emplace_back(v);
7         g[v].emplace_back(u);
8     }
9     int dfs_sz(int u, int p = -1) {
10         siz[u] = 1;
11         for(auto v : g[u]) {
12             if(!removed[v] && v != p) {
13                 siz[u] += dfs_sz(v, u);
14             }
15         }
16         return siz[u];
17     }
18     int calcCentroid(int u) {
19         int p = -1, sz = dfs_sz(u);
20         con:
21         for(auto v : g[u]) {
22             if(!removed[v] && v != p && siz[v] << 1 > sz) {
23                 p = u, u = v;
24                 goto con;
25             }
26         }
27         removed[u] = true;
28         return u;
29     }
30
31     explicit Centroid(int n) : g(n), siz(n), prevCen(n), removed(n), d(n) {}
32     vector<vector<pair<int, int>>> d;
33
34     void dfs(int u, int centroid, int lvl = 1, int p = -1) {
35         d[u].emplace_back(centroid, lvl);
36         for(auto v : g[u]) {
37             if(!removed[v] && v != p) {
38                 dfs(v, centroid, lvl + 1, u);
39             }
40         }
41     }
42
43     void build(int u = 0, int p = -1) {
44         u = calcCentroid(u);
45         prevCen[u] = p;
46         // do something...
47         for(auto v : g[u]) if(!removed[v]) {
48             dfs(v, u);
49         }
50         // ...
51         for(auto v : g[u]) {
52             if(!removed[v]) {
53                 build(v, u);
54             }
55         }
56     }
57 };
```

0.8.2 HLD

```
1 struct tree {
2     int root = 0;
3     vector<vector<int>> g;
4     explicit tree(int n) : g(n) { }
5     void add(int u, int v) {
6         g[u].push_back(v);
7         g[v].push_back(u);
8     }
9
10    vector<int> &operator[](int u) {
11        return g[u];
12    }
13
14    int cntDfs = 0;
15    vector<int> in, out, lvl, sz, top, par, seq;
16
17    void init(int rt = 0) {
18        root = rt;
19        in = out = lvl = top = par = seq = vector<int>(g.size());
20        sz.resize(g.size(), 1);
21        par[root] = top[root] = root;
22        dfs(root);
23        dfs2(root);
24    }
25
26    void dfs(int u) {
27        for(int &v : g[u]) {
28            lvl[v] = lvl[u] + 1;
29            par[v] = u;
30            g[v].erase(find(g[v].begin(), g[v].end(), u));
31            dfs(v);
32            sz[u] += sz[v];
33            if(sz[v] > sz[g[u][0]])
34                swap(v, g[u][0]);
35        }
36    }
37    void dfs2(int u) {
38        in[u] = cntDfs++;
39        seq[in[u]] = u;
40        for(int v : g[u]) {
41            top[v] = v == g[u][0] ? top[u] : v;
42            dfs2(v);
43        }
44        out[u] = cntDfs - 1;
45    }
46
47    int jump(int u, int k) {
48        if(k > lvl[u]) return -1;
49
50        int d = lvl[u] - k;
51        while (lvl[top[u]] > d) {
52            u = par[top[u]];
53        }
54
55        return seq[in[u] - lvl[u] + d];
56    }
57
58    bool isAncestor(int u, int v) {
59        return in[u] <= in[v] && in[v] <= out[u];
60    }
61
62    int lca(int u, int v) {
63        if(lvl[u] > lvl[v]) swap(u, v);
64        if(isAncestor(u, v)) return u;
65        while (top[u] != top[v]) {
66            if (lvl[top[u]] > lvl[top[v]]) {
67                u = par[top[u]];
68            } else {
69                v = par[top[v]];
70            }
71        }
72        return lvl[u] < lvl[v] ? u : v;
73    }
74    auto path_sorted(int u, int v) {
75        vector<tuple<int, int, bool>> U, V; // from, to, reversed
76        while (top[u] != top[v]) {
77            if (lvl[top[u]] > lvl[top[v]]) {
78                U.emplace_back(in[top[u]], in[u], true);
79                u = par[top[u]];
80            } else {
```



```

81         V.emplace_back(in[top[v]], in[v], false);
82         v = par[top[v]];
83     }
84 }
85 if(lvl[u] < lvl[v]) V.emplace_back(in[u], in[v], false);
86 else U.emplace_back(in[v], in[u], true);
87 U.insert(U.end(), V.rbegin(), V.rend());
88 return U;
89 }
90
91 int dis(int u, int v) {
92     return lvl[u] + lvl[v] - 2 * lvl[lca(u, v)];
93 }
94 };

```

0.8.3 LCA

```

1 struct LCA {
2     int n, Log;
3     vector<vector<int>>> up;
4     vector<int> lvl;
5     explicit LCA(vector<vector<int>>> &g, int root = 0) : n((int)g.size()), lvl(n), Log(__lg(n) + 1), up(n,
6     ↪ vector<int>(Log, root)) {
7         function<void(int)> dfs = [&](int u) -> void {
8             for(int v : g[u]) {
9                 if(v == up[u][0]) continue;
10                lvl[v] = lvl[u] + 1, up[v][0] = u;
11                for (int l = 1; l < Log; l++) up[v][l] = up[up[v][l - 1]][l - 1];
12                dfs(v);
13            }
14        };
15        dfs(root);
16    }
17    int k_ancestor(int u, int k) {
18        while(k) u = up[u][__builtin_ctz(k)], k &= k - 1;
19        return u;
20    }
21    int lca(int u, int v) {
22        if(lvl[u] < lvl[v]) swap(u, v);
23        u = k_ancestor(u, lvl[u] - lvl[v]);
24        if(u == v) return u;
25        for(int l = Log - 1; l >= 0; l--)
26            if(up[u][l] ^ up[v][l]) u = up[u][l], v = up[v][l];
27        return up[u][0];
28    }
29    int dis(int u, int v, int l = -1) {
30        if(l == -1) l = lca(u, v);
31        return lvl[u] + lvl[v] - 2 * lvl[l];
32    }
33 };
34
35 template<class T>
36 struct WLCA {
37     int n, Log;
38     vector<vector<int>>> up;
39     vector<vector<T>>> val;
40     vector<int> lvl;
41     explicit WLCA(vector<vector<pair<int, T>>> &g, int root = 0) : n((int)g.size()), lvl(n), Log(__lg(n) + 1), up(n,
42     ↪ vector<int>(Log, root)), val(n, vector<T>(Log)) {
43         function<void(int)> dfs = [&](int u) -> void {
44             for(auto [v, w] : g[u]) {
45                 if(v == up[u][0]) continue;
46                 lvl[v] = lvl[u] + 1, up[v][0] = u, val[v][0] = w;
47                 for(int l = 1; l < Log; l++) {
48                     up[v][l] = up[up[v][l - 1]][l - 1];
49                     val[v][l] = val[v][l - 1] + val[up[v][l - 1]][l - 1];
50                 }
51                 dfs(v);
52            }
53        };
54        dfs(root);
55    }
56    pair<int, T> k_ancestor(int u, int k) {
57        T ans;
58        while(k) {
59            ans = ans + val[u][__builtin_ctz(k)];
60            u = up[u][__builtin_ctz(k)];
61            k &= k - 1;
62        }
63        return {u, ans};
64    }
65    int lca(int u, int v) {
66        if(lvl[u] < lvl[v]) swap(u, v);
67        u = k_ancestor(u, lvl[u] - lvl[v]).first;
68        if(u == v) return u;
69        for(int l = Log - 1; l >= 0; l--) {
70            if(up[u][l] ^ up[v][l]) {
71                u = up[u][l], v = up[v][l];
72            }
73        }
74        return up[u][0];
75    }
76    int dis(int u, int v, int l = -1) {
77        if(l == -1) l = lca(u, v);
78        return lvl[u] + lvl[v] - 2 * lvl[l];
79    }
80 };

```


0.8.4 Link-Cut Tree

```

1  template<typename Info, typename Tag>
2  struct LinkCutTree { // 1-based
3      struct Node {
4          int ch[2]{}, p{}, v_sz{}, sub_sz{1};
5          Info val{}, sum{};
6          Tag lz{};
7          bool rev = false;
8      };
9      vector<Node> t;
10     LinkCutTree(int n, Info init = Info(0)) : t(n + 1) { for(int i = 1; i <= n; i++) t[i].val = t[i].sum = init; }
11     bool isRoot(int x) {
12         int p = t[x].p;
13         return p == 0 || (t[p].ch[0] != x && t[p].ch[1] != x);
14     }
15     void apply(int x, const Tag& lz) {
16         if (!x) return;
17         t[x].val.apply(lz);
18         t[x].sum.apply(lz);
19         t[x].lz.apply(lz);
20     }
21     void push(int x) {
22         if(!x) return;
23         if(t[x].rev) {
24             swap(t[x].ch[0], t[x].ch[1]);
25             if(t[x].ch[0]) t[t[x].ch[0]].rev ^= 1;
26             if(t[x].ch[1]) t[t[x].ch[1]].rev ^= 1;
27             t[x].rev = false;
28         }
29         if(t[x].lz.active()) {
30             if(t[x].ch[0]) apply(t[x].ch[0], t[x].lz);
31             if(t[x].ch[1]) apply(t[x].ch[1], t[x].lz);
32             t[x].lz = Tag();
33         }
34     }
35     void pull(int x) {
36         if(!x) return;
37         t[x].sum = t[x].val;
38         t[x].sub_sz = 1 + t[x].v_sz;
39
40         if(t[x].ch[0]) {
41             t[x].sum = t[t[x].ch[0]].sum + t[x].sum;
42             t[x].sub_sz += t[t[x].ch[0]].sub_sz;
43         }
44         if (t[x].ch[1]) {
45             t[x].sum = t[x].sum + t[t[x].ch[1]].sum;
46             t[x].sub_sz += t[t[x].ch[1]].sub_sz;
47         }
48     }
49     void rotate(int x) {
50         int p = t[x].p, g = t[p].p;
51         push(p), push(x);
52         int dir = t[p].ch[1] == x, b = t[x].ch[dir ^ 1];
53         if (!isRoot(p)) {
54             if (t[g].ch[0] == p) t[g].ch[0] = x;
55             else t[g].ch[1] = x;
56         }
57         t[x].p = g, t[x].ch[dir ^ 1] = p, t[p].p = x, t[p].ch[dir] = b;
58         if(b) t[b].p = p;
59         pull(p), pull(x);
60     }
61     void pushPath(int x) {
62         if(!isRoot(x)) pushPath(t[x].p);
63         push(x);
64     }
65     void splay(int x) {
66         pushPath(x);
67         while (!isRoot(x)) {
68             int p = t[x].p;
69             if(!isRoot(p)) rotate((t[p].ch[0] == x) == (t[t[p].p].ch[0] == p)? p : x);
70             rotate(x);
71         }
72     }
73     int access(int x) {
74         int last = 0;
75         for (int y = x; y; last = y, y = t[y].p) {
76             splay(y);
77             if (t[y].ch[1]) t[y].v_sz += t[t[y].ch[1]].sub_sz;
78             if (last) t[y].v_sz -= t[last].sub_sz;
79             t[y].ch[1] = last;
80             pull(y);

```

```

81     }
82     splay(x);
83     return last;
84 }
85 void makeRoot(int x) { access(x), t[x].rev ^= 1; }
86 int findRoot(int x) {
87     access(x);
88     while (t[x].ch[0]) push(x), x = t[x].ch[0];
89     return splay(x), x;
90 }
91 bool same(int x, int y) { return x == y || findRoot(x) == findRoot(y); }
92 bool link(int x, int y) { // make x root then add edge from y to x. x is child of y
93     makeRoot(x);
94     if (findRoot(y) == x) return false;
95     access(y);
96     t[x].p = y;
97     t[y].v_sz += t[x].sub_sz;
98     pull(y);
99     return true;
100 }
101 bool cut(int x, int y) {
102     makeRoot(x), access(y);
103     if (t[y].ch[0] != x || t[x].ch[1]) return false;
104     t[y].ch[0] = 0, t[x].p = 0;
105     pull(y);
106     return true;
107 }
108 bool cut(int x) { // cut with parent
109     access(x);
110     if (!t[x].ch[0]) return false;
111     t[t[x].ch[0]].p = 0, t[x].ch[0] = 0, pull(x);
112     return true;
113 }
114 void updatePath(int x, int y, const Tag& lz) { makeRoot(x), access(y), apply(y, lz); }
115 Info queryPath(int x, int y) { return makeRoot(x), access(y), t[y].sum; }
116 void setVal(int x, Info v) { access(x), t[x].val = v, pull(x); }
117 int lca(int x, int y) { return findRoot(x) != findRoot(y)? 0: (access(x), access(y)); }
118 int querySubtreeSize(int x, int rt) { return makeRoot(rt), access(x), t[x].v_sz + 1; }
119 int kthNodeOnPath(int x, int y, int k) {
120     makeRoot(x), access(y);
121     if (t[y].sum.sz <= k) return 0;
122     int curr = y;
123     while (curr) {
124         push(curr);
125         int l = t[curr].ch[0], l_sz = l ? t[l].sum.sz : 0;
126         if (k == l_sz) return splay(curr), curr;
127         if (k < l_sz) curr = l;
128         else k -= l_sz + 1, curr = t[curr].ch[1];
129     }
130     return 0;
131 }
132 };
133
134 struct Tag {
135     int64_t add = 0, st = -1;
136     bool active() { return ~st || add != 0; }
137     void apply(const Tag& p) {
138         if (~p.st) st = p.st, add = p.add;
139         else add += p.add;
140     }
141 };
142
143 struct Info {
144     int64_t sum = 0;
145     int sz = 1;
146     Info() : sum(0), sz(0) { }
147     Info(int64_t x) : sum(x), sz(1) {}
148     void apply(const Tag& lz) {
149         if (~lz.st) sum = (int64_t)sz * lz.st;
150         if (lz.add) sum += (int64_t)sz * lz.add;
151     }
152     friend Info operator+(const Info& l, const Info& r) {
153         Info res;
154         res.sum = l.sum + r.sum;
155         res.sz = l.sz + r.sz;
156         return res;
157     }
158 };
159
160
161 powerful link cut tree

```

```

162
163 #include <bits/stdc++.h>
164 using namespace std;
165
166 const int MAX = 2e5 + 5;
167
168 struct Data {
169     int sum, mn, mx, sz;
170     Data() : sum(0), mn(INT_MAX), mx(INT_MIN), sz(0) {}
171     Data(int val) : sum(val), mn(val), mx(val), sz(1) {}
172     Data(int _sum, int _mn, int _mx, int _sz) : sum(_sum), mn(_mn), mx(_mx), sz(_sz) {}
173 };
174 struct Lazy {
175     int a, b;
176     Lazy(int _a = 1, int _b = 0) : a(_a), b(_b) {}
177     bool lazy() {
178         return a != 1 || b != 0;
179     }
180 };
181
182 Data operator+(const Data &a, const Data &b) {
183     return {a.sum + b.sum, min(a.mn, b.mn), max(a.mx, b.mx), a.sz + b.sz};
184 }
185 Data& operator += (Data &a, const Data &b) { return a = a + b; }
186
187 Lazy& operator += (Lazy &a, const Lazy &b) { return a = Lazy(a.a * b.a, a.b * b.a + b.b); }
188
189 int& operator += (int &a, const Lazy &b) { return a = a * b.a + b.b; }
190
191 Data& operator += (Data &a, const Lazy &b) { return a.sz ? a = Data(a.sum * b.a + a.sz * b.b, a.mn * b.a + b.b, a.mx *
↵ b.a + b.b, a.sz) : a; }
192
193 struct Node {
194     int p, ch[4], val;
195     Data path, sub, all;
196     Lazy plazy, slazy;
197     bool flip, fake;
198     Node() : p(0), ch(), path(), sub(), all(), plazy(), slazy(), flip(false), fake(true) {}
199     Node(int _val) : Node() {
200         val = _val;
201         path = all = Data(val);
202         fake = false;
203     }
204 } tr[MAX];
205
206 vector<int> freeList;
207
208 void pushFlip(int u) {
209     if (!u)
210         return;
211     swap(tr[u].ch[0], tr[u].ch[1]);
212     tr[u].flip ^= true;
213 }
214
215 void pushPath(int u, const Lazy &lazy) {
216     if (!u || tr[u].fake)
217         return;
218     tr[u].val += lazy;
219     tr[u].path += lazy;
220     tr[u].all = tr[u].path + tr[u].sub;
221     tr[u].plazy += lazy;
222 }
223
224 void pushSub(int u, bool o, const Lazy &lazy) {
225     if (!u)
226         return;
227     tr[u].sub += lazy;
228     tr[u].slazy += lazy;
229     if (!tr[u].fake && o)
230         pushPath(u, lazy);
231     else
232         tr[u].all = tr[u].path + tr[u].sub;
233 }
234
235 void push(int u) {
236     if (!u)
237         return;
238     if (tr[u].flip) {
239         pushFlip(tr[u].ch[0]);
240         pushFlip(tr[u].ch[1]);
241         tr[u].flip = false;

```

```

242     }
243     if (tr[u].plazy.lazy()) {
244         pushPath(tr[u].ch[0], tr[u].plazy);
245         pushPath(tr[u].ch[1], tr[u].plazy);
246         tr[u].plazy = Lazy();
247     }
248     if (tr[u].slazy.lazy()) {
249         pushSub(tr[u].ch[0], false, tr[u].slazy);
250         pushSub(tr[u].ch[1], false, tr[u].slazy);
251         pushSub(tr[u].ch[2], true, tr[u].slazy);
252         pushSub(tr[u].ch[3], true, tr[u].slazy);
253         tr[u].slazy = Lazy();
254     }
255 }
256
257 void pull(int u) {
258     if (!tr[u].fake)
259         tr[u].path = tr[tr[u].ch[0]].path + tr[tr[u].ch[1]].path + tr[u].val;
260     tr[u].sub = tr[tr[u].ch[0]].sub + tr[tr[u].ch[1]].sub + tr[tr[u].ch[2]].all + tr[tr[u].ch[3]].all;
261     tr[u].all = tr[u].path + tr[u].sub;
262 }
263
264 void attach(int u, int d, int v) {
265     tr[u].ch[d] = v;
266     tr[v].p = u;
267     pull(u);
268 }
269
270 int dir(int u, int o) {
271     int v = tr[u].p;
272     return tr[v].ch[o] == u ? o : tr[v].ch[o+1] == u ? o + 1 : -1;
273 }
274
275 void rotate(int u, int o) {
276     int v = tr[u].p, w = tr[v].p, du = dir(u, o), dv = dir(v, o);
277     if (dv == -1 && o == 0)
278         dv = dir(v, 2);
279     attach(v, du, tr[u].ch[du^1]);
280     attach(u, du ^ 1, v);
281     if (~dv)
282         attach(w, dv, u);
283     else
284         tr[u].p = w;
285 }
286
287 void splay(int u, int o) {
288     push(u);
289     while ('dir(u, o) && (o == 0 || tr[tr[u].p].fake)) {
290         int v = tr[u].p, w = tr[v].p;
291         push(w);
292         push(v);
293         push(u);
294         int du = dir(u, o), dv = dir(v, o);
295         if (~dv && (o == 0 || tr[w].fake))
296             rotate(du == dv ? v : u, o);
297         rotate(u, o);
298     }
299 }
300
301 void add(int u, int v) {
302     if (!v)
303         return;
304     for (int i=2; i<4; i++)
305         if (!tr[u].ch[i]) {
306             attach(u, i, v);
307             return;
308         }
309     int w = freeList.back();
310     freeList.pop_back();
311     attach(w, 2, tr[u].ch[2]);
312     attach(w, 3, v);
313     attach(u, 2, w);
314 }
315
316 void recPush(int u) {
317     if (tr[u].fake)
318         recPush(tr[u].p);
319     push(u);
320 }
321
322 void rem(int u) {

```

```

323     int v = tr[u].p;
324     recPush(v);
325     if (tr[v].fake) {
326         int w = tr[v].p;
327         attach(w, dir(v, 2), tr[v].ch[dir(u, 2) ^ 1]);
328         if (tr[w].fake)
329             splay(w, 2);
330         freeList.push_back(v);
331     } else {
332         attach(v, dir(u, 2), 0);
333     }
334     tr[u].p = 0;
335 }
336
337 int par(int u) {
338     int v = tr[u].p;
339     if (!tr[v].fake)
340         return v;
341     splay(v, 2);
342     return tr[v].p;
343 }
344
345 int access(int u) {
346     int v = u;
347     splay(u, 0);
348     add(u, tr[u].ch[1]);
349     attach(u, 1, 0);
350     while (tr[u].p) {
351         v = par(u);
352         splay(v, 0);
353         rem(u);
354         add(v, tr[v].ch[1]);
355         attach(v, 1, u);
356         splay(u, 0);
357     }
358     return v;
359 }
360
361 void reroot(int u) {
362     access(u);
363     pushFlip(u);
364 }
365
366 void link(int u, int v) {
367     reroot(u);
368     access(v);
369     add(v, u);
370 }
371
372 void cut(int u, int v) {
373     reroot(u);
374     access(v);
375     tr[v].ch[0] = tr[u].p = 0;
376     pull(v);
377 }
378
379 int main() {
380     ios_base::sync_with_stdio(false);
381     cin.tie(NULL);
382
383     int n, m;
384     cin >> n >> m;
385     vector<pair<int, int>> edges;
386     for (int i=0; i<n-1; i++) {
387         int x, y;
388         cin >> x >> y;
389         edges.emplace_back(x, y);
390     }
391
392     for (int i=1; i<=n; i++) {
393         int w;
394         cin >> w;
395         tr[i] = Node(w);
396     }
397     for (int i=n+1; i<MAX; i++)
398         freeList.push_back(i);
399     for (auto [x, y] : edges)
400         link(x, y);
401
402     int rt;
403     cin >> rt;

```



```

404 while (m--) {
405     int k;
406     cin >> k;
407     if (k == 1) { // change root
408         cin >> rt;
409     } else if (k == 3 || k == 4 || k == 11) { // subtree query
410         int x;
411         cin >> x;
412         reroot(rt);
413         access(x);
414         Data ret = tr[x].val;
415         for (int i=2; i<4; i++)
416             ret += tr[tr[x].ch[i]].all;
417         if (k == 3)
418             cout << ret.mn << "\n";
419         else if (k == 4)
420             cout << ret.mx << "\n";
421         else
422             cout << ret.sum << "\n";
423     } else if (k == 0 || k == 5) { // subtree update
424         int x, y;
425         cin >> x >> y;
426         reroot(rt);
427         access(x);
428         Lazy lazy(k == 5, y);
429         tr[x].val += lazy;
430         for (int i=2; i<4; i++)
431             pushSub(tr[x].ch[i], true, lazy);
432     } else if (k == 7 || k == 8 || k == 10) { // path query
433         int x, y;
434         cin >> x >> y;
435         reroot(x);
436         access(y);
437         Data ret = tr[y].path;
438         if (k == 7)
439             cout << ret.mn << "\n";
440         else if (k == 8)
441             cout << ret.mx << "\n";
442         else
443             cout << ret.sum << "\n";
444     } else if (k == 2 || k == 6) { // path update
445         int x, y, z;
446         cin >> x >> y >> z;
447         reroot(x);
448         access(y);
449         pushPath(y, Lazy(k == 6, z));
450     } else { // change parent
451         int x, y;
452         cin >> x >> y;
453         reroot(rt);
454         access(y);
455         if (access(x) != x) {
456             tr[x].ch[0] = tr[tr[x].ch[0]].p = 0;
457             pull(x);
458             access(y);
459             add(y, x);
460         }
461     }
462 }
463
464 return 0;
465 }

```

Index

BCCTree, [25](#)
Berlekamp-Massey, [39](#)
BigNumber, [40](#)
BinaryTrie, [41](#)
Blossom, [26](#)

Centroid, [67](#)
CHT, [16](#)
Circles, [19](#)
combinatorics, [42](#)
Convolutions, [43](#)
Corasick, [57](#)

Data Structure, [2](#)
DominatorTree, [27](#)
DP optimization, [16](#)
DSU, [28](#)
DynamicPersistentSegmentTree, [2](#)
DynamicSegmentTree, [3](#)
DynamicSuffixArray, [58](#)

Eertree, [59](#)
Equation, [44](#)

FenwickTree, [4](#)
FFT, [45](#)
FWHT, [46](#)

Gauss, [47](#)
Generating-Functions, [48](#)
Geometry, [19](#)
GraphArray, [29](#)
Graphs, [25](#)

HelpfulFunctions, [49](#)
HelpFulMethods, [38](#)
HLD, [68](#)

Johnson, [30](#)

KMP, [60](#)

LargestLexSubstring, [61](#)
Lazy-PST, [5](#)
LazySegmentTree, [6](#)
LCA, [69](#)
Lehmer, [50](#)
LiChao, [20](#)
Lines, [21](#)
Link-Cut Tree, [70](#)

Manacher, [62](#)
Matching, [31](#)
Math, [39](#)
Matrices, [51](#)
MaxFlow, [32](#)
MinCostMaxFlow, [33](#)
Mo's, [7](#)

Notes, [22](#), [34](#), [52](#)
NTT, [53](#)

OrderedSTLS, [8](#)

Polygons, [23](#)
polynomial, [54](#)

RollingHash, [63](#)

SegmentTree, [9](#)
SGTBeats, [10](#)
Sieve, [55](#)
SOSDP, [17](#)
Sparse, [11](#)
SqrtDecomposition, [12](#)
Strings, [57](#)
Subset-Sum, [18](#)
SuffixArray, [64](#)
SuffixAutomaton, [65](#)

TarjanSCC, [35](#)
totalArea, [24](#)
Treap, [13](#)
TreeIsomorphism, [36](#)
Trees, [67](#)
TwoSat, [37](#)

WaveletMatrix, [14](#)

XorBasis, [56](#)
XorSegmentTree, [15](#)

Z_Algo, [66](#)